The University is committed to ensuring no student is academically disadvantaged by periods of industrial action. You are invited to outline here how the disruption impacted the development of your project / dissertation. This statement will provide context for markers so that your work can be assessed fairly, with due appreciation of context.

Student Name:	Kamile Kavaliauskaite
Student ID Number:	H00460022
Programme:	Computer Science
Dissertation / Project Title:	**Easy to remember**
Supervisor:	Fuad Marwan

Statement of Impact of the Disruption on Your Project / Dissertation

1. What specific impact did the disruption due to industrial action have on your dissertation / project? Outline the specific disruption and the sections of your work impacted. **Please be factual and concise. *(maximum 100 words)***
-
2. What adjustments have you made to your Project / Dissertation as a result of this disruption? • **Please be factual and concise. *(maximum 100 words)***
-

Student Signature *(type your name):* *Kamile*

Date: *09/04/2026*

Dissertation

by Kamile Kavaliauskaite

Research Report

Submitted in partial fulfilment for the degree of
BSc (Hons) Computer Science

Supervised by Dr./Prof. Fuad Marwan

Heriot-Watt University
School of Mathematical and Computer Sciences

9th April 2026

The copyright in this dissertation is owned by the author. Any quotation from the dissertation or use of any of the information contained in it must acknowledge it as the source of the quotation or information.

AUTHORSHIP DECLARATION

Course code and name	F20PA Research methods and requirements engineering
Type of assessment	Individual
Coursework title	Dissertation
Student name	Kamile Kavaliauskaite
Student ID	H00460022

By signing this form:

- I declare that the work I have submitted for individual assessment or the work I have contributed to a group assessment is entirely my own. I have not taken the ideas, writings or inventions of another person and used them as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements sections.
- I confirm that I have read, understood and followed the University's Regulations on academic misconduct as published on the University's website, and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on Academic Integrity and Plagiarism.

Student signature: First Name Last Name

Date: 20th March 2026

Your work will not be marked if a signed copy of this form is not included with your submission.

ACKNOWLEDGEMENTS

I am thankful for my project supervisor Marwan Fuad for guidance and inspiring meetings. Also, I am thankful to my family, friends and partner for moral support during work on this project.

ABSTRACT

Finding a phone number that appeals to the customer could be a challenging and time-consuming task. The number of available phone numbers could be overwhelmingly large. Furthermore, the concept of what is easy to remember can be quite subjective from customer to customer. Each person has their own combinations of numbers which carry certain personal meaning, like dates, lucky numbers. Because of these two problems, finding the optimal number could take a lot of time.

This project aims to develop a system that automates this process. It identifies and ranks numbers, also license plates, which fall into a category of memorable by matching certain rules to the data available in a dataset. The rules represent certain numerical patterns such as repetition, sequences, symmetry.

This system uses a modular design where each rule functions as an independent pattern detector that contributes to the overall memorability score computation. This way the system is easy to test and implement incrementally.

TABLE OF CONTENTS

Authorship Declaration iii
Acknowledgements iv
Abstract v
Table of Contents vii
1 Introduction 1
2 Background and Motivation 1
2.1 Aim and Objectives 2
2.2 Organisation 2
2.3 Background Research 2
2.3.1 Human memory and numeric patterns 3
2.3.2 Numeric memorability 3
2.3.3 Alphanumeric pattern memorability 4
2.3.4 Natural language processing 4
2.3.5 Similarity metrics and search 5
2.3.6 Rule-based weighted ranking system 6
2.3.7 Temporal-Difference learning 6
2.3.8 Modular multi-agent system 7
3 Methodology and work plan 7
3.1 System workflow for phone number system 8
3.2 Methods 8
3.3 Ranking rules and scoring 9
3.4 Requirements 10
3.4.1 Functional requirements 10
3.4.2 Non-functional requirements 10
3.5 Research goals 11
3.6 Research methods 11
3.7 Stakeholders and use cases 11
3.8 Tasks 12
3.9 Evaluation strategy 12
3.10 Scalability 12
4 Feasibility 13

4.1 Technical feasibility 13
4.2 Resource and time feasibility 13
5 Conclusions 14
5.1 Motivation and goals 14
5.2 Proposed work 14
6 Implementation 15
6.1 System overview 15
6.2 System architecture 16
6.3 User interface layer 16
6.4 System logic layer 17
6.5 Data handling layer 18
6.5.1 Generated datasets 18
6.5.2 Dictionary 19
6.6 Rules 20
6.7 Numeric pattern rules 20
6.8 Natural language inspired rules 21
6.8.1 Word matching rule 22
6.8.2 Leet transformation rule 22
6.8.3 Phonetic rule 23
6.8.4 Edit distance rule 23
6.8.5 Readability rule 23
6.9 Pattern matching rules for input 24
6.10 Ranking 24
6.10.1 Weighted scoring 24
6.10.2 Phone number ranking 24
6.10.3 Two stage ranking 25
6.11 Design decision and optimization 25
6.11.1 Shift to a database from csv only 26
6.11.2 Rule based approach 27
6.11.3 Temporal difference learning 27
6.11.4 Modular architecture 27
6.11.5 Precomputed leet normalized entries in the pre-ranking process 28

6.11.6 Summary of design decisions 28
7 Evaluation 28
7.1 Functional testing 29
7.1.1 Functionalities in the user interface 29
7.1.2 Rule level testing 29
7.1.3 Performance and scalability 30
7.2 Quality of ranking 30
7.2.1 Phone numbers and number plates inputs 31
7.2.2 Phone numbers and number plates suggest similar 31
7.3 Does system meet the requirements 32
7.4 Limitations 33
7.5 Summary of evaluation 33
8 Conclusions 34
8.1 Future ideas 35
References 36
A. Project Management 37
A.1. Gantt Chart 40
A.2. Risk Analysis 40
A.4. Rules 41
A.5. Diagrams 43
B. Generative AI 92

1 Introduction

Automating small tasks that can be time-consuming is a smart decision for any business, and a cost-effective solution. A distinctive phone number or a licence plate is not only useful for a client, making it easier to remember but also for a business to be recognisable and look more professional. However, the process of identifying an ideal number manually is a complex task. Each new number that is picked by a customer can be unique, depending on their personal preferences and needs, making it challenging to select the optimal choice.

Certain numerical patterns can be classed as easy to remember. They tend to adhere to simple rules, such as symmetry, repetition, and sequences. Number combinations such as 50505050 or 123321 stand out from other types of patterns, for example, 481632 (geometric sequence). That is why it is important to implement a ranking system that ensures that selections are optimal.

This project proposes a rule-based modular design system that has predetermined rules, which acts as independent pattern detectors. The combination of such modules evaluates appearing patterns in the numbers structure and contributes to the overall ranking score, which serves as the numbers' evaluation of how memorable the combination is. Optionally, temporal differences learning could be to update the weight of each rule based on user feedback. This design allows the system to be responsive to each user's preferences using dynamic reinforcement.

In this project terms like pattern detector, module, and agent are used interchangeably. The project proposes the artificial intelligence concepts which are optional, so the correct terms for such system components depend on the ai concept inclusion in the system. Also, the report is using term memorability to keep the consistency throughout the report, but that also include the level of interestingness of the pattern.

2 Background and Motivation

The motivation for this project is to develop a system that businesses can use to optimise their daily tasks, making it more profitable and timesaving. The purpose of recognising memorable phone numbers for a business can be influential to advertising and brand consistency. Similarly, individual customers often prefer numbers that are easier to remember or have significant personal value, including certain combinations of numbers resembling important dates, lucky, or angel number formations. Choosing the optimal number from a vast dataset of available ones can be an impossible task without the help of such an automated system.

This project is using principles of pattern detection, and natural language processing. Its modular design enables each rule acts as a separate process. The temporal difference learning is considered as an additional feature to provide adaptability to the system and

could enable the system to update its logic of what is memorable according to the user, with limited feedback.

2.1 Aim and Objectives

The aim of this project is to design a system that can identify and rank memorable phone numbers, or car plates using predetermined rules.
The objectives of this project to achieve this aim are listed below.

1.	Designing a modular rule-based system which can detect patterns in numeric, alphabetic or alphanumeric data.
2.	To integrate lightweight natural language processing methods such as approximate string matching, and similarity search, to determine if they can adequately contribute to memorability score when it comes to number plates.
3.	Develop a ranking system where each pattern detector contributes to an overall memorability score optionally using temporal difference learning on user feedback.

2.2 Organisation

This document is structured into six main sections to define the problem and main objectives, feasibility, and a review of the research methodology required for the completion of the system.

- Section 1 defines the motivation, aims and objectives for the creation of such a system.
- Section 2 provides background research analysis. The first part of this section is cognitive research for defining what is memorable, what the preferences are to refine the vague term of memorability, and to make an educated decision about the rule ranking. Part two of this section overviews system design and algorithmic methods.
- Section 3 describes the system workflow, methods, design, modular architecture, requirements, research methods, tasks, evaluation strategy. It recognises project stakeholders and use cases are overviewed in this section.
- Section 4 is dedicated to project feasibility. It confirms that from technical, resource and time perspectives, the project is feasible.
- Section 5 includes conclusions about this project.

2.3 Background Research

The research that is required for this project consists of cognitive psychology about numeric and alphabetic patterns that are proven to be memorable, modular multi-agent architecture, and natural language processing concepts. This section reviews foundation needed for developing pattern ranking logic.

2.3.1 Human memory and numeric patterns

The important part of this project is to define the specific patterns that are more memorable to people. This includes not only numeric sequencies, but also alphabetic combinations. Also, not every pattern is equally appealing, which raises the problem of the hierarchy of patterns. The decisions on ranking the patterns by their memorability must be made.

2.3.2 Numeric memorability

Research shows that humans prefer structured, meaningful or in some cases familiar patterns in random alphanumerical combinations. The article on memory of numeric security codes of Martin Ramussen and Floyd W Rudmin shows that few of the top strategies for remembering the number sequences are "The next most common mnemonic strategy, reported by 54%, was to remember the pattern the code makes when entered on the keypad. The strategy of using security codes from other contexts was reported by 40%, and choosing a code with a personal meaning was reported by 38%" (Rasmussen and Rudmin, 2010). This is quite an important observation for this project, considering what strategies people use to make it easier to remember number sequences. These results suggest that 54% of their study participants tend to rely on the visual placement of the number sequence in the keypad. Some easy-to-remember sequences following such a strategy could be straight line patterns such as 123, 963, 357. Also, personal meaning is an extremely helpful strategy for recalling numerical patterns, which emphasises the need for user input and choice functionality in the system. Furthermore, it shows that participants relied more on contextual cues rather than randomness.

An article written by Cowan. N and Hardman. K. O. shows that "trains were generally helpful to recall, especially when they fell within a triad but to some extent even when they crossed triad boundaries" (Cowan and Hardman, 2021, p, 744). It supports the rule about repeated number sequences being a top priority when it comes to numerical pattern recalls. Another observation has been done on the repetition of numbers, which indicated high memorability. "There was some evidence that lists with 4 unique digits produced better recall than lists with less repetition and therefore more unique digits" (Cowan and Hardman, 2021, p. 752). It indicates that the least amount of unique numbers improves memorability. Also, this study shows how grouping can emphasise certain numerical patterns, leading to quicker evaluation of appealing combinations.

Furthermore, the Jones, Mari Riess, and June J. Baird's article explores the recall of three numerical patterns, such as symmetrical patterns, arithmetic sequences, and random patterns. "In general, it appears that symmetrical patterns are indeed simpler than patterns with other types of structure… In both serial and free recall modes, sequences with higher-order nominal redundancies were more accurately reconstructed than ones with arithmetic rules or lower-order structure alone" (Jones and Baird, 1979, p. 640).

The findings show that symmetrical patterns take the first place as memorable patterns, followed by simple sequences.

2.3.3 Alphanumeric pattern memorability

Similar concept applies to license plates, which adheres to all the patterns mentioned before, although contains additional complexity. The addition of alphabetic characters to the number combinations raises the problem of applying the natural language processing concept to the system to recognise linguistic and phonetic patterns. Semantic meaning containing letter strings has a superiority when it comes to memorability in comparison to simple patterns adhering to numbers such as symmetry, repetition, or sequencies.

In visual cognition symmetry and shape similarity improves the memorability. So certain letters or letter combinations would be recognized faster. For example, AHA, TOT in comparison to DOD or ERE.

Probably the most important pattern for alphabetical combinations is relatability with semantically meaningful words. Either the letter combination is exact match with a word or having numbers which either phonetically represents parts of words, or numbers physical resemblance to letters. Such combination is still highly recognizable and memorable.

2.3.4 Natural language processing

Since number plate memorability depends on recognizing meaningful words, or phonetic interpretation of numbers, the system must use natural language processing components, which can detect exact words, or distorted versions of words with certain numeric digits representing letters.

Although memorability for alphabetical structures does obey the same rules as numeric patterns, nonetheless semantic meaning is highly important. The combinations of letters that can represent words will require number to letter transformations, approximate matching, also phonetic number and letter values taken in consideration.

To handle such complexity, the system will incorporate character level natural language processing to analyse sequencies letter by letter. This is beneficial to such system, where the number of characters is highly limited, and they mostly contain resemblance to, or shortened versions of real words. With this approach system can compare how closely singular strings which consists of alphabetic and numeric combination resemble actual words, recognising linguistic patterns.

The character level similarity search techniques are applicable to short text analysis, where text distortions can be present. Hybrid lightweight models combining character level and similarity metrics proposed by Wibowo in Lightweight String Similarity Approaches for Duplicate Detection in Academic Titles are closely related to system design proposed in this report. Such models ensure that even alphanumerical combinations can be interpreted as deviation from words with semantic meaning.

The exact dictionary match alone, can miss on close match cases, which can still be appealing to the user. The letter combination can closely resemble words with extra letters or numbers interjecting the combination. For example, CAAR, MOO7N. In this case the approximate string matching would enable the system to detect potential suggestions to the user even with slight deviations from meaningful words.

Another challenge for such system is the phonetic pronunciation of the number. This can be used to resemble a word or even few words with least number of characters. For example: 4ever, me2, 2ring. Many number plates use leet transformations, where numbers are matched to the letters depending on their physical appearance resembling those letters. For example, 8-B, 1-I or L, 5-S. Such substitutions can be treated as character level variation as described in approximate string matching. This enables the wider potential number of car plates to be considered as memorable. Although "the problem of approximate string matching as that of finding the text positions that match a pattern with up to k errors." (Navarro, 2001) Therefore, the system must not consider cases, where the dataset record has too big of a difference from the given example string. Nevertheless, this concept ensures that close match with small deviations cases are not overlooked by the system. In this case approximate string matching is beneficial.

For each of these functionalities such as exact approximate dictionary match, phonetic mapping, and leet substitution, the system will generate the memorability score that indicates the presence of such pattern and its strength. In this case, the appropriate weight of each concept in license plate pattern detection is as important just as phone number memorability weighting. For each type of pattern, the normalized scored must be given, which then can be combined into one memorability score.

The natural language processing enables the system to treat the number plates, not only by their numeric pattern presence, but also by phonetic and semantic meaning.

2.3.5 Similarity metrics and search

Similarity search in this system allows user to get suggestions of similar patterns to their preference. The problem of finding such combinations in the dataset may be problematic, for no exact match being present in the dataset. To overcome this problem the system will use different distance and similarity metrics to suggest the closest matches. These may include vector-based similarity, and character level distance metrics.

2.3.6 Rule-based weighted ranking system

The integration of multiple rules which include numeric, alphabetic and sematic types, creates a need of a unified and sophisticated ranking system with predetermined rules.

The systems general logic consists of rule-based modules responsible for detecting the rules presence in analysed number or car plate and combined into general memorability score. Each module is responsible for singular type of pattern detection, such as symmetry, word similarity, sequencies. This type of system closely resembles the hybrid lightweight frameworks described in Wibowo work. Independent components contribute to united similarity score.

Alphabetic patten detection uses natural language processing methods such as character approximate string matching, dictionary lookup, leet transformations. Dictionary word detection is one of the most important methods to be used for alphabetical pattern detection. The random letter combination which has a semantic meaning is much easier to recall. To detect such letter combinations, the leet speech type of mapping may be applied to potential letter sequencies which can represent the resemblance of certain numbers to letters. For example, see the rules attached in section Appendix . If a word is detected, then the pattern detector dedicates appropriate score, which is suggestive of the strength.

Because each pattern is a modular component, extra rules can be added without modification of the overall system. The separate pattern detectors contribute to the memorability score.

2.3.7 Temporal-Difference learning

Temporal difference learning is considered as an optional feature to upgrade the system into learning from user feedback. The user will most likely make from 3 to 5 asks for new, different number recommendations, before just picking the most appealing one. The more users interact with the system, the easier it is for the system to tailor suggestions, adhering to the users' preference logic. To make the system optimal, the learning functionality must be quick to adapt. For that reason, the learning method to be used for this type of system must be model-free to make adaptive weight adjustments without predefined assumptions of user actions.

In the Sutton and Barto book "Reinforcement learning: an introduction," it is defined as "TD learning is a combination of Monte Carlo ideas and dynamic programming (DP) ideas. Like Monte Carlo methods, TD methods can learn directly from raw experience without a model of the environment's dynamics. Like DP, TD methods update estimates based in part on other learned estimates, without waiting for a final outcome" (Sutton and Barto, 1998, p. 120). This combination is perfect for this project because it allows each user's choice to be treated as a learning event. This way, the system is dynamically

reinforcing its decision on what the memorable number is. Such system is delivering more accurate personalised recommendations with each user's choice.

When personal preferences of what is memorable are considered, the temporal difference learning process would enable the adjustment of the rule hierarchy by incrementally adjusting their weights. This could make the system highly responsive and personalised.

2.3.8 Modular multi-agent system

The system's modular design allows for treating each rule as a separate agent, maintaining its own weight and TD updates. Learning agents collectively determine the overall memorability score by their independent contributions, which are based on individual rules. A box diagram may be seen attached in Appendix.

This type of structure, where multiple agents collaborate towards a common goal, is similar to Kurt-Nelson and Redish's proposed multi-agent architecture. “Each µAgent maintains an independent hypothesis about the state of the world, and a separate value-estimate of taking actions within that hypothesized state. The overall agent thus instantiates a flexible representation of an evolving world-state” (Kurt-Nelson and Redish, 2009, p. 3). This way system achieves modularity, each improves the ranking system logic in comparison to a singular agent for making an overall estimation on the possible weight updates rather than a single global model.

Furthermore, this structure aligns with the multi-agent architecture proposed by Russell and Norvig “Cooperative game theory considers settings in which agents can make binding agreements to form coalitions to cooperate. Solution concepts in cooperative game attempt to formulate which coalitions are stable (the core) and how to fairly divide the value that a coalition obtains (the Shapley value).” (by Russell and Norvig, 2021) All the agent contributions, and their differently weighted combination, make the final memorability score.

By maintaining modularity, the system ensures to be extensible, explainable, adaptable. This supports isolated testing, explainability and scalability of the system.

3 Methodology and work plan

In this section the workplan for this project is outlined and explanation provided of how the researched methods will be used within the system. The proposed system design incorporates the following concepts: rule-based ranking, character level natural language processing, approximate string-matching technique and possibly adaptive learning. This ensures that the system can adequately evaluate the memorability of each item in the dataset and give appropriate suggestions to the user. With these methods, the system will be capable of detecting exact matches and closely related with small deviation matches.

3.1 System workflow for phone number system

The system is designed as a modular workflow and consists of atomic operations. Each operation has a specific ranking and pattern-recognition task. The following workflow is depicted in a cross functional diagram which is attached to Appendix. To start with, the system must load the available phone numbers from a given dataset and calculate a memorability score. The system will calculate the scores for each available phone number using a weighted combination of predefined memorability rules, such as symmetry, repetition, sequences, keyboard patterns, and unique digit count. This way, the user gets the first 5 suggestions of phone numbers with the very best rank.
The user then has three options.

1. Select one of the five suggested phone numbers as theirs.
2. Request similar numbers to one of the five suggested numbers as an example combination. In this case, the cosine similarity method could be applied to find similar number patterns.
3. Input a sequence of numbers with a desirable pattern.

In case the user decides to input their own sequence of numbers, as a priority, the system must try to find an exact match of the pattern present in the available numbers dataset using string comparison. If the exact match does not exist containing the sequence, then the system uses Manhattan distance to determine the closest distance between the suggested and available numbers. Finally, the system makes use of the memorability score to make sure that the number is not only the best match to the user's input but also is one of the highest scoring suggestions for the memorability score, applying weighted ranking.

Optionally, if temporal difference learning is implemented, the system could gradually adjust the weights with each user interaction, to improve the personalised suggestions. The TD error will be calculated to determine how much the expected and observed memorability score differ. Then the TD error will be used by micro agents in the calculation of individual weight updates. The agent takes into consideration of how much the rule contributed to the number, which is set by the score of that feature. The system will use predefined weights if such functionality is not implemented. Finally, the system combines agents' calculations to make a new personalised memorability score and gives the user the best ranking number.

3.2 Methods

To achieve the objectives, the following methods will be used: natural language processing, rule-based ranking, modularity, multi-agent system, and optionally temporal difference learning.

Each rule is an atomic operation that analyses one aspect of the phone number or a license plate structure, adhering to a modular design. This will ensure an easier testing

procedure and data integrity. The rules check the symmetry, repetition, sequences, keyboard patterns, and unique digit count. Each outcome is normalised to a number from 0 to 1 for compatibility. Then the features are combined using a weighted score. This will produce the estimate for a memorability final score for a phone number or a number plate.

The natural language processing methods perform character level analysis of alphanumerical sequencies. The methods related to it will perform string matching, dictionary search, and leet transformations. This enables the system to detect meaningful words despite of alphanumeric combinations containing deviations.

The ranking mechanism in this system could optionally use temporal difference learning to incrementally adjust the weights for each of the rules, making the suggestions more personalised. Without this feature, the system will use predefined weights. Because each rule acts as a separate meta or micro agent, the weight adjustments will be calculated independently and combined by the system into an overall adjusted memorability score.

3.3 Ranking rules and scoring

To rank the numbers as memorable, as mentioned in previous sections, it is important to distinguish the exact rules and hierarchy. At the beginning, the system will not have the users' preferences for what is memorable to them, which is why the ranking is important. If temporal difference learning is implanted, the further in the process of picking the phone number or a car plate, users' picks will implement the system's decisions of what is memorable in this specific case.

The exact rules are provided in Appendix, which are going to be used for initiating the ranking. They are organised into sections of numeric patterns, alphabetical patterns, combination of numeric, and alphabetic, and semantically meaningfulness. Taking into consideration that the cognitive studies analysed in the research section for number combination memorability are not a single study with further defined rules ranked systematically against each other, there is no initial hierarchy of the rules. Memorability may be quite subjective, so ideally suggestions will not be assumed to be the most appealing ones for the user. The user's choices will implement the system to find the most memorable case by comparison and weight. The final score will be a combination of preset logic rules and model user preferences. Each user interaction will feed further model learning.

As micro agents evaluates each numeric or alphabetic feature present in the given phone number or car plate, eventually system will combine numeric pattern scores and natural language processing-based ranking into a single memorability score.

3.4 Requirements

The Requirements for this system are written using the Mosco method. They are ordered into the requirements that must be implemented; otherwise, the system would not be developed to the point where the main objectives could be tested. The requirements ranked as should have defined extra functionalities that such a system would benefit from. The requirements with could have ranking defines the requirements which may be implemented if there is enough time. The requirements ranked as won't are mentioned only for the future development of the project but will not going to be implemented at this stage of the project development.

3.4.1 Functional requirements

FR 1	The system must load a dataset of available phone numbers.
FR 2	The system must detect numerical patterns based on predetermined rules.
FR 3	The system must detect linguistic patterns in number plates using NLP methods such as:
FR 3-1	Dictionary matching.
FR 3-2	Approximate string matching.
FR 3-3	Phonetic similarity.
FR 3-4	Leet transformation.
FR 4	The system must successfully calculate a memorability score.
FR 5	The system must display top ranked suggestions to the user.
FR 6	The system must manage modular rule components, which calculate weight updates that contribute to overall memorability score.
FR 7	The system should enable the user to select one of the suggested phone numbers as the best one.
FR 8	The system should enable user to request for similar suggestions to desired numeric patterns.
FR 9	The system should allow user to enter a custom numeric or alphanumeric pattern.
FR 10	The system should store previous users' choices and inputs.
FR 11	The system could update the ranking weights dynamically based on user feedback using temporal difference learning.
FR 12	The system could provide explanations for the ranking decisions.
FR 13	The system will not work on external dataset integration.
FR 14	The system will not support user authentication.

3.4.2 Non-functional requirements

NFR 1	The system must be able to read at least 1000 phone numbers or car plates from a dataset.
NFR 2	The system must be able to rank at least 1000 phone numbers in under 5 seconds.

NFR 3	The system must show adequate suggestions of phone numbers or car plates, clearly adhering to memorability rules.
NFR 4	The system should be able to work with larger datasets, for example 10 000 phone numbers, performing under 10 seconds.
NFR 5	The system should provide error messages.
NFR 6	The system could allow the user to select a preference for which rule of memorability adheres the most.
NFR 7	The system could show the rationality analysis of why the numbers were selected and suggest to the user.
NFR 8	The system won’t collect any personal data.
NFR 9	The system won't have a mobile version application.

3.5 Research goals

The research aims to answer the following questions.

Can a modular rule-based architecture effectively separate alphabetic, and numeric pattern detection into independent pattern detectors? (Ob1)
Can lightweight natural language processing methods be successfully integrated to evaluate the memorability of alphanumeric number plates? (Ob2)
Can a rule-based weighted system optionally using the TD learning method create a reliable memorability ranking? (OB3)

3.6 Research methods

For the research, which was needed for this project, the Heriot-Watt University Discovery website was used. Only the sources that support the method decision was used. The deductive reasoning method was used with a top-down approach. After defining the main deliverables of the system, such as learning from user data, finding similarity, and defining the memorability, I have started narrowing down by methods that are not applicable for this project or do not align with the predefined workflow.

After main deliverables were defined, the research continued by narrowing down choices for methods. Some techniques, such as supervised learning, clustering were researched but named as unsuitable for such project due to not aligning well with modularity, small user feedback, or timeframe. The methods that have been chosen, aligned well with system workflow. This deducted reasoning approach ensures that the project is technically feasible.

3.7 Stakeholders and use cases

This system's main stakeholder is the telecommunications business. A system as such would automate the process of picking the phone numbers, optimising the finding of the most distinctive, appealing, and memorable phone number for the user. The use case for

this type of system could be in the shop while working with clients in real-time, also integrated into an online application for ordering a new phone number online, making the process more accessible to the user.

3.8 Tasks

This project is following Agile methodology, which perfectly aligns with the systems modular design. Iterative implementation and testing align well with this methodology, so each task can be set as a 2-week sprints for first semester and 1-week sprints for the second semester. There will be concurrent work related to implementation and testing of the system, also the dissertation writing. The project plan mostly separated in weekly sprint style can be seen attached in Appendix.

3.9 Evaluation strategy

The project evaluation will consist of two parts: computational evaluation and functional behaviour validation.

In computational evaluation, the main focus will be the non-Functional requirements and efficiency. The system will be tested against bigger datasets including 10000 phone numbers, or number plates. This is needed to verify that the system can cope with large amounts of data and will perform well in real life conditions. Also, rule evaluation, and natural language processing pattern detection will be inspected as separate module of the system. This adheres to Objective 1, to confirm that each rule can operate independently, added or removed from the overall system without impact.

For objective 2, the systems ability to detect meaningful patterns in alphanumerical character combinations will be evaluated. This will reveal if the system can match distorted letter combinations to real world words. This includes evaluating performance of approximate string-matching algorithms. This ensures reliability across all modules present in the system and protects the memorability score from distortion.

If temporal difference learning is implemented, adaptability evaluation will ensure that temporal difference learning mechanism work effectively for objective 3. If not implemented, evaluation will focus on predefined rule weights and their influence on suggestions. The weight adjustments will be recorded, to inspect if rule weights adjusted adequately and logically.

3.10 Scalability

This system may be scaled in respect of adding new rules, which can easily be added as separate individual modules with minimal adjustment to overall system.

For future enhancement semantic embedding could be implemented to the system. That would ensure that if the user input certain words, the system would be capable of finding synonyms.

This section outlined the project plan and system workflow, evaluation strategy. The feasibility is given in the next section 4.

4 Feasibility

This section presents the technical and resources feasibility. The system design aims to be lightweight. The modularity enables the scalability and diversion to different kind of system with minimal loss of changes and resources. This project is achievable within the given time scale.

4.1 Technical feasibility

The system design is technically feasible. The system will be built using the Python language, making use of libraries such as Pandas, NumPy. The ranking system is designed in a lightweight way, using micro-agents, which will allow the system to rank and compare a large amount of data in real time. The system is using a modular architecture, keeping data integrity, enabling optimal testing and scalability.

The temporal difference learning algorithm is considered as an optional feature to ensure that the system is responsive to users' preferences without requiring excessive feedback. If implemented, each micro-agent could contribute to the overall memorability score. Such system would be model-free. This approach ensures that the system could be lightweight.

Overall, the algorithms required for such system include rule-based pattern detection, natural language processing methods and temporal difference learning, which can be elegant and straightforward to implement.

4.2 Resource and time feasibility

This project will use open-source software, personal computer, and synthetic datasets; therefore, it will be cost-effective. No external APIs, cloud infrastructure, commercial datasets, or user information required.
The modular design aligns with the semester timeline, allowing the system to be implemented incrementally. See the Gantt chart in section in Appendix.

This section proved, that the proposed system is not only feasible, but also scalable, due to the modular lightweight design. See conclusions in the next section 5.

5 Conclusions

This section is dedicated for conclusions. The first part of conclusions is overviewing motivation, and goals define why such system is useful. And the section Proposed work aims to overview the overall project and possibility for scalability.

5.1 Motivation and goals

This project proposal is an overview of the development of the system that automates the process of finding an easy-to-remember phone numbers or car plates. It takes into consideration users' cognitive preferences, by implementing appropriate computational algorithms. The aim is to determine if a system which is capable of identifying and ranking memorable phone numbers can be developed using predetermined rules.

The proposed system uses rule-based pattern matching, and natural language processing concepts. The cognitive research helped to identify main pattern features and gain insight in people preferences when it comes to memorability. The temporal difference learning may be implemented as optional enhancement to improve the personalization and overcome the rigidness of the predefined importance of rule weights with limited user feedback.

5.2 Proposed work

The design of the project is feasible within the project's timeline and available resources. The Agile methodology used for project planning and management ensures that modular components can be implemented, improved, and tested independently. The Agile methodology ensures that the development process is iterative. This enables efficient development and ease of incremental integration of new functionalities.

In addition, this project is structured with consideration of future scalability. The modular systems design could be upgraded to similar systems which deals with numerical, alphabetical or combination of both data, with minimal changes. The possible improvements with bigger dictionary integration and more advanced methods such as semantic embedding for synonym search.

In conclusion, this project combines cognitive research, and natural language processing concepts to develop a system which is feasible, lightweight, and optimal.

6 Implementation

This chapter describes the design and implementation of the proposed system, which recommends a memorable phone number or car plate from a dataset. This system was developed using modular design. The main concepts used to create such application are rule-based scoring, use of a database, string matching, leet normalization, and a user interface. The implementation chapter first walks through overall architecture of the system, then software layers, rule design, ranking strategy.

6.1System overview

The system was developed in a modular style using classes for each new object or rule. Its aim is to rank and choose the most memorable either phone numbers or number plates which are given in a dataset. The system is combining rule-based ranking and lightweight natural language processing methods for scoring and choosing each item.

The proposition of this project was looking into creating two similar in concept, but different in complexity operations. This is the reason, for the requirement to split the system into two separate operations as phone number and number plates pickers. The workflow of these two systems is similar. First the system loads the data from a csv file and recomputes the data into the database for easier use. During this stage, the system sends to the database new fields with scores, which are later used in final ranking. It increases efficiency. For the car plates, it includes leet normalization. This ensures that after preprocessing, the system contains more valuable candidates for later chooses. For example, plates containing valid words, which contains numbers symbolizing the letters are included, such as: B00K, M00N.

The application was designed in a way that combines personalized search and partially autonomous memorability ranking. Although the system has preset assumptions of what supposed to be a highly scoring suggestion when memorability is concerned, the user may say with their actions the systems priorities on certain type of patterns, letter combinations or meaningful words. This makes system a more advances recommendation tool.

The dual applications' modes, which are dedicated to a phone number picker and car plates picker, have the same workflow, but differ in complexity. The phone number picker finds candidates for suggestions while taking into consideration only numerical structures and their patterns such as repeated digits, sequences, lack of unique numbers, symmetry. While on the other hand car plate picker combines, not only the previously mentioned numerical structure, but also the alphabetic and linguistic structures. This introduces the higher complexity and need of heavier filtering and ranking logic. The decision to keep the number plate and phone number pickers separate, not only adhered to the goal of keeping the system modular but also helped to keep the implementation manageable. Although, both modes share similar architecture, the plate mode requires more complex implementation, using semantics and leet transformations.

6.2System architecture

The system has a layered architecture, which includes user interface, application logic, rule processing, and data handling. This type of architecture ensures that the system is adhering to modularity principles and separates each component to its individual functionality. This enables easier testing, and scalability, maintenance, and further implementation.

The cross functional flow chart, which was given in the project proposal has been updated and refined. It can be found in Appendix. The updated chart can be found in the appendix section. It is illustrating the interactions between all four previously mentioned layers. This visual representation of the system decision process shows, how the layers interact, the data movements in between them, and how candidates scoring and ranking is done.

The user starts the system by choosing the mode. That triggers the system to get pre-processed data, with relevant features which were created in the process of the database production. The user is greeted with best ranking candidates, which are not necessarily are aligning with what the user would consider as desirable memorable suggestion. Then user can request of the system to give different suggestions by entering a custom pattern, or the ask for similar suggestion to a given one. The system interprets input, gets adequate candidates from the database, applies matching and ranking logic, and returns either a, exact, or a close match.

Such separation and at the same time cooperation between layers and functions of the system, enables to reduce complexity and ease the testing and troubleshooting. Each layer only performs the functions that are appropriate to them. The user interface handles user interactions, the application logic layer dictates search, ranking and suggestion updates, rule layer evaluates features which are related to the memorability, and the data layer is responsible for getting and storing pre-processed data. This also improves systems scalability. The data flows from the user interface to the application logic, which gets candidate subsets from the database, applies ranking and matching logic using the rule outputs and brings them back to the user. Therefore, the application can be further expanded into additional user focused rules could be added, without redesigning application logic. This ensures that the system is capable to handle future implementations.

6.3User interface layer

User interface layer handles the interaction with the user. It was implemented using Python Tkinter library. The main purpose of it, is interactions with the user, such as inputs, suggestions, error and success messages. In this layer user selects which mode of

the system will be used: phone number, or car plates picker. User actions are passed to the application logic, which manages the ranking and return of adequate results. The design of the systems front end is kept lightweight, to not overcomplicate the users' interactions with the system, which adheres to usability. Although there is much space for improvement.

The interface is simple and minimalistic. User is greeted with a start screen and two options to proceed to either the phone number picking or the car plate picking system side. Either way the user chooses, the system takes them to the suggestion screen for either of functionalities. There are five suggestions waiting for the user. The action buttons allow user to pick one of the given candidates, ask for a similar suggestion, or input a desired pattern in an input field.

After user chooses to either get similar suggestions to one of the given ones, or search the desired pattern, the system updates the suggestions to different ones, so that they more match the input.
The system contains input validation, which ensures that the inputs for phone numbers only contains digits, between 2 and 12 digits adhering to the UK phone number count. For the number plates. It accepts more than 2 and less than 8 alphanumeric characters. Also, the system converts any alphabetic character input into a capital letter. The error messages warn the user about invalid input because of the number of characters or the restricted symbols.

The user interface also displays small explanations underneath each of the suggestion, indicating, which of the rules, or patterns contributed the most. This improves the systems transparency on recommendation decisions.

6.4System logic layer

The logic layer is the centre of the application. It is responsible for connecting the user interface to the rules and the database. This layer decides how user actions affect the ranking process and further suggestions. The logic is split in two modes: PhoneMode and PlateMode classes. Both modes have similar workflow but differ in complexity. While the PhoneMode considers only numeric memorability, the PlateMode combines alphanumeric patterns. In this layer the records are loaded from the database, makes decisions which type of ranking to perform, then applies rule combinations accordingly, and returns suggestions to the interface.

In the logic layer, the mode for phone numbers deals with initial ranking of memorable numbers, or search for numeric candidates. Exact match is a highest of priority, and approximate matches follows to keep the number of 5 suggestions.

In the number plate mode, the logic is a bit more complex. The system still supports either suggest similar, or pattern input interactions. For the input, users typed in pattern

is kept, so it is not reduced too much to lose its original idea or meaning. For the suggest similar functionality the chosen item is reduced to smaller letter clusters, to find semantically meaningful words easier.

The plate mode gets the candidate subsets from the database using raw plate values, digits, letters, and leet normalized transformations. The rule scoring is applied, and highest candidates are returned to the user. After one of the candidates are chosen by the user, the candidates' availability is updated and related caches deleted, so it is not available to be chosen again.

6.5Data handling layer

The requirements for this system specify that csv files should be read as a dataset for the system. The start of the development of the application begins with reading the csv files with no help of database functionality. Due to low amount of data entries, initially the system managed to cope with handling data in a reasonable time. One of the important functionalities for the system is its scalability. With increasing amount of data entries, the system had to be upgraded to using SQLite database and caching. This immensely improved the systems capabilities of dealing with millions of data points and optimized the scoring. In the beginning of the systems development, the data was loading all records into python objects, computes all the features in the memory. Although it was working for smaller datasets, it was not aligning with scalability concept. The systems design was later updated into loading csv files to a SQLite tables. That enabled the system to adhere more to scalability. It improved the query retrieval times and capability with dealing with bigger datasets. At the query time, the system only takes a subset of candidates, which get more advanced scoring. This way the database stores scored features of each item, which improves its responsiveness.

In the final stage of implementation, the csv files are required only to build the databases, which are mainly used of the system. The SQLite table for phone number includes: the raw value, digit only form, availability, individual numeric type of rule scores, and a base score. The car plate table includes fields such as raw plate, digit part, letter part, leet normalized form, availability, numeric type rule scores, and a base score.

The process of the database building process is also optimized with inserting records in batches.

6.5.1 Generated datasets

The data is synthetically generated by plates_generator and number_generator. The phone number generator creates a random combination of numbers which start in 07, all together of length 11. The car plates generator considers not only the contemporary new

style number plates (2001 - present), but also prefix (1983 - 2001), suffix (1963 - 1983), and dateless (1963 and older). Each of the style have their own weights, to simulate the quantity and rarity. The records in the csv files only depicts the raw value as a phone number, or a number plate, and its availability as a binary string.
After the system reads the csv file data, it performs feature extraction, and stores structured, and updated version of the data to the database. For the number plates, in the preprocessing, the digit only, letter only values, leet normalized forms, and precomputed scores are separated.
For the car plates, the database table stores raw plate, digit part, letter part, leet normalized representation, the availability state, and scores for symmetry, repetition, sequence, alternation, and a base score. The number plates have some restrictions for character use, such as: O is not allowed, because of visual similarity to a digit 0. This type of data storing enables queries for alphabetical queries such as book to be retrieved in variations like b00k, because similar looking letters are transformed into digits.
Furthermore, the extracted features are then combined into a base memorability score, which is used as a ranking metric at the beginning. The preprocessing in such way enables the system to be more responsive, ensure the systems scalability, and adheres to requirement of suggesting under 5 seconds.

After user chooses one of the options, entry gets updated in the dataset as taken. This ensures that the dataset is up to date and does not repeat itself.

Here is a table showing the fields in the databases for phone numbers and plate numbers. Also, their structure is depicted in the screenshots attached in Appendix.

Table	Fields	Purpose
Phone_numbers	raw, digits, availability, symmetry, repetition, sequence_score, alternating, base_score	Stores precomputed phone number features
Plate_numbers	Raw, digits, letters, leet_normalized, availability, symmetry, repetition, sequence_score, alternating, base_score	Stores precomputed car plate features

6.5.2 Dictionary

For processing number plates and finding meaningful words, which are often much more desirable by users, the system is using a dictionary, which has Creative Commons Attribution-ShareAlike 4.0 International License open license for copy, transformation, and remix irrevocably. It was sourced from Kagle. The dictionary cleaner was developed, to only keep the relevant words to the system. It cleans the dataset from longer than 7 and shorter than 2 characters words, also excludes any words including any other character than a letter. This refined dictionary is then used by the system for

finding meaningful combinations of letters, which either identically match, or has a similar structure.

The system also stores all the user actions into a json file for testing future TD learning implementation purposes.

6.6Rules

The rule processing layer is responsible for setting mechanisms of how the pattern can be detected in the phone number or car plate. That feeds into overall memorability score rule by rule. This layer adheres to the systems modular design and contains each rule as a separate class. The main goal of each rule is to evaluate and give a score on a one specific feature in the candidate. Each individual rule class inherits the same structure from abstract Rule class. It ensures, that the structure is consistent. Also, any new rules can be added without a need to edit the already existing rules.

All the rules in the system can be divided into three main categories such as: numeric pattern rules, natural language-based rules, and user input matching rules.

The system is build using a modular design, which contains rules that influence the memorability score. This way a specific pattern can be evaluated and combined into finding the best candidate for top suggestions.

The advantage of the rule processing layer is that it enables individual features to be tested separately. Each rule is created as a separate class with a standardised structure. This allows the testing of each rule individually, to ensure that it performs as expected. Most of the rules are overlapping in the computation process and otherwise the detection of single rule not working becomes a much bigger and complex task.

This type of rule architecture improves the understanding of how the exact computation is carried out, because each rule, which is applied in order to get the suggestion can give the rules name. For example, a suggested output may have the indications of what features were primarily used in order to explain the rationale, such as: symmetry, dictionary match, or repeated digits. These rule-based structure adheres to explainability, and it ensured the systems transparency about which rule was dominant in the decision-making process for giving certain recommendations.

6.7Numeric pattern rules

The numeric pattern rules analyse digit-based structures. The exact patterns that system analyse adheres to a symmetry, repetition, sequence, alternating and lack of unique digits. These rules are lightweight and perform on digit sequences which are present in both, phone numbers and car plates.

The symmetry rule detects the numeric combinations in which numbers appear in a sequential pattern and ends mirroring itself backwards. In order to detect such number combinations, the digits get compared at opposite positions in such sequence. The outermost digits must move towards the centre of the combination. Such sequences can appear in length n, so then the rule checks if the number i matches the digit on the other side, which index is n – i – 1. The score is produced by calculating the proportion of matching pairs. The longer the sequence, the bigger the score. The value produced out of the score gets normalized between 0 and 1. Such pattern makes it easier to remember than irregular combinations.

The repetition rule identifies how many often digits repeat within the combinations. This rule is taking into consideration the possibility of using chunking to its benefit. Not only it indicates if repeated digits appear but also recognizes the length of such sequence. Repeated items heavily reduce the complexity.

Sequence rule identifies if the given collection of digits proceeds in the manner of an ordered sequence. Such patterns have a predictable structure, which improves the memorability. It is often easily spotted by human eye; therefore this rule is a valuable addition and contributor to the top ranked results.

The alternating digit rule captures the digit combinations with patterns such as 1313, or 5050. The rule checks if the digit which is two positions further match the checked one and evaluates the length of such sequence. The rule produces the score, which reflects the strength of the pattern by its length. This type of rhythmic pattern captures the digit combinations, which not necessarily adhere to the given repetition rule, but can be evaluated as a repetition by the user.

The unique digits rule captures the lower count of unique digits. In case, the number combination would not contain an easy to spot pattern as such, it still may be a better candidate for a higher memorability. The digit repetition throughout the combination raises a higher chance of making other types of number combinations, which are not identified by previously mentioned rules.

6.8Natural language inspired rules

For car plates only, additional rules were introduced. They enable the system to recognize meaningful word patterns against a dictionary, user inputs, and similarity comparisons. These rules include use natural language inspired methods. The word matching rule enables the system to recognize meaningful words from combinations of letters, comparing them to the preset dictionary words. The leet transformations has digits mapping to letters using the physical similarity of the characters, for example 8 – B. This expands the available suggestions for number plates, even if exact match is not found. Also, the system is using a phonetic rule, which transforms certain the digits to phonetic combinations of letters. Finally, the system uses character matching based on

Levenshtein similarity in order to find strings with least possible differences. It is suitable for short strings, by capturing minimal character transformations.

The system is capable of dealing not only with exact matches, but also approximate matching. The exact match search in dataset uses substrings. While the approximate matching uses Levenshtein character level similarity. This way the system is capable of returning the suggestions which are closest to the user's query.

6.8.1 Word matching rule

The word matching rule detects if the alphabetic part of the car plate has exact or an approximate matching word in the dictionary. Such find, indicates, that the character combination is a meaningful word. The implementation of this rule gets the normalized dictionary entries, which then can be compared against the candidate strings. All the letter combinations are converted to upper case and combined into letter clusters which keep the order but skip the unused by leet transformation digits. This rule is one of the most important rules in capturing memorability in car plates.

The rule deals with different levels of matching. The exact match, where the candidate string exactly matches a dictionary word. The substring on a string can be also matched to a valid word. Approximate matching in this rule is based on normalised string similarity. Each variation of this rule carries different weight, indicating their importance against the other two. This rule adheres to previously overviewed research in this work, where semantically meaningful words improve memorability.

6.8.2 Leet transformation rule

The leet transformation rule enables the interpretation of numerical values as alphabetic characters, which visually are similar. This rule also supports the letter to digit conversions and generates alternative search items. This type of functionality expands the possibilities for more number plates to contain a meaningful word and can act in a similar manner as previously mentioned word rule. In the final stages of systems implementation, leet transformation rule is applied to e database preprocessing. For each plate, the normalized leet transformations are stored in order to expand available pool for later state of scoring.

Due to limitations of available number plates, the task of picking one which has all the needed alphabetic characters, in some cases can become close to impossible. To counter this problem leet transformation rule opens more possibilities to find a good candidate. This rule contains the mapping of digits to letters and replaces the digits to their matching letters. After the replacement is performed, substring extraction and comparison to dictionary words may be done. This rule enables such letter combinations as "B00K" or "8AD" to be detected, despite of their non alphabetical character containment. At the

same time, the leet transformation rule does detect patterns within some cases more numerical values, than alphabetical, there fore some suggestions could be less obvious of why it supposed to be a memorable or attention worthy car plate. For example, “54D”, or “84D” is less desirable than “D4D”, “8UD”. The rule also works from letters to digits direction. This allows for word like search but gives equivalents which could be read as the same word. The weight is set as moderate, to ensure that this rule does not take the priority over others. This rule allows leet transformation related alphanumeric combinations to be interpreted by the system as similar forms of the word.

6.8.3 Phonetic rule

The phonetic rule identifies the candidate strings which adhere to phonetic words written like a letter. This rule similarly to leet transformation rule contains mapping of certain numbers corresponding to words. This rule transforms the string and applies exact and approximate substring matches using the similarity metrics. This rule allows to recognize such combinations as “ME2” or “B8”. Although this rule can give suggestions carrying even more ambiguity than the leet transformations, so the weight is handled accordingly. In the final state of implementation this rule was given lower influence, so that it does not act as one of the dominant rules.

6.8.4 Edit distance rule

The edit distance rule is measuring the distance between two strings. This captures the cases where the most minimal changes would be made to have the string still recognizable. It is using levenshtein distance metric which measures minimal character changes. This rule computes the normalized similarity score which indicates distance between candidate and target strings.
This rule enables the cases where neither leet transformation nor phonetic rule scores their feature high. It is very useful for tiny differences detection, but the similarity threshold is applied to avoid unrecognisable results.
In the plate mode, this rule is in particularly useful when similar item is requested by the user, to find near word alternatives.

6.8.5 Readability rule

The readability rule is responsible for improving the ranking of the visually more readable plates. It is responsible for tracking the minimal number of transitions needed between letters and digits. Although a single or couple letter interpretability can be clear, the more the more alterations between digits and letters, the harder is to capture meaningful transformed letter combination. This rule raises the score for the plates, which contained bigger clusters of alphabetical characters. In the end the suggested plates appear less clustered and easier to interpret at first sight.

6.9 Pattern matching rules for input

The rules which are comparing the items against the user input, enable personalized search functionality. The raw substring rule finds identical substring of users input in the candidate. This rule takes the lead in the score compared to others. It enables the user to find exact lucky numbers, a name or a birthday present. It is important to stress such occurrences, to not get them diluted by combinations of other patterns appearing in the candidates.

In order to produce appropriate suggestions for the users' inputs, the other rules are used. The user letter rule concentrates on letter clusters, the leet rule works with letter to digit transformations in words and edit distance rule finds the least number of changes required alternatives if no exact match was found. The leet normalized database field. enables the system to get the candidates such as B00K with user queries likes BOOK.

The final design ensures, that a user is enabled to find exact matches for their queries, but also possible close alternatives as suggestions.

6.10 Ranking

This section analyses the ranking engine in the system. It includes the weighted scoring, and the overview of two stage scoring different system modes.

6.10.1 Weighted scoring

The final memorability score is calculated, by using weighted combination of rule outputs. Each rule contributes a score using predefined weights, which then gets multiplied by a set weight. The final score is the sum of weighted normalized scores. In this ranking system, weight plays a crucial role in emphasis on the importance of certain rules over others. For example, in car plates, the meaningful words present are much more desirable than numeric patters. The weights were chosen based on this logic and previously discussed research in users' preferences when it comes to memorability, then adjusted in the process of testing the expected systems performance.

6.10.2 Phone number ranking

In the phone mode, ranking is using pre-computed features which are stored in the database. Each candidate suggestion contains the weighted sum of evaluated features by the following rules: symmetry, repetition, sequence score, unique digits score, alternating score. Each features contribution to the final score is summed up in row_score() function with equal weights.

When no user pattern is given, the system retrieves available phone numbers ordered by base_score from the database and ranks them using precomputed features. When the system gets an input from the user, the priority is to get the exact match from the database. If there are less than 5 suggestions, the system searches for a close match using

similarity on the digits, which are same in input and candidate. Those candidates are ordered using similarity and then memorability score.

This way the priority stays with the exact matches, and at the same time relevant suggestions can fill up the space if needed.

6.10.3 Two stage ranking

The number plates ranking computation includes two stages: candidate retrieval and final ranking. On the first stage, the database is used to get candidate subset based on their letters, digits, base score, and leet normalizations. On the second stage, the candidates are rescored using more computationally heavy rules, such as approximate word matching, leet matching, raw substring matching, edit distance, phonetic similarity, readability, and memorable pattern bonus. The different patterns weights ensure, that some of them affect the final score more than others. This way the linguistic based rules outweighs the numeric ones and the final results are better balanced. This type of two stage ranking reduces the computations time, because they are applied to a smaller number of candidates.

In the first stage, the search pool is reduced, in order to limit the expensive further computations. When a user enters a pattern as an input, the system increases importance for using pattern matching rules. When no pattern is provided by the user, the system is using preset memorability related rules. On the second stage a more complex rules applied.

For the number plates, the memorability part of the score is a sum of weighted features as such:

- Base score – 0.20
- Word score – 0.40
- Readability – 0.28
- Memorable pattern bonus – 0.12

When a user query is active, the final score sums:

- Pattern match – 0.55
- Memorability – 0.45

The pattern match is the candidate's ability to match the given query. This ensures that the user queries are prioritised in the ranking, while memorability still contributes to the result.

6.11 Design decision and optimization

At the beginning or the systems development, the datasets were reasonably small in number of entries. The system had no problem of coping with them. Although in order to develop the application, which is as close as possible represents dealing with real life

scenarios, the increase of the entry number was needed. From 1 000, it was raised to 100 000, and eventually to 1 000 000 entries. This required the additional optimization.

To improve the performance, caching, database processing, use of index for the database, and two-level ranking were implemented. During the database construction, the system preprocess csv records and their rule related features and stores them in SQLite tables. For the phone number table those features include raw value, digit only form, availability, individual numeric rule scores and a base score. For the number plates table, the fields include raw value, digit only form, letter only form, leet normalized form, availability, numeric rule scores, base score.

To use the SQLite functionalities to its fullest, the indexes were implemented for faster candidate retrieval. In the phone number database, indexes cleated on digits, base_score and availability. In the number plate database, on digits, letters, leet normalized form, base score and availability straight after an entries insertion. Python list comprehensions were used in parts of implementation in order to keep the code concise. For syntax lookups the python official documentation, SQLite documentation, and Tinter documentation was used.

If same query is repeated, the results are brought back, without a need of extra computation. Both modes use cache to avoid that. Additionally, a rule such as edits distance and phonetic matching, which are heavy on computation, are applied only on a subset of candidates from the database. That improved the time for computation related to number plates. On further stages of implementation, the leet transformed searchable normalized field was introduced to the database. This immensely improved the suggestion quality and improved the query execution time.

To develop this system, several key decisions had to be made, to achieve its main functionalities. Such as data storage management, the architecture of the system, the ranking logic. Due to factors such as efficiency and optimization some minor changes have been made.

6.11.1 Shift to a database from csv only

The process of development brought some challenges, which were not fully thought through at the beginning of designing and requirement setting stage. At the finalization of the systems development, although the minor components were changed, the system is achieving its' goals and functionalities. The things that have not been considered is the size of the dataset, that the system will need to be capable of dealing in the real-world environment. The move to SQLite enabled the system to store extra fields with precomputed scores and leet normalized forms, which was inefficient to be calculated for each query at the beginning stages of development. Measures such as caching, internal database implementation and preprocessing before the user even starts the full interaction to the system, had to be implemented, to achieve the nonfunctional

requirement of fitting the user query time under a set time. This not only improved the systems responsiveness time but also improved the system to be more robust against bigger size datasets.

6.11.2 Rule based approach

A rule-based approach was chosen, because memorability, can be easily broken down into alphanumeric pattern rules, and indications of the most contributing rules send to the user interface as explanation. The modular design of the system allows easy addition for any extra rules for future implementation if needed, without affecting the existing ones.

Also, some rules were dropped in the development process, which were suggested in the project proposal. The reason for that is a lack of time, and also the system reaches its goals at a decent level with the rules used. The additional rules may be the future implementations, which further refine the systems sensitivity on picking up the perfect candidates for the user.

6.11.3 Temporal difference learning

The temporal difference learning was not applied to the system at this stage. This enhancement was considered at the project proposal as an additional functionality, which could enrich the user experience. Supposedly, it should update the feature weights, taking into consideration the user preference gathered from their queries. This way the system would gradually learn and adapts to what the user considers memorable. One of the reasons why it was excluded out of implementation, is because the adaptive learning requires more interactions with the system, for it to benefit from such implementation. Although, for future implementations, the system leaves a possibility for such upgrade if needed. Now the application does save users choices, which could be used as a learning data.

6.11.4 Modular architecture

The modular architecture, which is using classes for each systems component is maintained throughout the implementation. These components include objects, modes, rules, repositories and user interface components. This enabled the system to have clear boundaries in-between each of the functionality. The modular architecture also supported later systems updates mentioned in the previous sections and enabled the separation between user input search and the systems behaviours for suggesting similar candidates for the plate mode.

6.11.5 Precomputed leet normalized entries in the pre-ranking process

During the later stage of development of a number plate mode, the leet precomputed normalized field was added to the database. It is responsible for storing conversions of visually similar digit characters into alphabetical characters. This enables the system to make some additional variations of plates to be read as a potential candidate for queries as inputs with a meaningful word. It not only improved the time taken for the query to be executed, but also improved the results, making the candidates way more relevant to the queries.

6.11.6 Summary of design decisions

This chapter explained the phone number / car plates picking systems design, workflow, and key decisions. The system has a Tkinter based interface, SQLite database, set of scoring rules interpreting possible suggestions, and ranking logic engine. The finalized systems design deals with memorability ranking, personalized input search, and suggestion generation, while producing adequate suggestions. The number plate search functionality is strengthened with additional language related rules, and leet normalized preprocessing to get the best possible suggestions.

The next chapter evaluates the systems capabilities to perform the before mentioned functionalities in practice. The evaluation is considering correctness, responsiveness, and quality of the recommendations.

7 Evaluation

This chapter evaluates the final system performance and ranking quality. The testing proceeded in the further sections in order to confirm if the project requirements are met and if modular rule-based design is a good approach for such system. The evaluation is done on user interface functions, response times of the system on different size database, and investigation on harder cases of queries.

Testing for this system was done with 10 000 000 dataset entries. The evaluation is done in four parts. Firstly, the system is evaluated if it can perform the tasks that it should. The second section of testing discusses how systems responsiveness times vary depending on the size of the database. The third section is summarizing how successfully the system complies with requirements and research goals. And lastly, the ranking quality is tested by observing the systems suggestions to queries as more obvious and more difficult cases.

7.1Functional testing

In this section evaluates if the system performs according to the project requirements. The tests cover dataset loading, suggestions, suggestions generator, similar suggestions picker, input validation, candidate suggestions state updates.

The approach for testing is heavily reliant on the project's requirements. Each test case reveals weather the output matches with an expected result. The proof of outcome for the testing of each component will be depicted in screenshots, which are referenced in appendix.

7.1.1 Functionalities in the user interface

The outcome and proof screenshots of the testing for this section is stored in the Appendix. It contains the proceeded test id, the feature being tested, the expected outcome, actual outcome, pass or fail of the test and a proof of the test results. The table describes a feature, what kid of input does the system take, the expected and actual results.
After the testing was proceeded, the conclusion can be drawn that all the buttons are working properly, the state of the items chosen by user updates successfully and error messages are given to the user for unexpected inputs.

7.1.2 Rule level testing

The following testing outcome with proofs are stored in a table, which can be found attached to the appendix. The table depicts the rule testing through the user interface. This testing is aiming at recognizing that visually the rule was applied and the suggestions are adequate. Although this testing table does not prove the mathematic correctness, but it shows if rule behaviour can be recognized in suggestions. This is highly important to be observed, in order to prove if the rules working as intended and visibly can preserve the approximate patterns of the given words.

The functional resting showed that the final system can successfully perform the tasks given. The system successfully loads the data, has predefined rules, which do help to detect patterns and find proper candidates. The system also is using a dictionary, so some of the suggestions contain meaningful words. The system is following the modular design. The approximate string matching is using similarity-based comparison and Levenshtein character level matching. The leet transformation is also used, to ensure, that some of the meaningful words would be still recognized and used as options despite of certain letters not apparent on the number plates. The system does successfully calculate the memorability score which contains of weighted rule scores. The user does det top ranked suggestions, can get some more similar suggestions, input a query themselves, and is enabled to pick one phone number, or car plate. The users' chosen

items are recorded by the system in a json file. Also with each candidate, the system is giving a hint which rule was the most important in making the decision to pick it.

7.1.3 Performance and scalability

The system was tested for the performance time with different sizes of databases. The outcomes of this testing were recorded in a table which can be found attached to the Appendix section of this work. The table includes time measurements for 1 000, 100 000, 1 000 000, and 10 000 000 data entries. The requirements were specifying only 5 s time for queries retrieval on a 1 000 entries csv file. At the first stages of implementation, the system started taking too long for even simple tasks on 10 000 data entries. This rose the problem for unexpected optimization such as caching, database use, two level ranking, leet normalization forms storing. The testing was proceeded with much bigger dataset in order to see how well the system can adhere to scalability at this stage. This testing is especially important for the number plates mode, because of the more complex ranking procedure, including more computationally expensive tasks, which does take more time. The purpose for this testing is recognizing if upgrade to making use of databases, caching, subset of candidates from the database, split in two stages of computation for fetching the queries can ensure that the system can be scalable to much larger datasets.

The results of the performance testing shown that the system is capable of dealing with a bigger dataset than anticipated and is responsive across all the database sizes chosen. As expected, the phone number ranking does perform quicker than number plate picker, because of the simpler logic. Although number plate picker does perform slower, it is still within the preset response times for its' actions. This testing approved that the optimization did improve the systems scalability.

7.2 Quality of ranking

This section overlooks the evaluation of the quality of the returned suggestions. Although functional testing showed that the system works, it does not necessarily is taking the more complicates queries in consideration, which would be less clear, with mixed rules, to evaluate if the system produces answers, which would be actually useful. It is not only tests, how robustly the system deals with user inputs, but also finds its' boundaries and weaknesses, which will be discussed more in debt in the limitations chapter.

To reflex the systems dual input handling such as inputs and request of similar, the testing results are sectioned in two parts. The first one is assessing how the system is dealing with the users' custom input, taking in consideration the full query, then as closely related suggestions as possible. The second section overwrites the results of the testing of a request to find similar patterns to already given suggestions. The main difference is that for the similar suggestions functionality, there are no exact matches,

and the system must observe an object on its own, without searching a matching one in the database, to avoid duplication. The suggestions therefore are to be expected to be vaguer, but as an attempt of the system to do a calculated decision, in what could appeal as memorable preserving the given item.

The testing evaluates the quality of scoring by three parameters with a given scoring in observation, such as: query preservation, memorability relevance, and plausibility. The scores are given to each to indicate the achievement in these three qualities: 0 for poor, 1 for partial, 2 for strong. The total score might vary from 0 to 6. This type of testing practically shows of how well some inter-rule queries are handled by the system. In some cases, the weaker scoring does indicate that the database might be lacking appropriate candidates. The table is produced with the test results and attached to the appendix section.

7.2.1 Phone numbers and number plates inputs

The results for the direct user inputs, reveals, that this type of user query performs well. It is to be expected, that the direct inputs would find a more memorable looking suggestions in comparison to the suggested similar requests. It already indicates to the system what type of suggestions the user is looking for and prefer. Quite often the inputs would appear for this side of the system shorter than max length of suggestions, leaving space for more variations to the possible suggestions.

In phone number picker, the user given patterns are preserved well, and the system seemingly takes in consideration the users pattern repetition, also the same digit use in other characters or sequence appearance. The system manages to find the exact user given number combinations as exact matches.

The number plates, also perform strongly, taking into account the number plates structure, the suggestions with interruptions of numbers in a word still considered as a perfect match candidate. The queries involving short meaningful words are performed well and preserves the user given patterns. Although the character transformation to digit rule, did not perform that well. This might indicate the ranking limitations, such as lack of available candidates.

7.2.2 Phone numbers and number plates suggest similar

The suggest similar functionality also produced good testing results in general. It produced visually adequate suggestions. It visually preserved apparent patterns.

For the phone number picker, the visually present patterns in the phone number were preserved if present and prominent, such as repetition, same digit use, sequence, or

numeric clusters, leaving only small number of digit variating from the initial phone number.

For the number plates picker, the strongest results were, when the system was given plates containing the words having semantic meaning. The system was able to detect the same word appearing in different positions in the plate, and also a character of two change away similar looking words also containing semantic meaning. Although, the lowest scoring tests were related to the number plates suggest similar functionality, which did not include semantic meaning words, because it made harder to define for this type of picker of what could be similar. Although the system then suggested the number related rules to take higher importance.

Overall, the ranking quality is acceptable, with most queries giving adequate suggestions, which adhere to the predefined rules in the system. Although the system is less reliable with queries which appear more ambiguous, or longer phonetic and semantic combinations.

7.3Does system meet the requirements

The system meets the requirements which were proposed in project proposal and research part of this document. The only requirement that was not achieves is the FR11 on td learning, which ended up out of scope for the final system. This functionality is left for future implementation. The system had few upgrades which were not specified in the functional requirements such as: preprocessing and storing csv entries in a database. The application does load csv files successfully, detect alphanumeric patterns, and applies lightweight natural language processing style rules for number plates. Also, the system is using a dictionary to find semantically meaningful words, phonetic matching and leet transformations. The system does calculate the memorability score using weighted rule calculations and then display the top scoring suggestions to the user with the hints of which rule is the leading one. The user does have options to request similar looking suggestions to the given ones or input a custom query. The user choices are stored in the json file. Rules being separated in their own classes adhering to modularity concept.

The system also satisfies the non-functional requirements. It can read bigger datasets, than anticipated at the initial design of the system. Also, error messages are provided to the user indicating invalid inputs. Also, no personal data is collected, keeping the system as lightweight desktop application as it was stated in nonfunctional requirements. Overall, the system does adhere to all but one requirement listed as must of should.

The evaluation shows that the research goals and main objectives were achieved. The modular design for this system successfully separates each functionality into individual components, which adheres to the objective 1. Furthermore, the lightweight natural language processing methods is contributing to overall alphanumeric memorability in

the system. This supports the objective 2. The temporal difference learning was not implemented to the system, because the fixed weight ranking proved to be sufficient enough for giving adequate suggestions to the user.

7.4Limitations

The limitations for this system needs to be taken in consideration, in order to evaluate the systems performance.

The first limitation is that memorability is subjective. Although the system does produce clear structure of rules and their importance which could explain it, there is no unified scientifically approved measurement for such concept. Each user will understand the memorability from their personal perspective. Due to the psychological research done in this work in order to grasp the main patterns applicable to most people, the rules such as similarity, sequence, meaningful words were implemented.

The second limitation is related to the synthetically generated datasets, and a filtered limited, not updated since dictionary. This is suitable for this stage of such systems development, but most likely does not fully represent real world evolving language or personally meaningful not globally used new words, foreign countries number plates, or phone numbers.

Finally, another limitation is the fine line, what each user would consider a readable similar suggestion, which could variate greatly depending on their knowledge in common language rules and trends, and personal experiences.

7.5Summary of evaluation

The evaluation showed that the system is capable to perform to its initial purpose. The functional part of testing confirmed that the application can load the data, do the necessary computations, provide the error messages, display rule base explanations for its decision making to the user, and generate top suggestions.

The performance testing showed that the system responsive across the tested dataset sizes. The optimization of the system with two stage ranking, caching, indexing, preprocessing, made it to be more scalable. The phone number mode performed slightly faster than plate mode, because of simpler ranking logic.

The ranking quality testing showed that the system is performing best, when the user inputs clear patterns adhering to the rules for giving suggestions. Although more complex, or longer patterns in number plates are more difficult to handle.

Overall, the system satisfies the functional and non-functional requirements, with only exception the TD learning, which remained out of the projects scope.

8 Conclusions

This project is created to evaluate if a modular rule-based system is a good design for recommending memorable phone numbers and number plates from large alphanumerical datasets. This project is based on research exploring memorability as a concept, and a modular weighted rule-based ranking, and some natural language processing inspired methods. The memorability concept in the research was discovered to be highly related to semantic meaning or digit patterns. This highly influenced the decision in what rules should be included in the final system, and prioritization of some rules in comparison to others.

The final system ended up separated in two modes: phone number picker, and car plate picker. The phone number picker focuses on recognizing the numerical patterns and offering the user the closest matches to their queries. The car plate picker follows similar logic, but it expands on the alphabetic and semantic processing. The system is developed using modular design with separated classes for rules, modes, repositories and user interface components. The biggest pivot in the system, is the optimization, where databases were introduction of SQLite preprocessing of data in ranking first stage to the system. This improved immensely the systems performance and ensured that the system is more scalable than the initial version.

The evaluation of the system showed that all but one requirement were achieved. The TD learning ended up staying out of this projects scope. Despite that, the system is still capable of suggesting the candidates which most of the time either contain the exact alphanumerical combination given as input or at least give close suggestions. The system successfully updates the picked item as taken, to ensure the unique plate cannot be chosen twice. Also, the testing revealed, that the system best performs on direct user queries containing clear numerical patterns or letter clusters that contain semantic meaning which can be matched to the dictionary. The system performs better than anticipated because of mid-development optimisation.

The project answered main research objectives. Firstly, the modular rule-based design did successfully separate functionalities as independent components. Secondly, lightweight natural language processing methods did enable the system to be capable of dealing with alphanumeric data. And lastly, the weighted ranking ended up being sufficient enough to make decent suggestions even without the TD learning implementation, which ended up outside of the scope of the project. Some limitations were revealed at the development of this project, which were related to the things which are out of scope at this stage. Such as, personal priority towards what could appeal memorable, synthetic set datasets, the readability aspect of the suggestions where a technically correct candidates are not perceived by everyone as easy to memorise. These

limitations indicate the possible future development attention. At this stage the system is more of a guided recommendation tool, rather than perfected prediction of users' preferences.

Overall, this project proved that modular rule-based design is feasible for a recommendation system for picking phone numbers and car plates. The methods of precomputing, database support, pattern rules enabled the system to adhere to robustness and scalability. The final outcome achieved the main aims of the project.

8.1Future ideas

For the future development of this system there are few improvements that can be made. Due to memorability factor depending on personalized preferences, it would be a good idea for the system to be more personalized and adjust to each user, by analysing the previously used queries and adapting the weights accordingly. Td learning might improve memorability further and make the weight adjustments more personalized.

Another possible improvement would be implementation of countries selection, so that the system would be capable of dealing with any length of phone numbers and number plates. Also, for the number plates, a more advanced phonetic matching and more refined semantic system would give better suggestions. And a user testing could open up new useful implementations.

Finally, the system could be expanded into a mobile and web-based applications. Because of the modularity of the system, additions like extra rules, or preference control could be implemented to the system without a need of redesigning the system.

References

[1] Cowan, N. and Hardman, K.O. (2021) *Immediate recall of serial numbers with or without multiple item repetitions, Memory (Hove)*, 29(6), pp. 744–761. Available at: https://doi.org/10.1080/09658211.2021.1942920 (Accessed: 09/11/2025)

[2] Jones, M. R, and Baird J. J. (1979*) Memory for Symmetry: Real or Artifact, The American Journal of Psychology*, vol. 92, no. 4, pp. 627–51. Available at: https://doi.org/10.2307/1421793 (Accessed 09/11/2025)

[3] Kurth-Nelson, Z. and Redish, A.D. (2009) 'Temporal-Difference Reinforcement Learning with Distributed Representations', *Plows one*, 4(10), pp. e7362–e7362. Available at: https://doi.org/10.1371/journal.pone.0007362 (Accessed: 11/11/2025)

[4] Levy, A., Shalom, B.R. and Chalamish, M. (2025) *A guide to similarity measures and their data science applications, Journal of Big Data*, 12(1), pp. 188–57. Available at: https://doi.org/10.1186/s40537-025-01227-1 (Accessed: 09/11/2025)

[5] Navarro, G. (2001) 'A guided tour to approximate string matching', *ACM computing surveys*, 33(1), pp. 31–88. Available at: https://doi.org/10.1145/375360.375365. (Accessed 02/11/2005)

[6] Nezahatkk(2024) *10.000 English words CERF labelled*. [Dataset] 4(3), pp. 416–426. Available at: https://www.kaggle.com/datasets/nezahatkk/10-000-english-words-cerf-labelled/data (Accessed 08/14/2026)

[7] Python Software Foundation. *Python 3.14.4 documentation*. Available at: https://docs.python.org/3/ (Accessed 08/04/2026)

[8] Python Software Foundation. *Sqlite3 – DB-API 2.0 interface for SQLite databases*. Available at: https://docs.python.org/3/library/sqlite3.html (Accessed 08/04/2026)

[9] Python Software Foundation. *Tkinter – Python interface to Tcl/Tk*. Available at: https://docs.python.org/3/library/tkinter.html (Accessed 08/04/2026)

[10] Rasmussen, M. and Rudmin, F. W. 2010. *The coming PIN code epidemic: A survey study of memory of numeric security codes. Available at:* https://munin.uit.no/bitstream/handle/10037/3031/article.pdf?isAllowed=y&sequence=1 (Accessed: 09/11/2025)

[11] Russell, S, & Norvig, (2016), *Artificial Intelligence: a Modern Approach*, Global Edition: A Modern Approach, Pearson Education, Limited, Harlow. Available from: https://rudyct.com/ai/Artificial%20Intelligence%20A%20Modern%20Approach,%203rd%20Edition%20by%20Stuart%20J.%20Russell,%20Peter%20Norvig-2016.pdf . (12 November 2025).

[12] SQLite *Documentation* Available at: https://www.sqlite.org/docs.html (Accessed: 08/04/2026)

[13] Sutton, R.S. and Barto, A.G. (1998) *Reinforcement learning: an introduction*. Available at: https://muse.jhu.edu/book/60836 (Accessed 09/11/2025)

[14] Viappiani, P. (2020) *Robust winner determination in positional scoring rules with uncertain weights, Theory and decision*, 88(3), pp. 323–367. Available at: https://doi.org/10.1007/s11238-019-09734-3 (Accessed: 09/11/2025)

[15] Wibowo, F.M., Nafan, M.Z., Gustalika, M.A., Fernando, H., Hussain, M. and Sahadun, N.A. (2025) 'Lightweight String Similarity Approaches for Duplicate Detection in Academic Titles', *Journal of Informatics and Web Engineering*, 4(3), pp. 416–426. Available at: https://doi.org/10.33093/jiwe.2025.4.3.25. (Accessed 09/11/2025)

A. Project Management

For the project management, an Agile approach will be used. It aligns well with the project's modular system design, enabling iterative implementation in short sprints. This type of project management style is flexible and adheres to the need of working with continuous updates, depending on constant feedback. This enables rapid, constant improvements. Each sprint will focus on the development of a handful of operations and testing it.

For tracking the progress, and keeping the meeting minutes with the project supervisor, Notion is used for this project. The OneDrive folder is set up for keeping back up versions of any documentation, diagrams, or code needed for this project. The GitHub repository set, and the access is given to this project supervisor.

Here is the link to project log in Notion https://www.notion.so/Honours-Project-board-283630eded408024a682d3ef169d4f1f?source=copy_link

Here is the link to GitHub
https://github.com/KamKava/easy_to_remenber/blob/main/README.md

Semester 1 – Research and design

Weeks 1 – 2 Topic picking and refinement

- Defining research questions
- Identify project scope
- Meeting the project supervisor
- Aims objectives and deliverables

Week 3 – 7 Literature review

- Cognitive memorability research
- Patterns in random number combination
- Similarity metrics
- Adaptive learning systems
- Write findings into a project proposal / research document

Week 8 Requirements and rules

- Functional and Non-functional requirements
- System workflow
- Determine scoring system and rules

Week 9 Methodology and work plan section of project proposal

- Feasibility
- Define stakeholders and use cases
- Methodology section
- Research natural language processing concepts

Week 10 – Work on report

- Feasibility video
- Finalise report

Week 11 Final submissions

- Ensure the project proposal is up to standard
- Incorporate supervisor feedback
- Submit the final project proposal and feasibility video

Semester 2 – dissertation and system development

Week 1 – Setup and data preparation

- Preparation of datasets that are going to be used for this system.
- Defining testing strategy and develop the project structure.

Weeks 2 – 3 Core rule implementation

- Implement the predefined rules for numeric patterns to the system
- Begin development of ranking logic

Week 4 – Ranking and weighted score system

- Implement the predefined rules for alphabetic patterns to the system
- Ensure compatibility between rules
- Add dictionary matching and leet pattern handling

Week 5 – Ranking and matching

- Integrate the approximate string matching
- Implement weighted scores logic
- Tune parameters

Week 6 – Optimization and improvements

- SQLite database, caching, indexing and precomputed features functionalities
- Ensure that the system works seamlessly with all the components is scalable and responsive

Week 7 – Interface development

- Create screens, buttons, input fields
- Error messages for wrong inputs

Week 8 – Testing

- Testing all the components
- Fining and logging the errors
- Parameter adjustments if needed
-

Week 9 – Dissertation writing

- Analysis of test and runs
- Comparison of expected performance and actual outcomes

Week 10 – evaluation

- Recognise what are strengths and weaknesses of the system
- Finalise dissertation with findings
- Create graphs
- Write implementation section

Week 11 – Final writing and editing

- Format dissertation and ensure that it adheres to standards
- Submit work
- Recognise what are strengths and weaknesses of the system
- Finalise dissertation with findings

Week 12 – Submission and Q and A

- Format dissertation and ensure that it adheres to standards
- Submit work

- Prepare a presentation

A.1. Gantt Chart

The Gantt Chart is designed using Microsoft excel. This chart depicts the tasks set for each week.

Task	Start Offset	Duration	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16	W17	W18	W19	W20	W21	W22	W23
Topic picking and refinement	0	2																							
Literature review	2	5																							
Requirements and rules	7	1																							
Methodology and work plan	8	1																							
Work on report	9	1																							
Final proposal submissions	10	1																							
Setup and data preparation	11	1																							
Core rule implementation	12	2																							
Ranking and weighted score system	14	1																							
Ranking and matching	15	1																							
Optimisation and improvements	16	1																							
Interface development	17	1																							
Testing	18	1																							
Dissertation writing	19	1																							
Evaluation	20	1																							
Final writing and editing	21	1																							
Submission and Q&A	22	1																							
	Blue bars show active weeks for each task.																								

Figure 1: Gantt chart

A.2. Risk Analysis

Here are some possible risks that may occur during the project's development. Also, on the right side of the table, there are according to mitigations.

	Risks	Likelihood	Impact	Mitigations
1.	Difficulty implementing TD learning.	Medium	High	Ensuring the weight adjustment operations work predictably before integrating user feedback.
2.	Attempting to implement non-essential features before core functionalities are finalized.	High	Medium	Consult the project plan for ensuring that prioritised features are the ones that are being implemented.
3.	Faults in distance logic implementation.	Low	Medium	Testing each distance metric's functionality after implementation.
4.	The ranking operation is too slow for a larger dataset.	Medium	Medium	Identify bottlenecks and prioritize optimization.

5.	Overlapping of the rules.	Low	Medium	Redesign the rules in a way that avoids overlapping.
6.	Dataset quality	Low	Medium	Clean and process the dataset and ensure that missing values are handled.
7.	Struggle to implement all the modules	High	Medium	Integrate incrementally, keeping in mind prioritization.

A.4. Rules

Here are the predetermined rules proposed for the system.
The numeric rules.

	Rule	Explanation	Example
1.	Symmetry	Number sequences that can be read the same from left to right and from right to left. 1 indicates the longest symmetrical sequence and 0 the absence.	2345432, 12321, 1221
2.	Single-digit repetition	Repeated single or collections of digits in a row. It indicates the times number repeats. 10 indicates all numbers repeating, and 0 indicates no repetition present.	222, 344566, 455455
3.	Numerical pattern repetition	Repeated numerical patterns such as pairs or triplets.	233233, 2525
4.	Periodicity	Repeated pattern every few steps.	75767 780781
5.	Sequences	Number sequences. 1 indicates all the single digits, and 0 indicates the absence of sequences in the phone number.	5678, 456
6.	Simple keyboard patterns	Combinations of numbers that make simple rows on the keyboard counter. From max 2 patterns to 0.	123, 456, 789, 741, 852, 963, 159, 753, 357, 951, 8520, 0258.
7.	Unique number count	Unique numbers are present in the phone number. 0 indicates all the unique numbers included in the number, and 1 indicates only one number repeated.	12322123, 45445444
8.	Chunking	Measures the other pattern appearance in certain segments of the number.	555-333, 6969-111

Alphabetic rules

	Rule	Explanation	Example
9.	Letter repetition	Repeated letters in the number plate.	AAA, BB
10.	Letter sequence	Letter sequences appearance.	ABC, EFD
11.	Letter palindromes	Symmetry in letter appearance.	ALA, GYG
12.	Letter symmetry	Visually symmetric letters	IHI, UMU

Alphanumeric rules

	Rule	Explanation	Example
13.	Digit to letter	Digits that look like letters.	H3LLO, C4T
14.	Combination repetition	Combination of a letter and number is repeated.	P1P1, R2R2
15.	Combined symmetry	Symmetry with letter and digits combination.	A22A, B8B
16.	Chunking	Standard plate structure adherence.	ABC 321, AB6 777

Semantic rules

	Rule	Explanation	Example
17.	Match with semantic meaning	Matches the letter combinations to semantically valid words.	DOG, SKY
18.	Match with semantic meaning with digit to letter	Matches the letter and digits like letters combinations to semantically valid words.	B1N, 5KY
19.	Phonetic formation	Matches certain numbers and letters to their phonetic pronunciation in combination with letters.	B4, GR8

Implemented rules summary table

This table shows what the rules apply to and what purpose do they serve.

Rule	Applies to	Purpose
Symmetry_rule	Phone number, car plate	Detect mirrored structure
Repetition_rule	Phone number, car plate	Detect repeated characters
Sequence_rule	Phone number, car plate	Detect ordered numeric runs
Alternation_rule	Phone number, car plate	Detect repeated numeric clusters
Unique_digit_rule	Phone number	Reward low variation in digits
Word_match_dict_rule	Car plate	Detect meaningful words
Leet_rule	Car plate	Make digit letter transformations
Phonetic_rule	Car plate	Detect phonetic similarity on numbers
Edit_distance_rule	Car plate	Detect approximate string similarity
Raw_substring_rule	Car plate for input	Detect perfect match
Numeric pattern	Phone number, car plate input	Detect exact or partial numeric input match
Readability_rule	Car plate only	Reward shorter and more readable plates
User_letters_rule	Car plate only	Extracts letter clusters

A.5. Diagrams

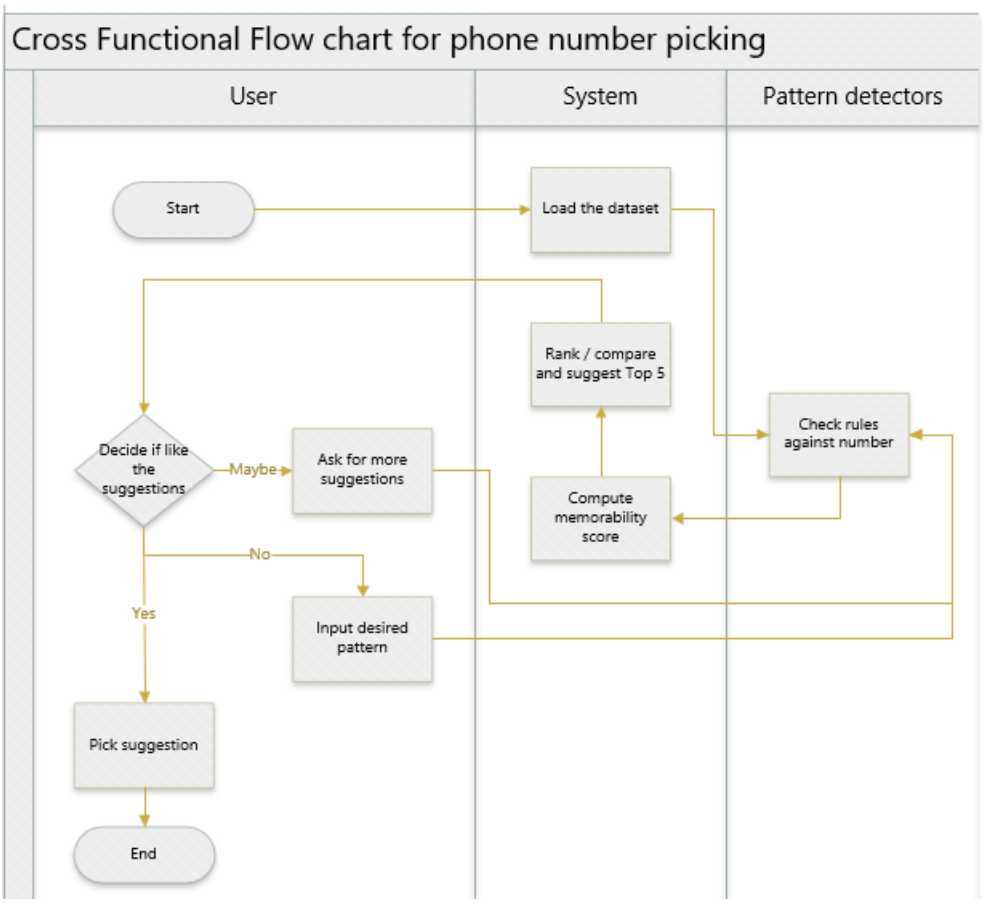

Figure 2: Cross functional flow chart for phone num picker.

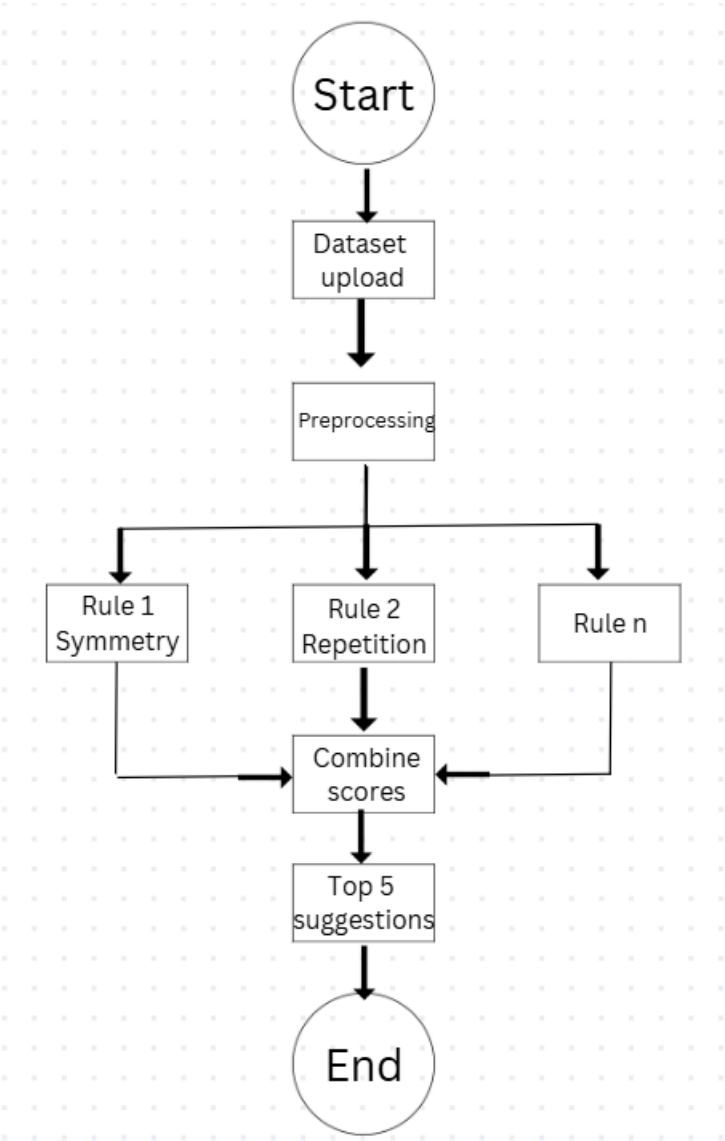

Figure 3: Box diagram.

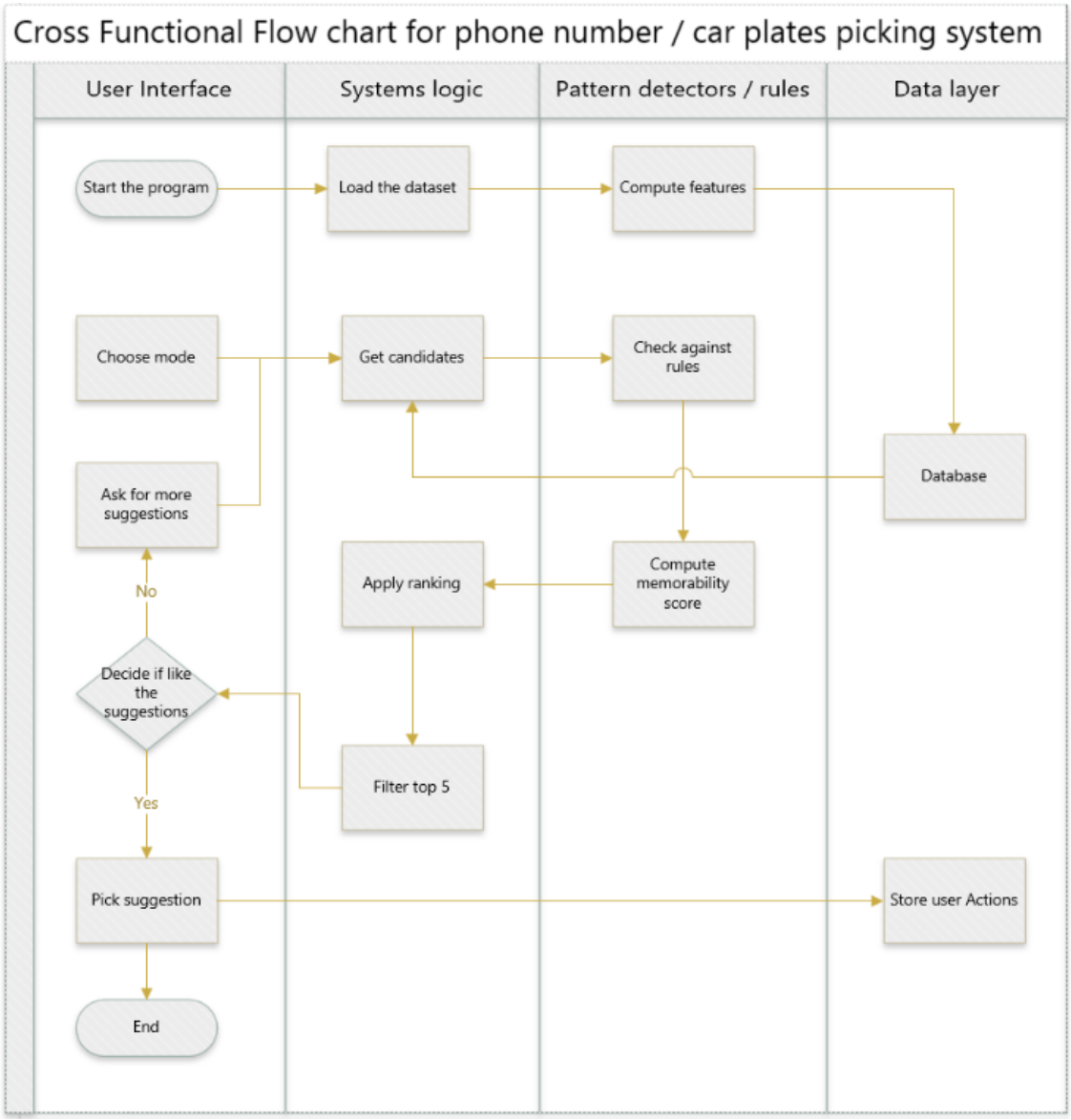

Figure 4: Cross functional chart for either modes of system

Tests

Functionalities in user interface

Test ID	Feature	Input	Expected result	Actual result	Pass /Fail	Proof

T1	Phone dataset loading	Button click “Find your number”	“Suggested number plates” screen appear	“Suggested number plates” screen appear	Pass	Outcome
T2	Plate dataset loading	Button click “Find your number plate”	“Suggested number plates” screen appear	“Suggested number plates” screen appear	Pass	Outcome
T3	“Back” button phone dataset	Button “Back” clicked in phone mode	Main screens reappear	Main screens reappear	Pass	Outcome
T4	“Back” button plate dataset	Button “Back” clicked in plate mode	Main screens reappear	Main screens reappear	Pass	Outcome
T5	Initial phone suggestions	Button click “Find your number”	System gives top 5 suggestions	System gives top 5 suggestions	Pass	Outcome
T6	Initial car plates suggestions	Button click “Find your number plate”	System gives top 5 suggestions	System gives top 5 suggestions	Pass	Outcome
T7	Input phone pattern search	Input “2323557”	System returns results either matching the search, or similar	System returns results either matching the search, or similar	Pass	Outcome
T8	Input plates pattern search	Input “REST”	System returns results either matching the search, or similar	System returns results either matching the search, or similar	Pass	Outcome
T9	“Suggest similar” to in number picker	“Suggest similar” is pressed	Button “Suggest similar” gives similar	Previous pick contains most of same digits and keeps the	Pass	Outcome

			looking numbers	alternating and repeated numbers. pattern present		
T10	Suggest similar to in plates picker	"Suggest similar" is pressed	After the button press, suggestions change into similar looking number plates	After the button press, suggestions change into similar looking number plates	Pass	Outcome
T11	Pick the number out of the dataset	Press button "I choose this one"	After picking the number, congratulating screen appears and if now the same number will be picked, system should not give it, because it is made taken	After picking the number, congratulating screen appears and if now the same number will be picked, system should not give it, because it is made taken	Pass	Outcome
T12	Pick the car plate out of the dataset	Press button "I choose this one"	After picking the car plate, congratulating screen appears and if now the same car plate will be picked, system should not give it, because it is made taken	After picking the number, congratulating screen appears and if now the same car plate will be picked, system does not give it, because it is made taken	pass	Outcome
T13	Button "home" takes user to main screen	Press button "home" after picking number	User should be taken to the main screen	User is taken to the main screen	Pass	Outcome
T14	Button "home" takes user to main screen	Press button "home" after	User should be taken to the main screen	User is taken to the main screen	Pass	Outcome

		picking car plate				
T15	Invalid input containing letters	Input letters	Error message	Error message appears	Pass	Outcome
T16	Invalid input plats	Too short of an input	Error message	Error message appears	Pass	Outcome

Rule level testing results screenshots

Test ID	Rule	Test input	Expected behaviours	Test outcome	Pass / Fail	Proof
T17	Alternating	1616	Rule would be recognised and suggestions would contain similar looking pattern.	The suggestions did contain similar looking number combinations and indication, that the rule was applied 161616	Pass	Outcome
T18	Word match	Bad	Exact match of an input is found	Exact match of an input is found	Pass	Outcome
T19	Leet	Bird	Suggestions should include similar variations containing word like suggestions including numbers like 8IRD	Parts of the word represented as digits	Pass	Outcome

T20	Phonetic	Me too	The suggestions should give plates like ME 2	The output contains ME2	Pass	Outcome
T21	Edit distance	Jack	The suggestions should include the letter combinations with near distance such as JA 78 CK	The output included suggestions as such: DJ 93ACK, MJ 99 ACK.	Pass	Outcome
T22	NumericPattern	2635	The suggestions should keep the exact or nearly exact pattern given by user present in suggestions, like 2635	The suggestion contained the pattern 2635.	Pass	Outcome
T23	LettersUser	Paw	The suggestion should include preserved letter clusters when possible, and produce exact matches like 56REST	The suggestions did include word paw.	Pass	Outcome
T24	SequenceRule	1234	Suggestions should include sequential pattens like 2345, or exact match 1234	The suggestions did include the sequence 1234	Pass	Outcome
T25	SymmetryRule	1221	The suggestions	The suggestions	Pass	Outcome

			should include a mirrored number patterns such as 1221, 3223	did include 122.		
T26	RepetitionRule	444	The suggestions should contain repeated digits such as 5555, 4444	The outcome did show the repetitive 444.	Pass	Outcome
T27	UniqueDigitsRule	555566	The lower character variate should be observable in suggestions	The suggestions did mostly contained same numbers.	Pass	Outcome
T28	RawSubstrRule	EST	At least few suggestions should contain exact match to EST	The suggestions did contain EST.	Pass	Outcome
T29	ReadabilityRule		The strong alphabetic match is available; it should be prioritized over number clutered alternatives and top suggestions be the most readable.	The top sugggestions are the most readable.	Pass	Outcome

Times

Testing ID	What is tested	1 000 * 2	100 000 * 2	1 000 000	10 000 000

T30	Time taken to turn the app on	1s	1s	1s	1s
T31	Time taken to turn the mode on	Instant	0.5s	0.5s	2s
T32	Time taken to get similar suggestion	Instant	1s	2s	4.5s
T33	Time taken to take an input and give suggestion	Instant	1.5s	1s	2.5s
T34	Time taken to turn the mode on	Instant	2s	1.5s	3.5s
T35	Time taken to get similar suggestion	Instant	2s	2.5	4.5s
T36	Time taken to take an input and give suggestion	Instant	1.5s	3s	4.5s

0 – poor, 1 – partial, 2 – strong, overall total 0-6

Inputs

Phone numbers

Testing ID	Query	Query preservation	Memorability relevance	Plausibility	Total	Proof
T37	33333333	2	1	2	6	Outcome
T38	123456789	2	2	2	6	Outcome
T39	12677621	2	2	2	6	Outcome
T40	1515	2	2	2	6	Outcome
T41	741	2	1	2	6	Outcome

Number plates with clear words

Testing ID	Query	Query preservation	Memorability relevance	Plausibility	Total	Proof

T42	Best	2	2	2	6	Outcome
T43	Ace	2	2	2	6	Outcome
T44	Peace	0	1	1	2	Outcome
T45	King	2	1	2	5	Outcome

Number plates for harder cases

Testing ID	Query	Query preservation	Memorability relevance	Plausibility	Total	Proof
T46	Moon	2	2	2	6	Outcome
T47	Forever	0	0	0	0	Outcome
T48	4ever	1	1	1	3	Outcome

Suggest similar
Phone numbers

Test ID	Query	Query preservation	Memorability relevance	Plausibility	Total	Proof
T49	07417677678	2	2	2	6	Outcome
T50	07767767878	2	2	2	6	Outcome
T51	07767765388	2	2	2	6	Outcome

Number plates

Test ID	Query	Query preservation	Memorability relevance	Plausibility	Total	Proof
T52	YA39 RAY	1	1	1	3	Outcome
T53	BE 79 LLY	0	1	0	1	Outcome
T54	TB 78 ELL	1	0	0	1	Outcome

Images

Name	Type	Schema
Tables (1)		
phone_numbers		CREATE TABLE phone_numbers (id INTEGER PRIMARY KEY, raw TEXT NOT NULL,
id	INTEGER	"id" INTEGER
raw	TEXT	"raw" TEXT NOT NULL
digits	TEXT	"digits" TEXT NOT NULL
availability	TEXT	"availability" TEXT NOT NULL
symmetry	REAL	"symmetry" REAL
repetition	REAL	"repetition" REAL
sequence_score	REAL	"sequence_score" REAL
unique_digits	REAL	"unique_digits" REAL
alternating	REAL	"alternating" REAL
base_score	REAL	"base_score" REAL
Indices (3)		
idx_phone_availability		CREATE INDEX idx_phone_availability ON phone_numbers(availability)
idx_phone_base_score		CREATE INDEX idx_phone_base_score ON phone_numbers(base_score)
idx_phone_digits		CREATE INDEX idx_phone_digits ON phone_numbers(digits)
Views (0)		
Triggers (0)		

Figure 5: SQLite table structure for phone numbers

Name	Type	Schema
Tables (1)		
plate_numbers		CREATE TABLE plate_numbers (id INTEGER PRIMARY KEY, raw TEXT NOT NULL,
id	INTEGER	"id" INTEGER
raw	TEXT	"raw" TEXT NOT NULL
digits	TEXT	"digits" TEXT NOT NULL
letters	TEXT	"letters" TEXT NOT NULL
leet_normalized	TEXT	"leet_normalized" TEXT NOT NULL
availability	TEXT	"availability" TEXT NOT NULL
symmetry	REAL	"symmetry" REAL
repetition	REAL	"repetition" REAL
sequence_score	REAL	"sequence_score" REAL
unique_digits	REAL	"unique_digits" REAL
alternating	REAL	"alternating" REAL
base_score	REAL	"base_score" REAL
Indices (5)		
idx_plate_availability		CREATE INDEX idx_plate_availability ON plate_numbers(availability)
idx_plate_base_score		CREATE INDEX idx_plate_base_score ON plate_numbers(base_score)
idx_plate_digits		CREATE INDEX idx_plate_digits ON plate_numbers(digits)
idx_plate_leet_norm		CREATE INDEX idx_plate_leet_norm ON plate_numbers(leet_normalized)
idx_plate_letters		CREATE INDEX idx_plate_letters ON plate_numbers(letters)
Views (0)		
Triggers (0)		

Figure 6: SQLite table structure for car plates

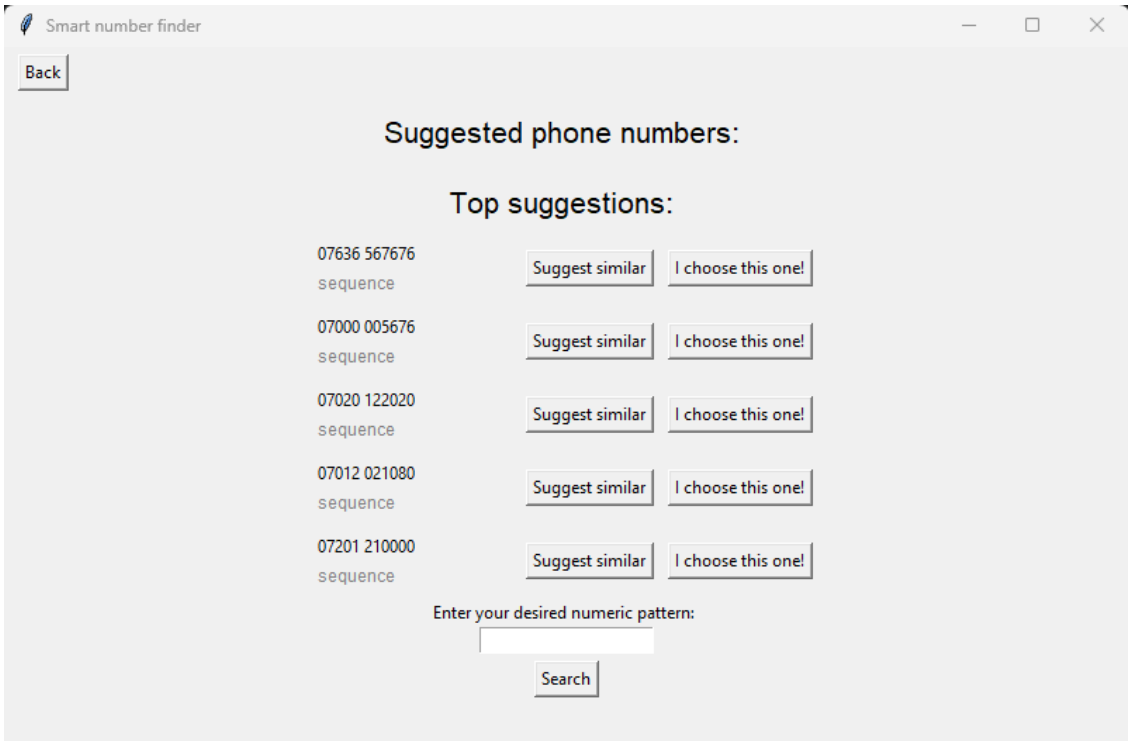

Figure 7: T1 result.

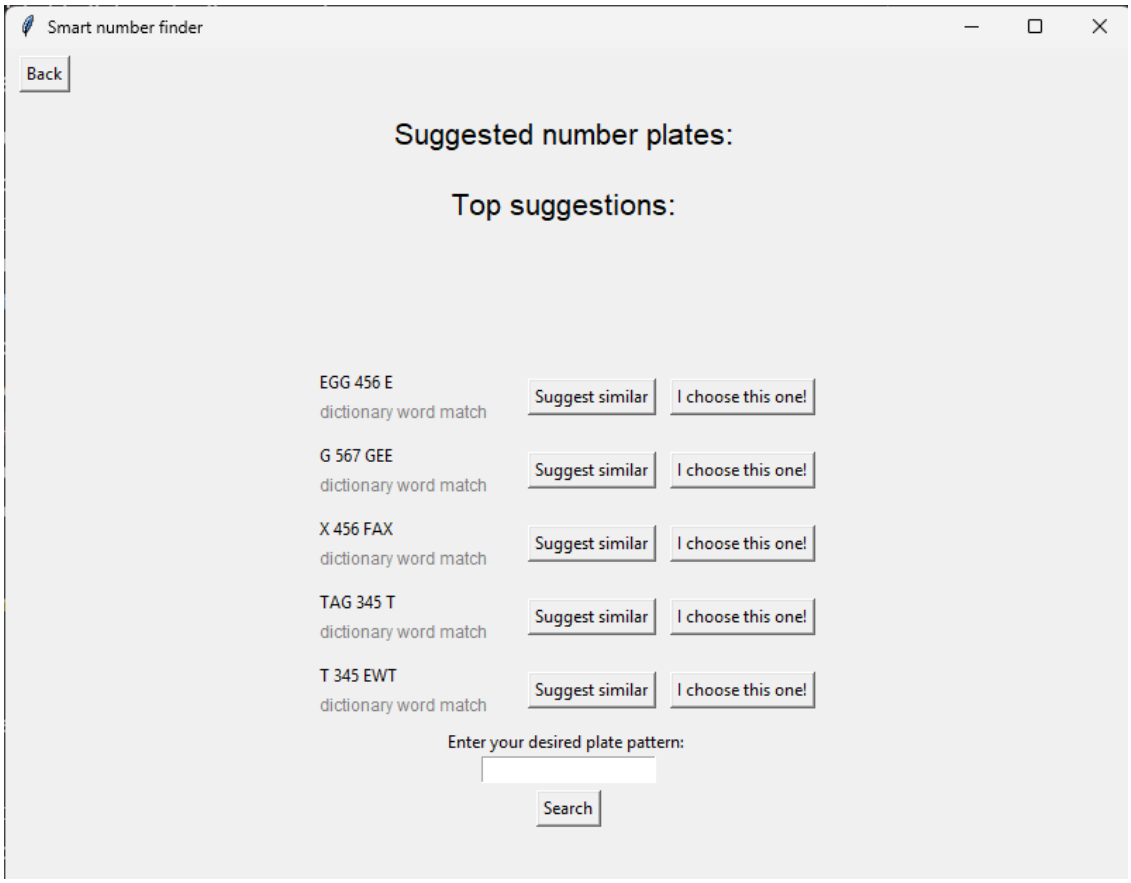

Figure 8: T2 outcome.

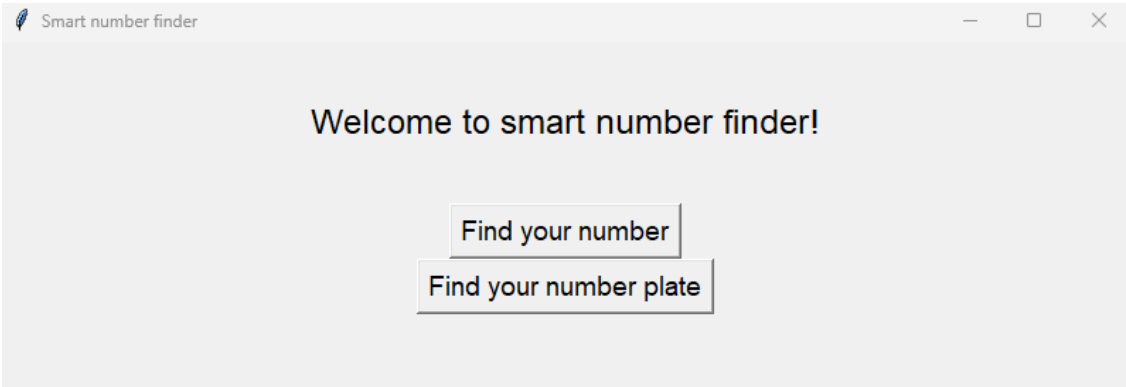

Figure 9: T3 outcome.

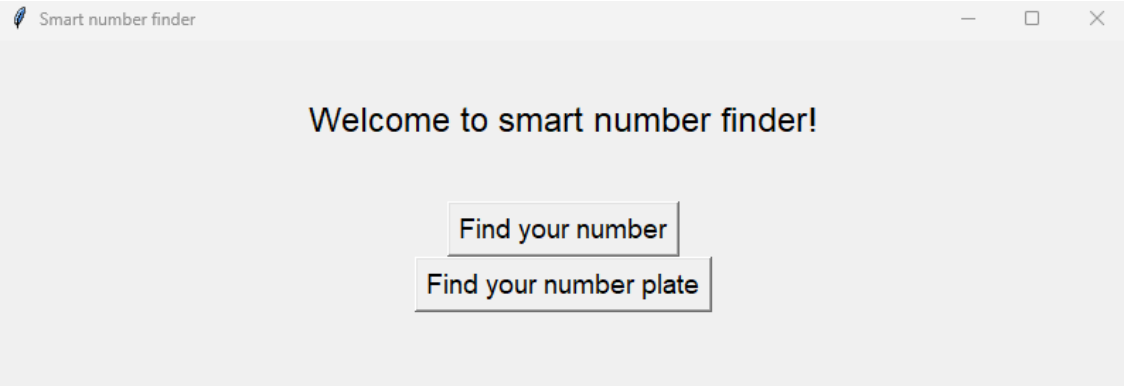

Figure 10: T4 outcome.

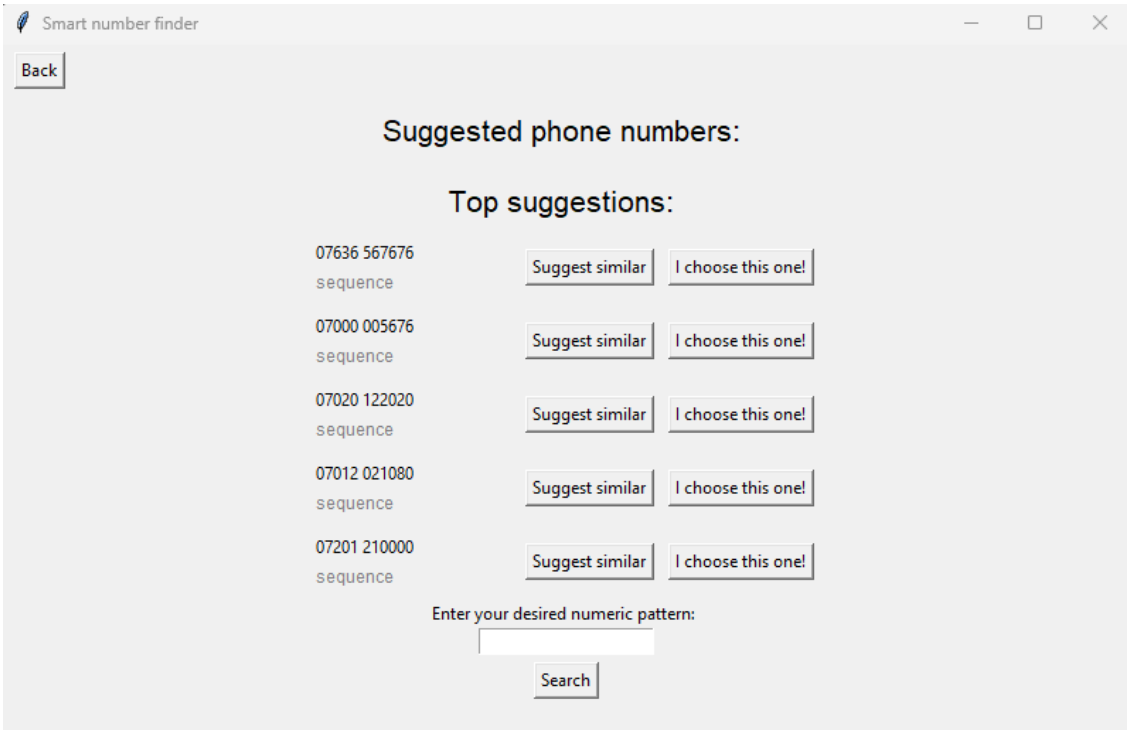

Figure 11: T5 outcome.

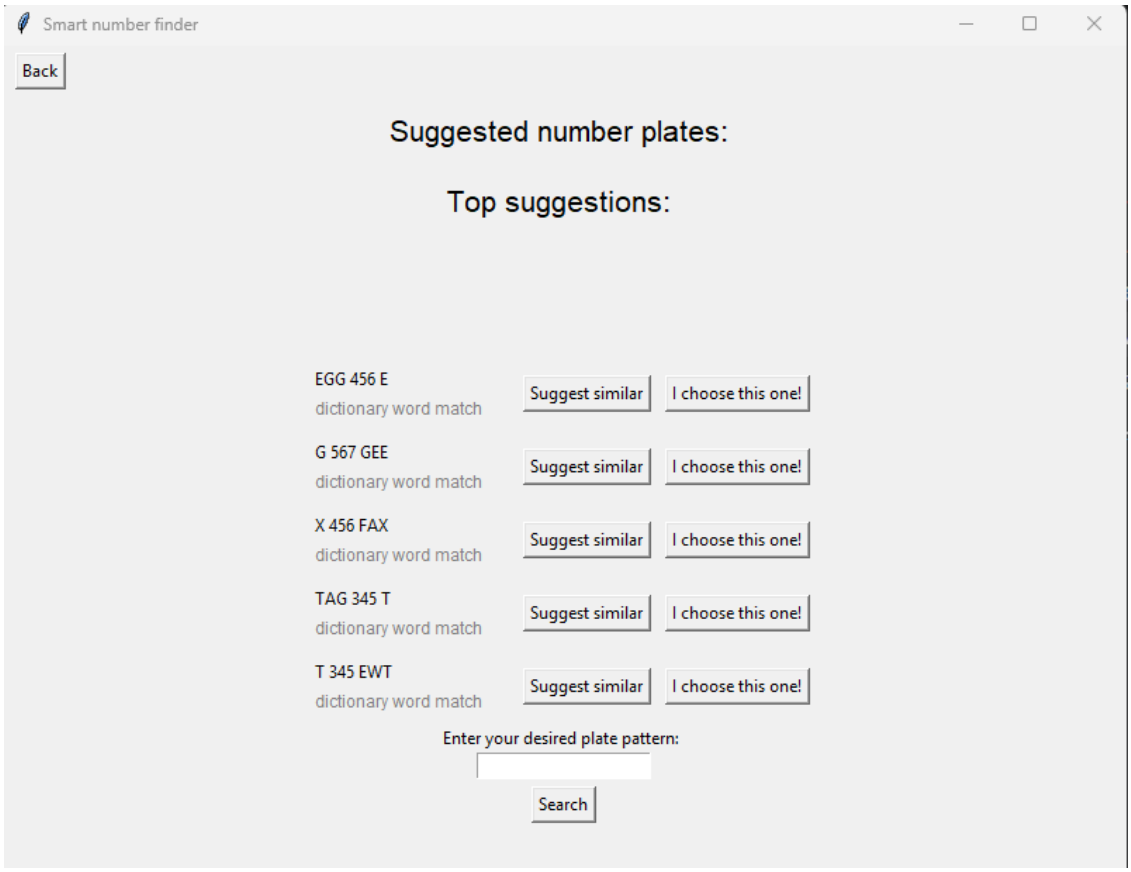

Figure 12: T6 outcome.

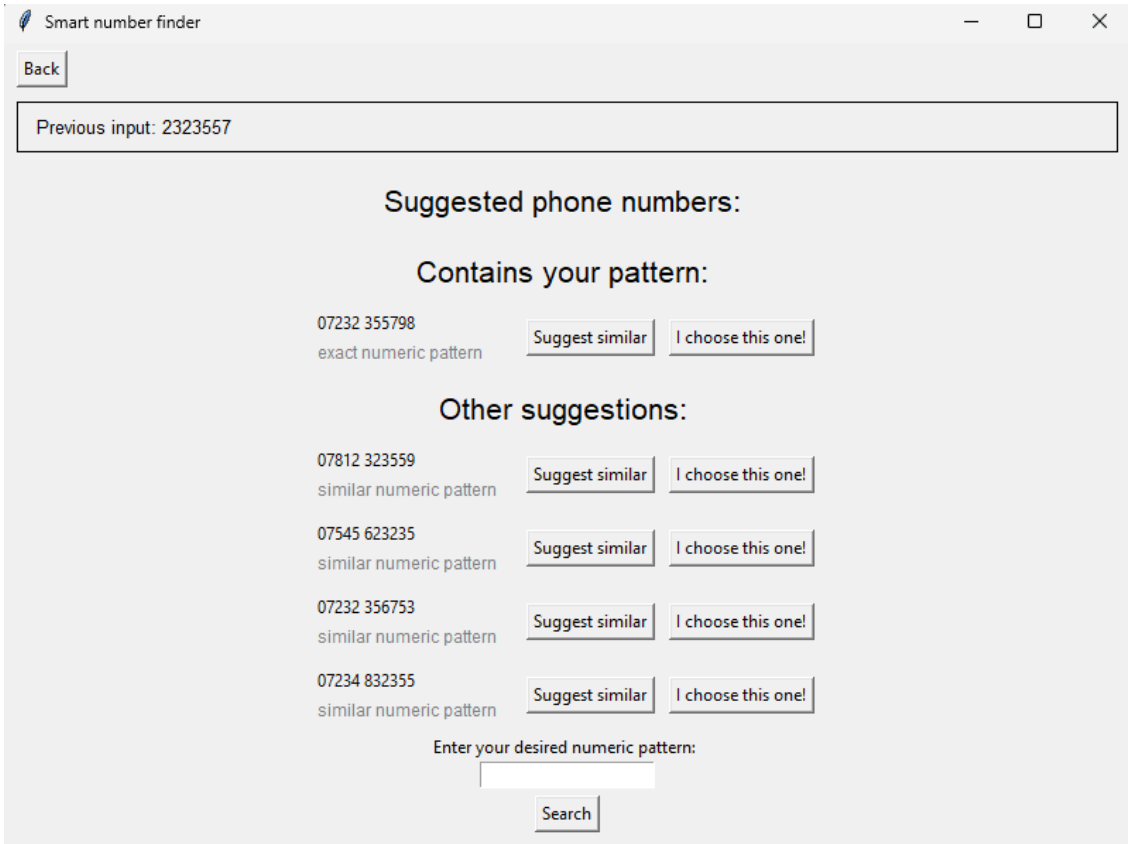

Figure 13: T7 outcome.

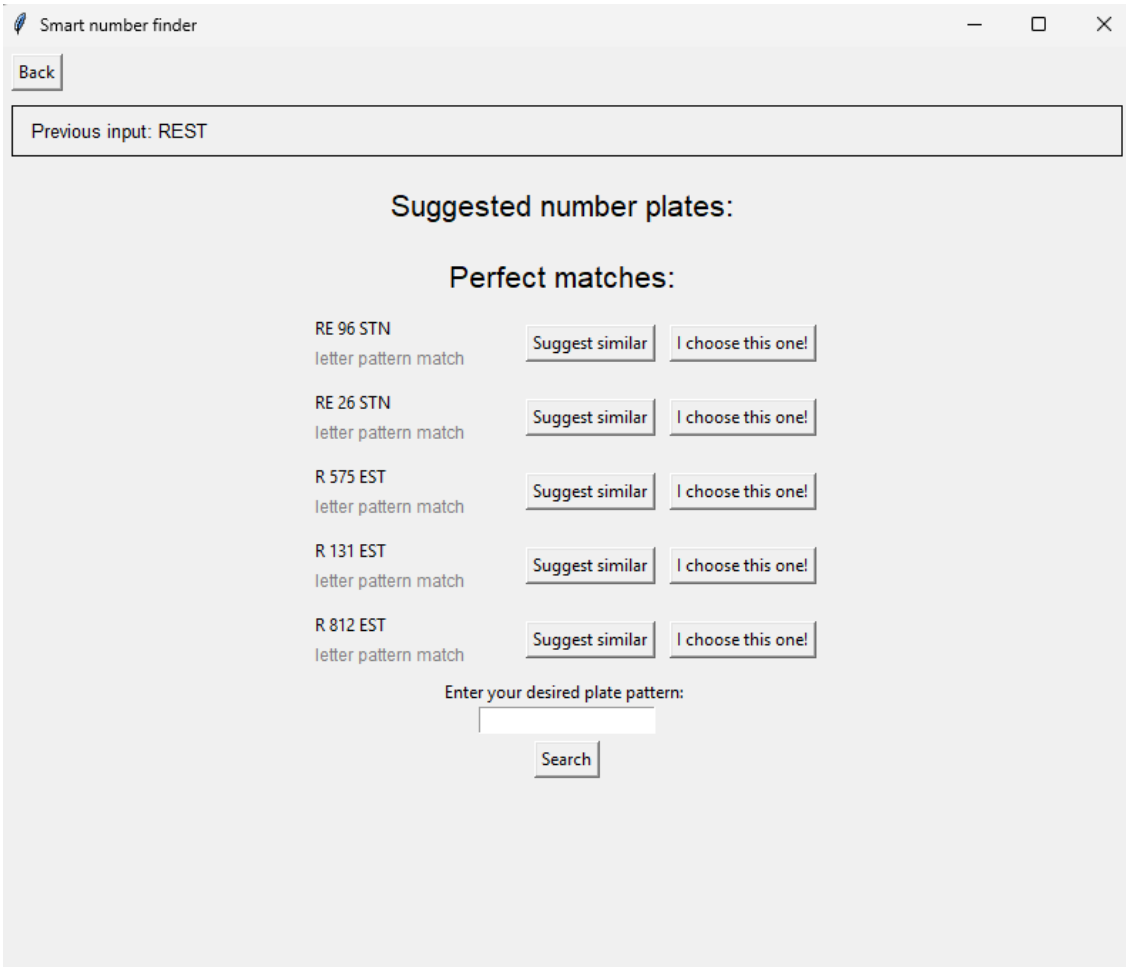

Figure 14: T8 outcome.

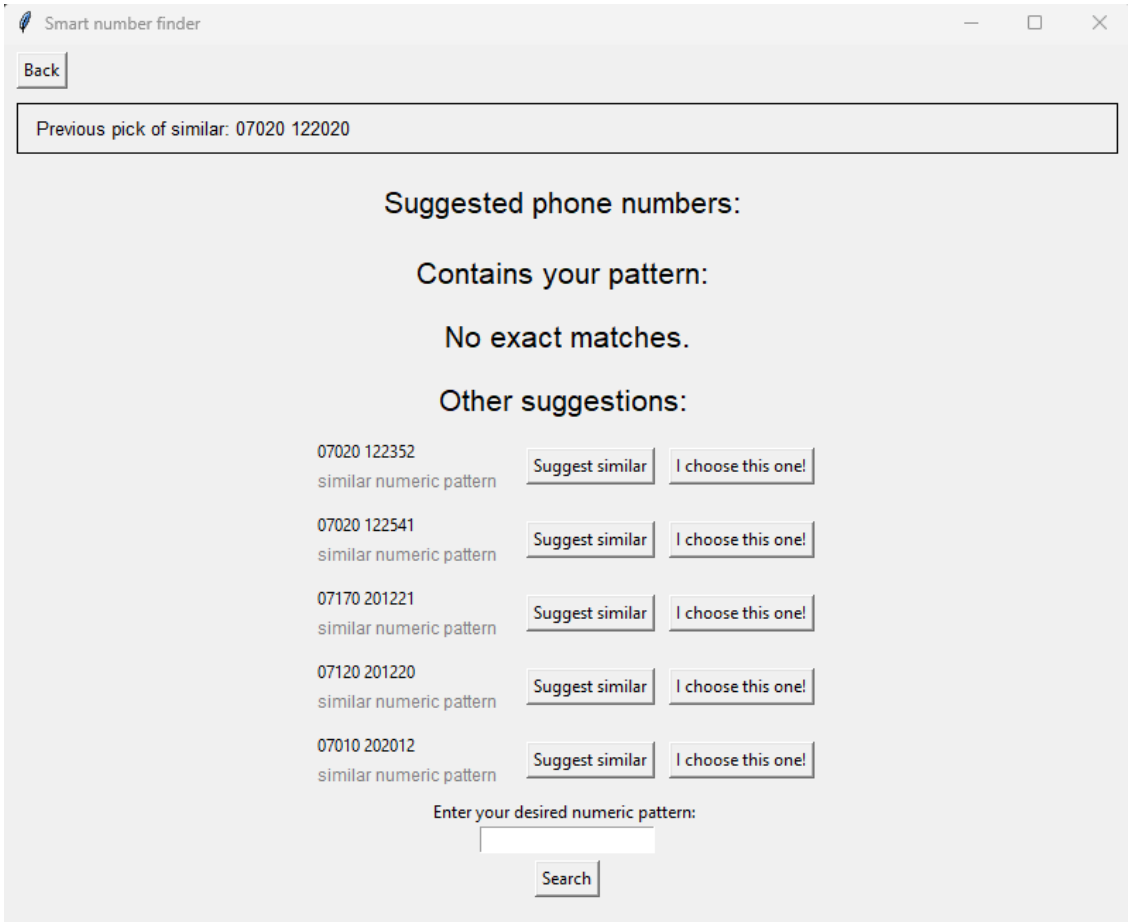

Figure 15: T9 outcome.

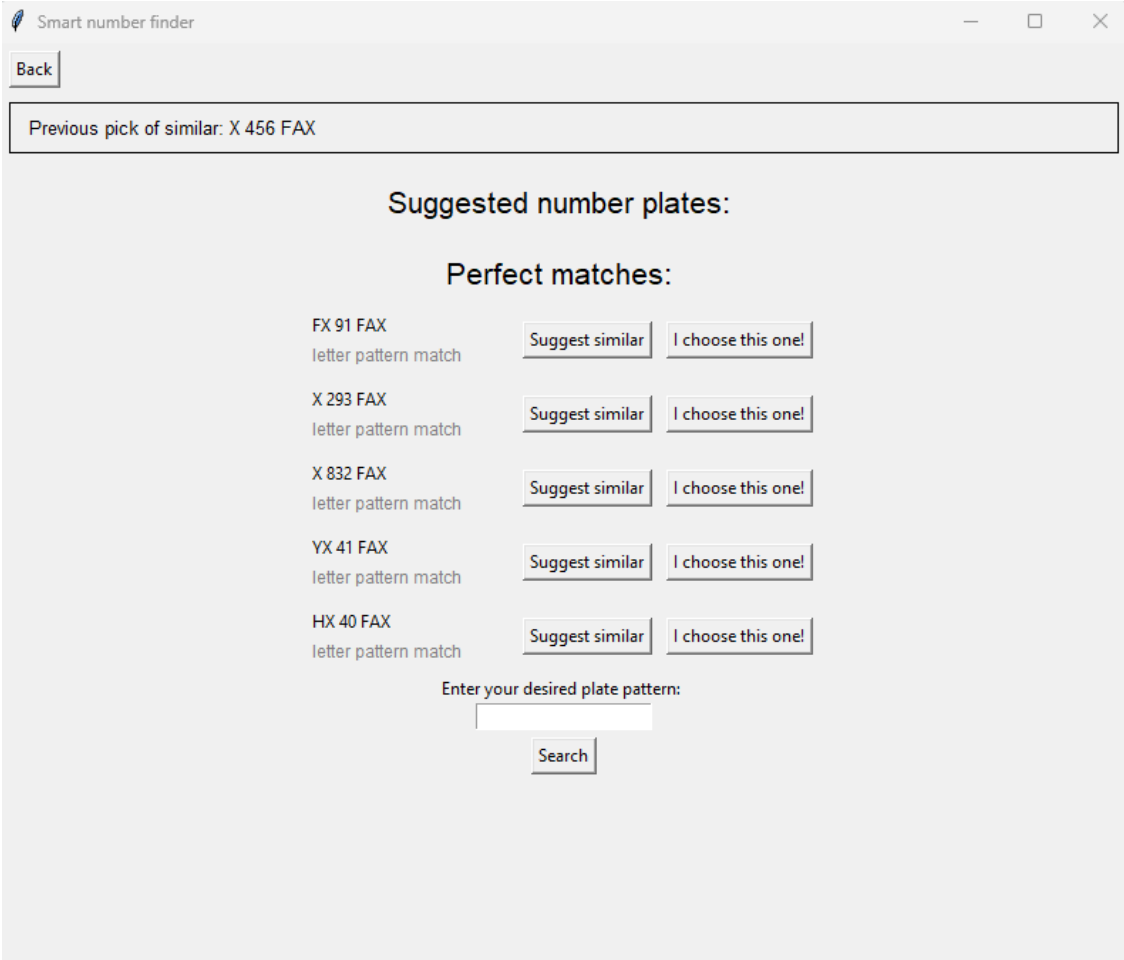

Figure 16: T10 outcome.

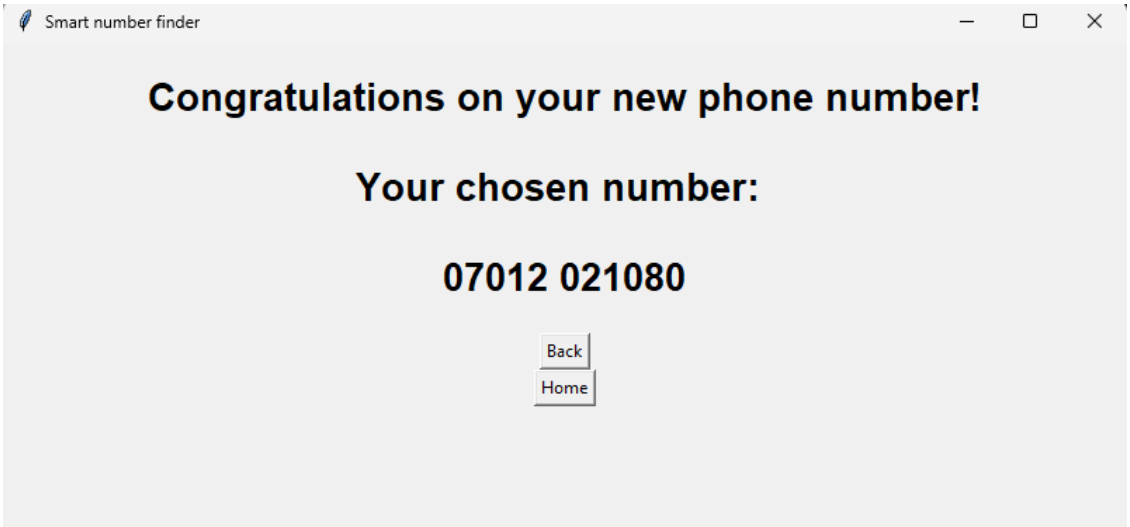

Figure 17: T11 outcome.

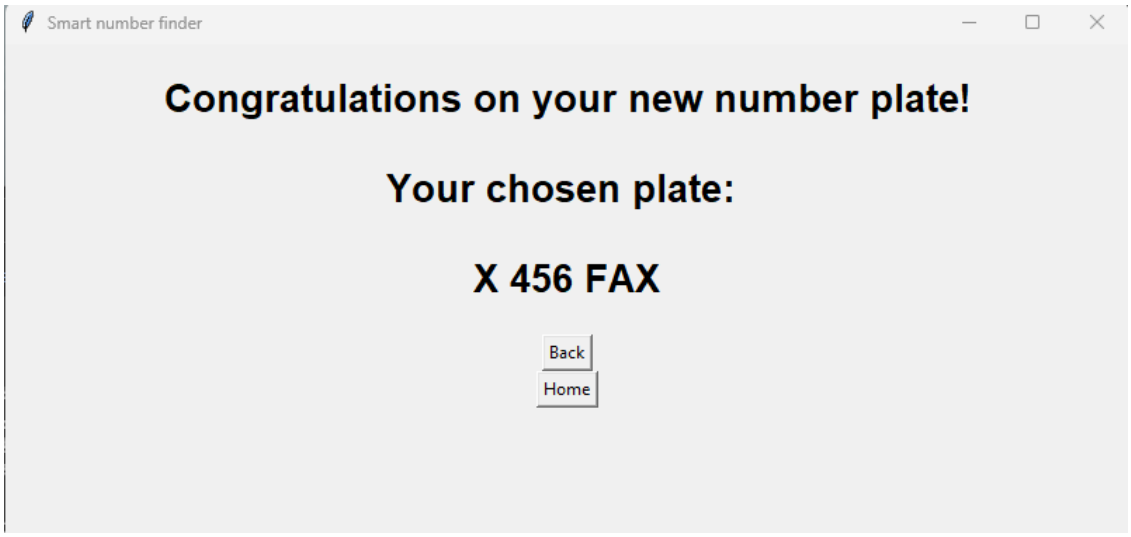

Figure 18: T12 outcome.

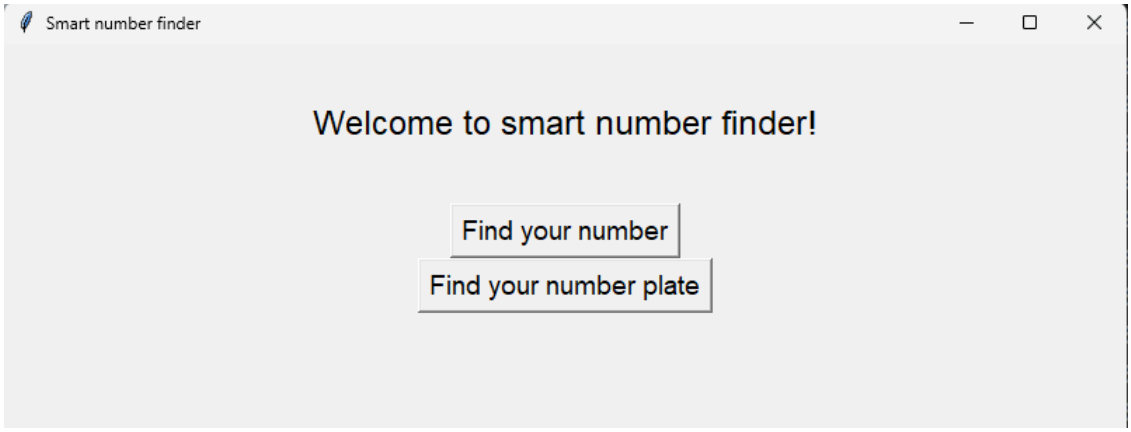

Figure 19: T13 outcome.

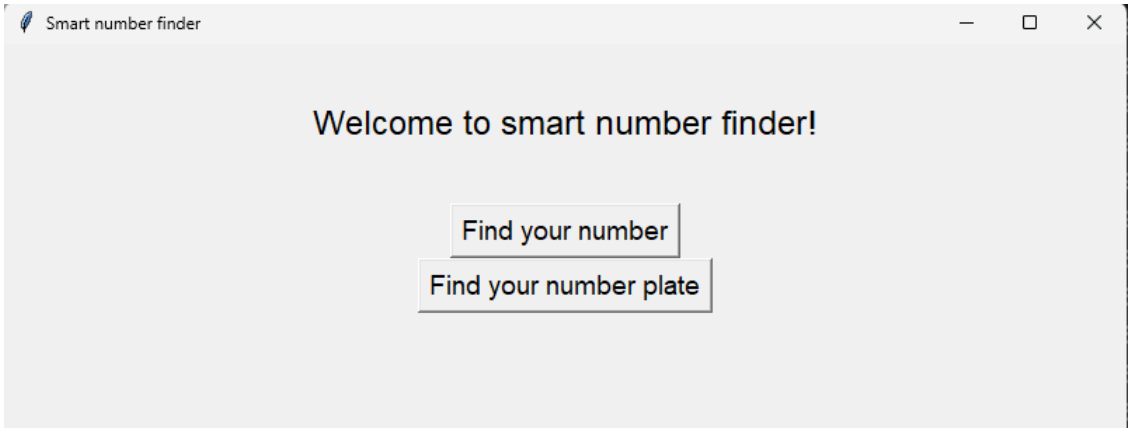

Figure 20: T14 outcome.

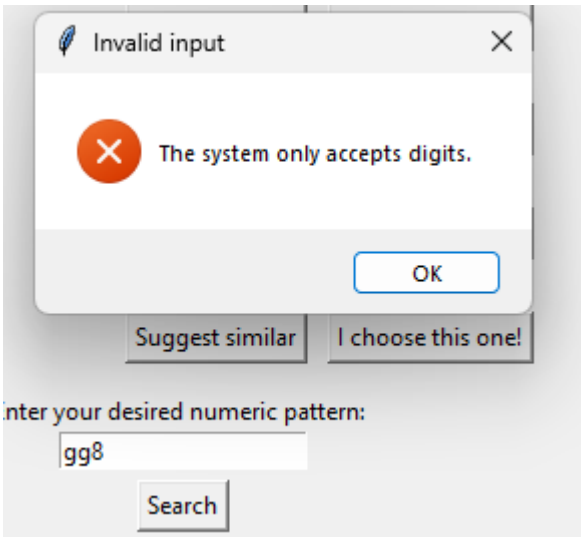

Figure 21: T15 outcome.

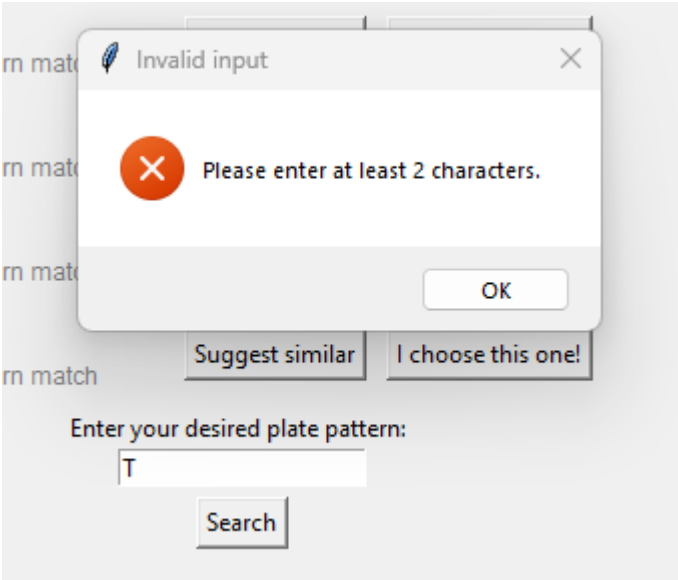

Figure 22: T16 outcome.

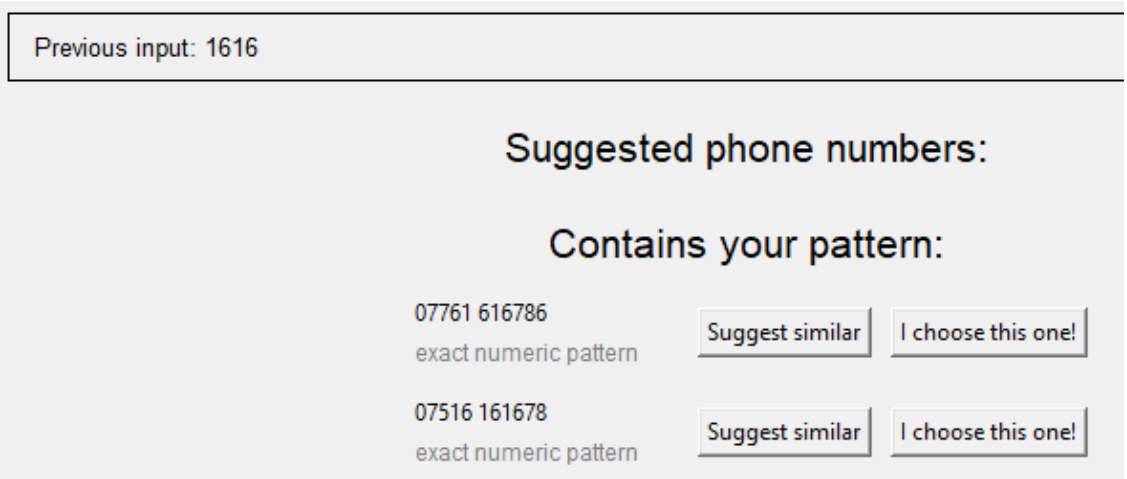

Figure 23: T17 outcome.

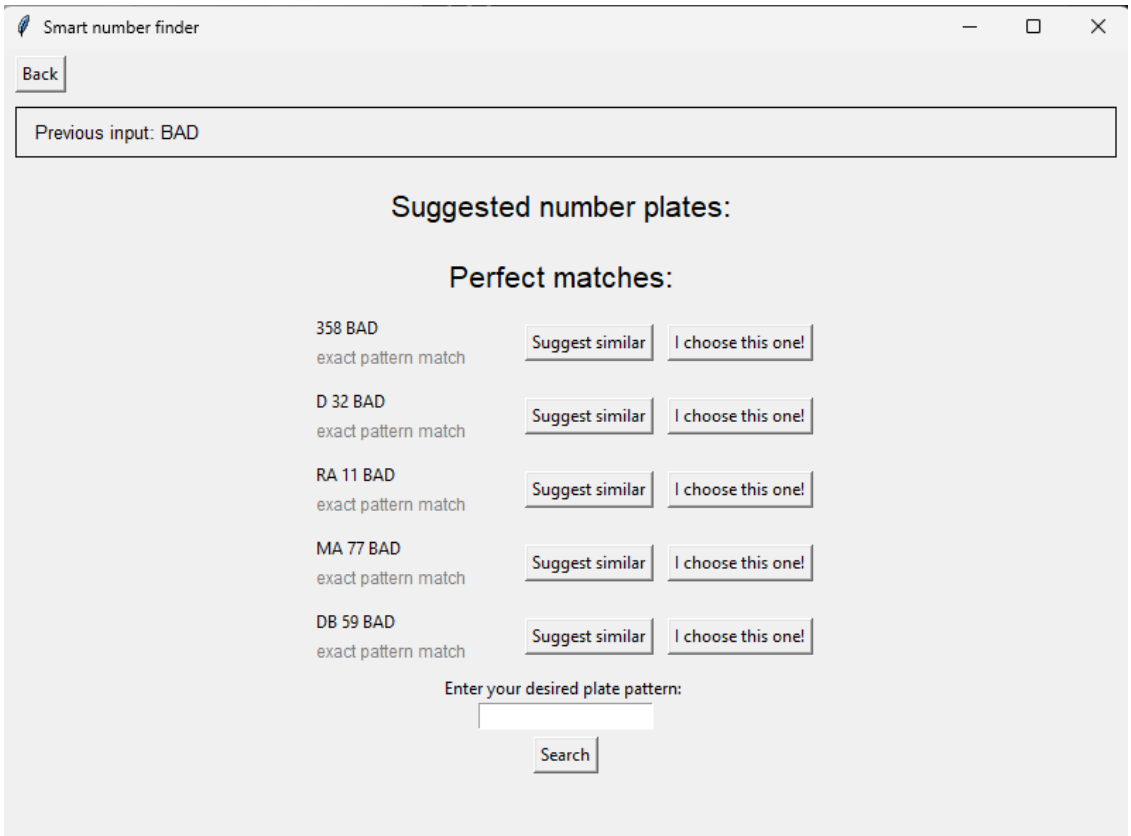

Figure 24: T18 outcome.

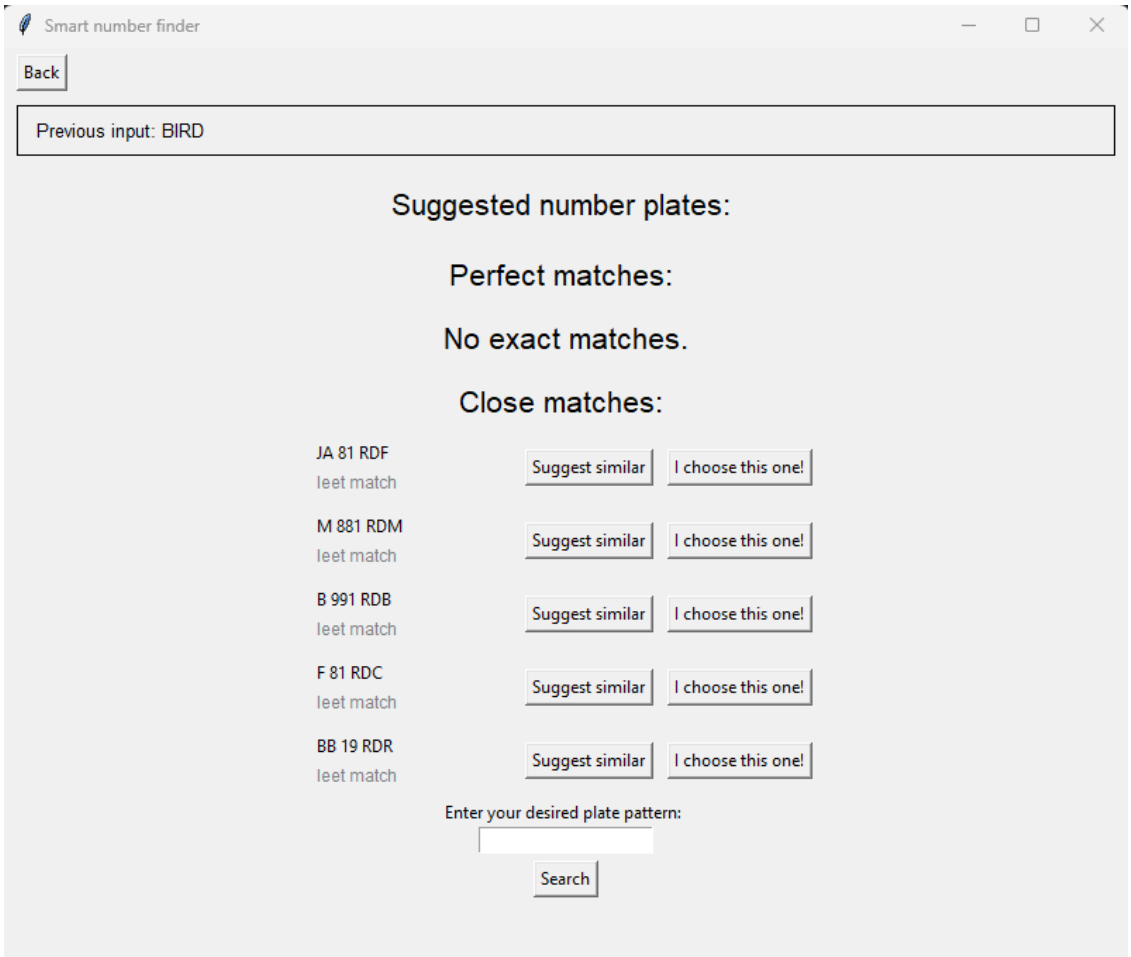

Figure 25: T19 outcome.

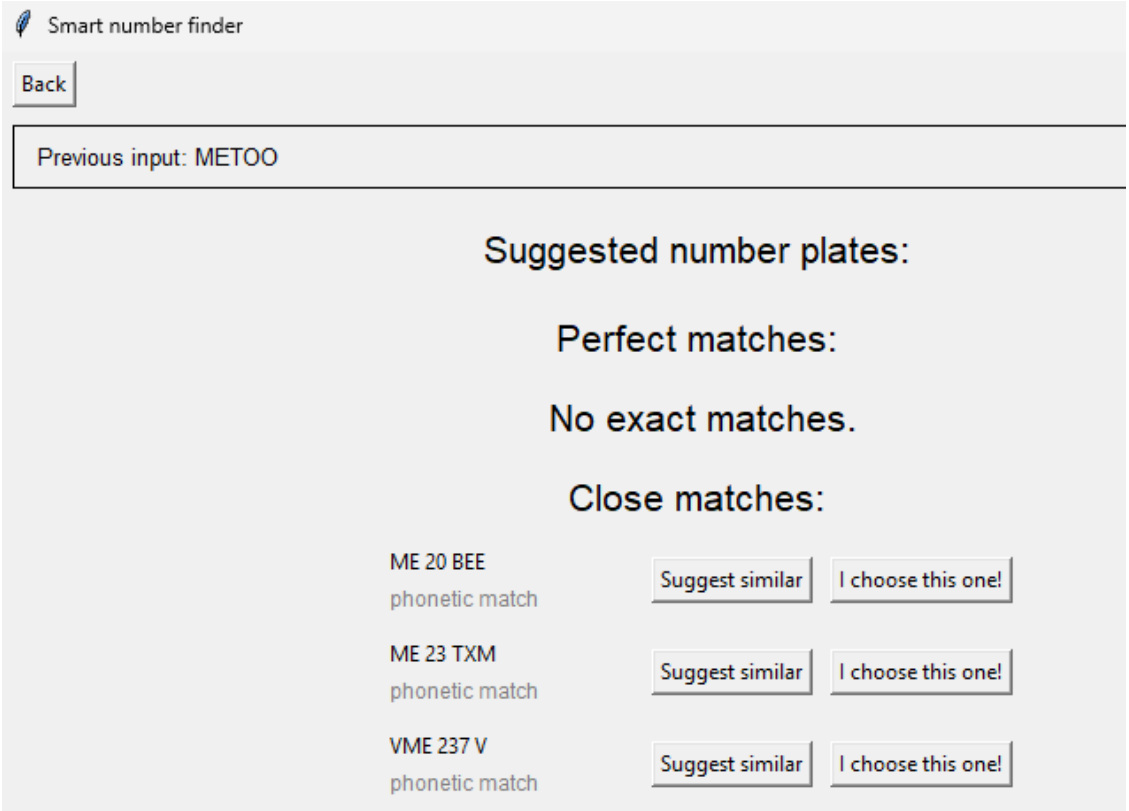

Figure 26: T20 outcome.

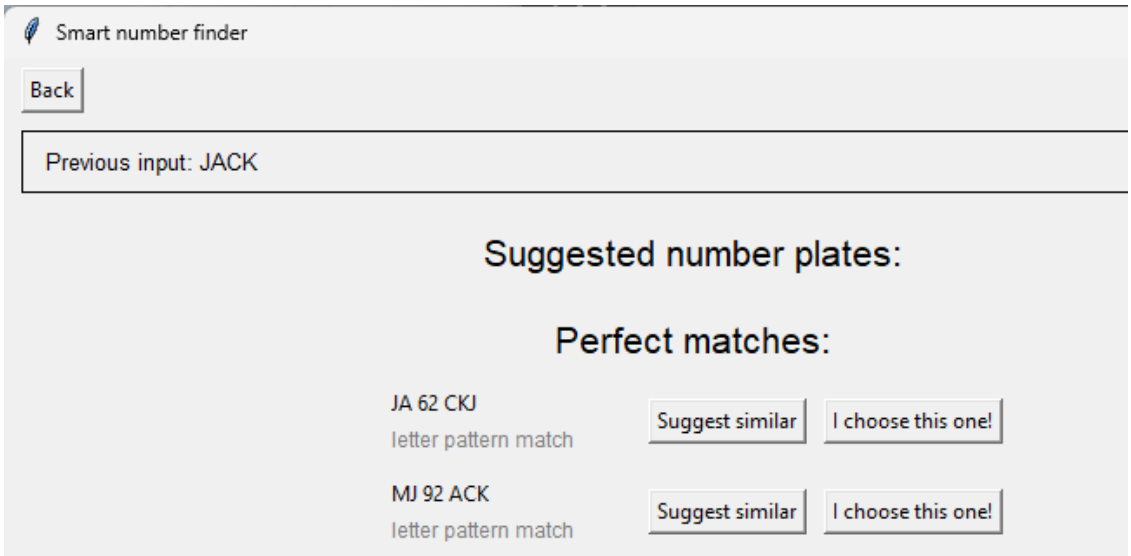

Figure 27: T21 outcome.

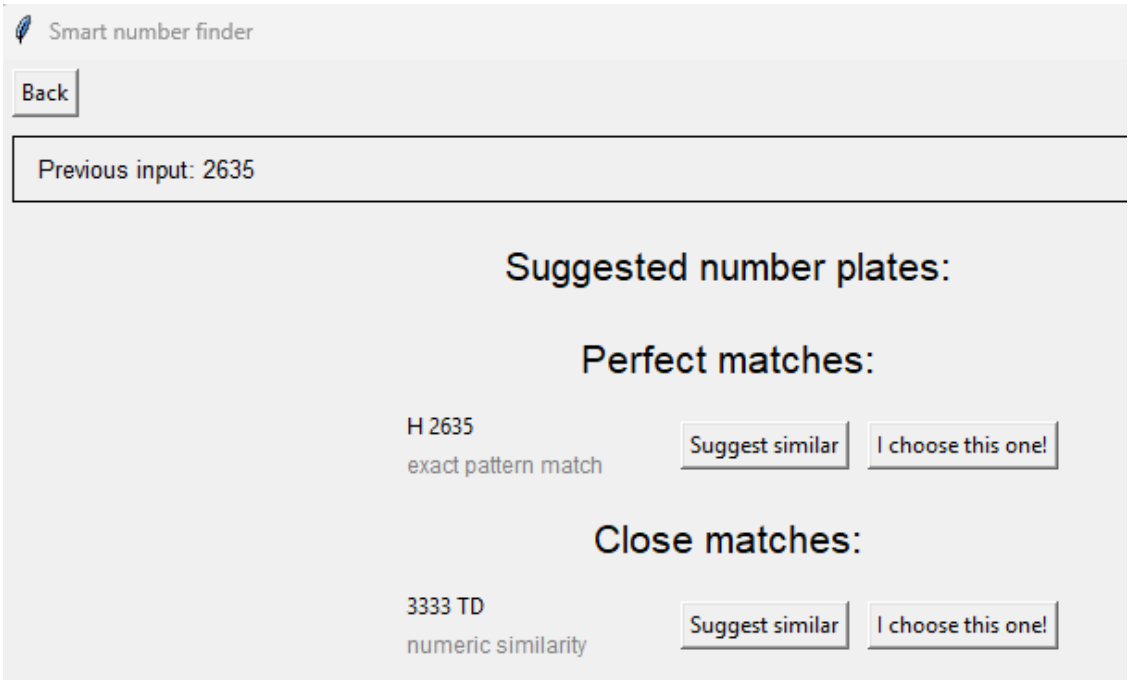

Figure 28: T22 outcome.

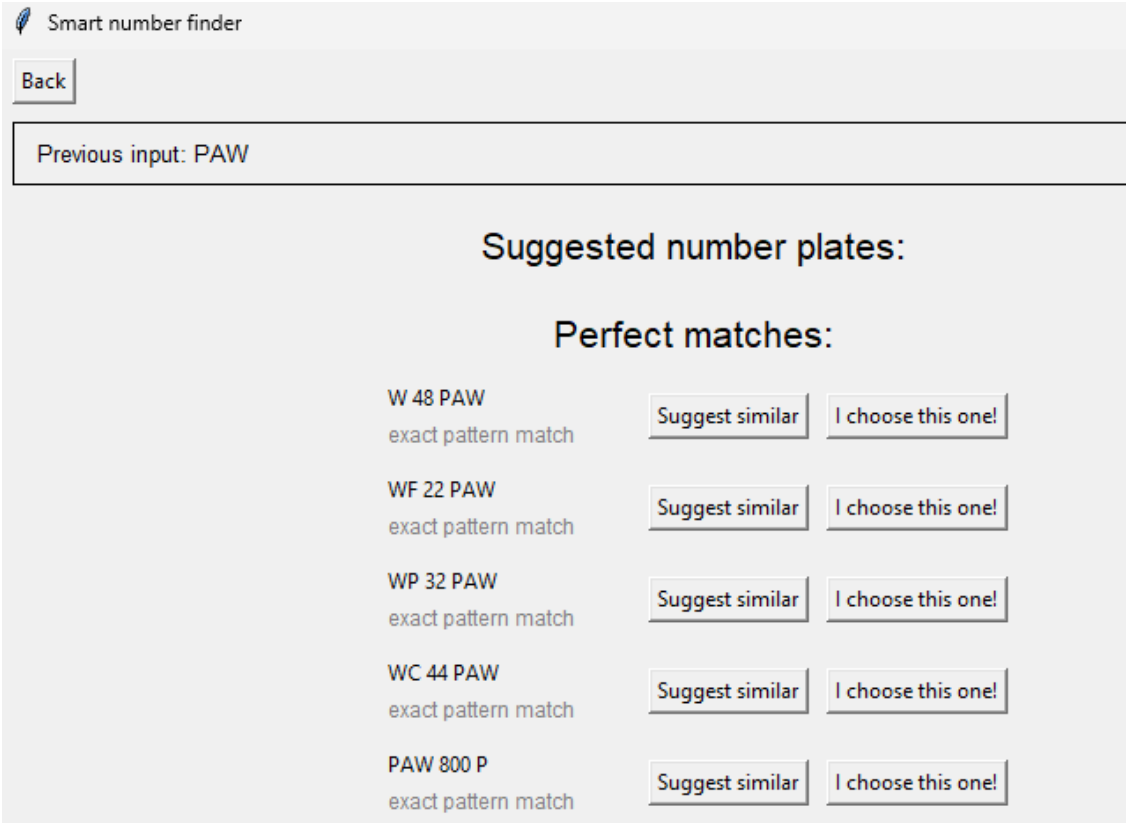

Figure 29: T23 outcome.

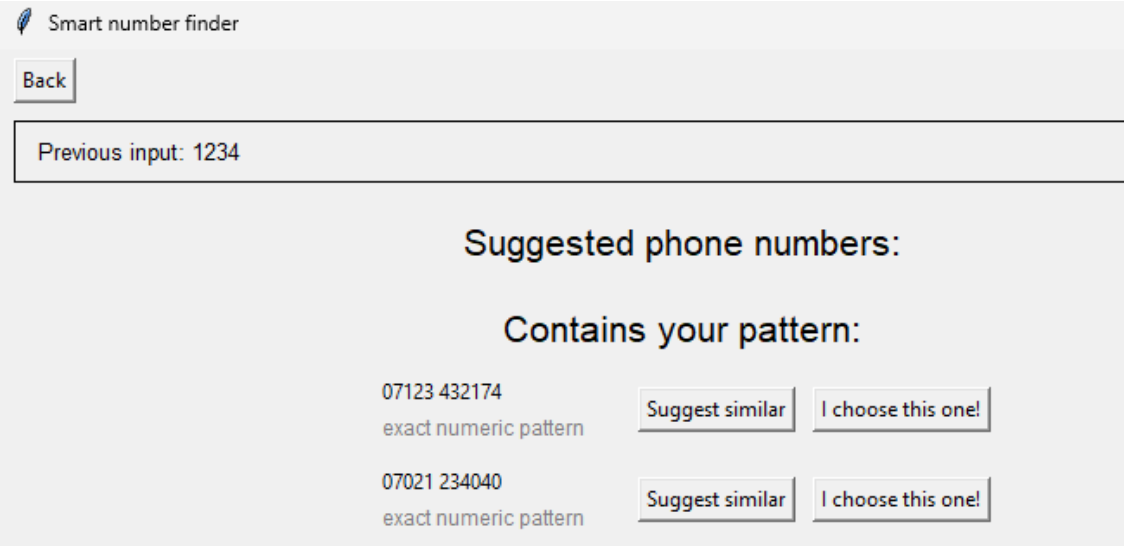

Figure 30: T24 Outcome.

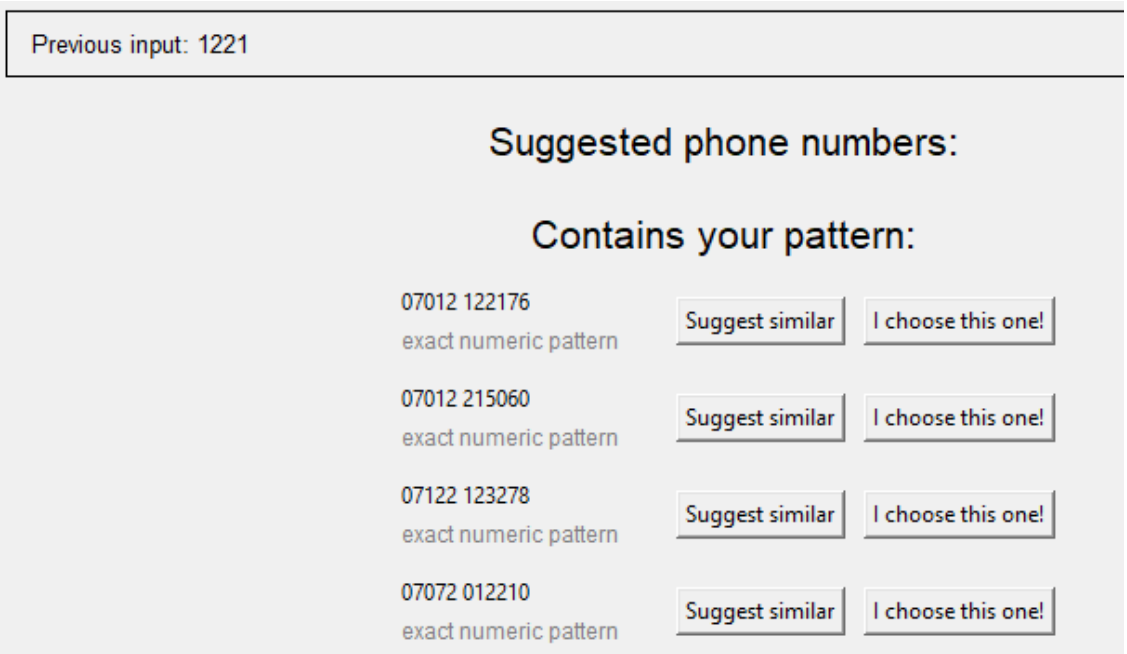

Figure 31: T25 outcome.

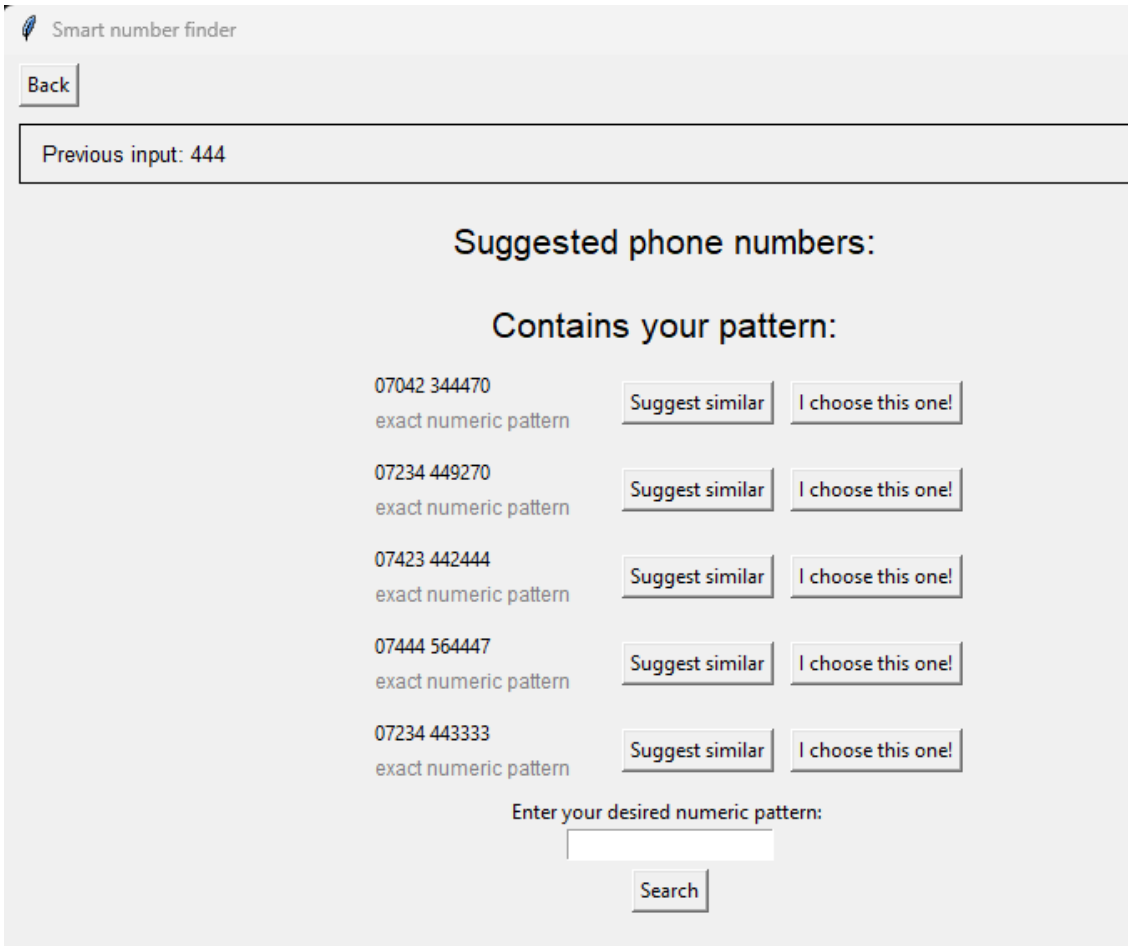

Figure 32: T26 outcome.

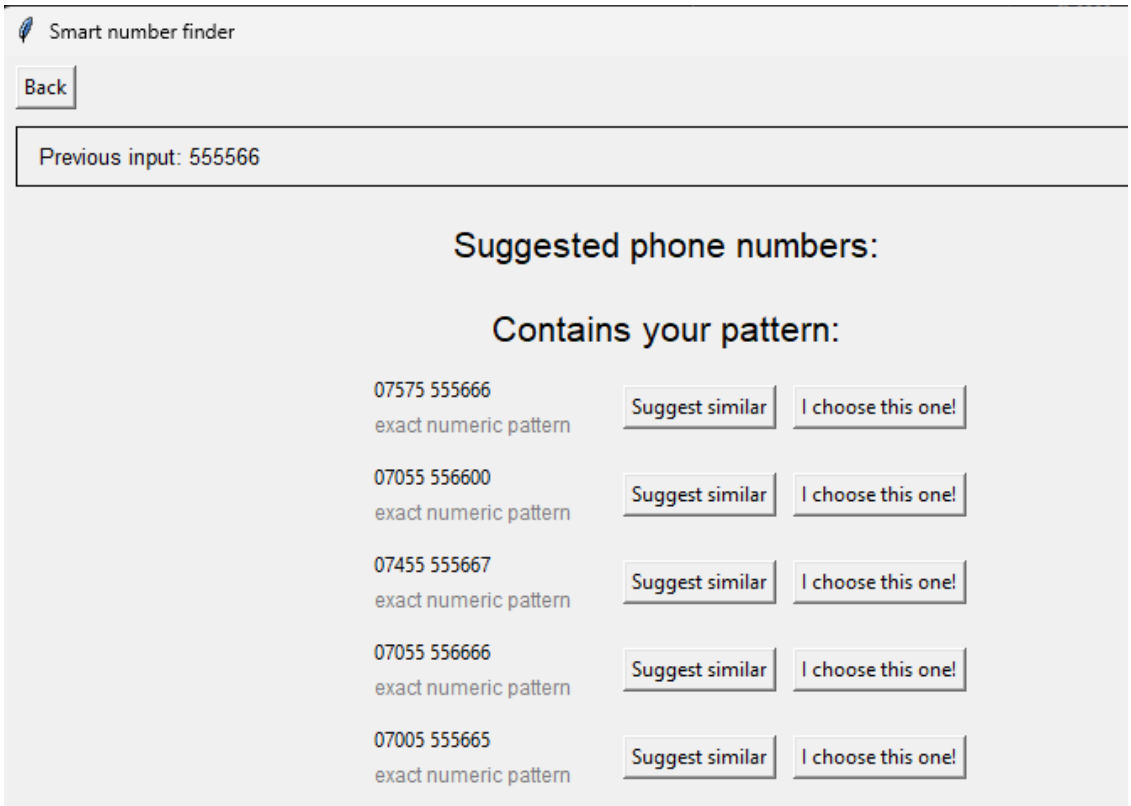

Figure 33: T27 outcome.

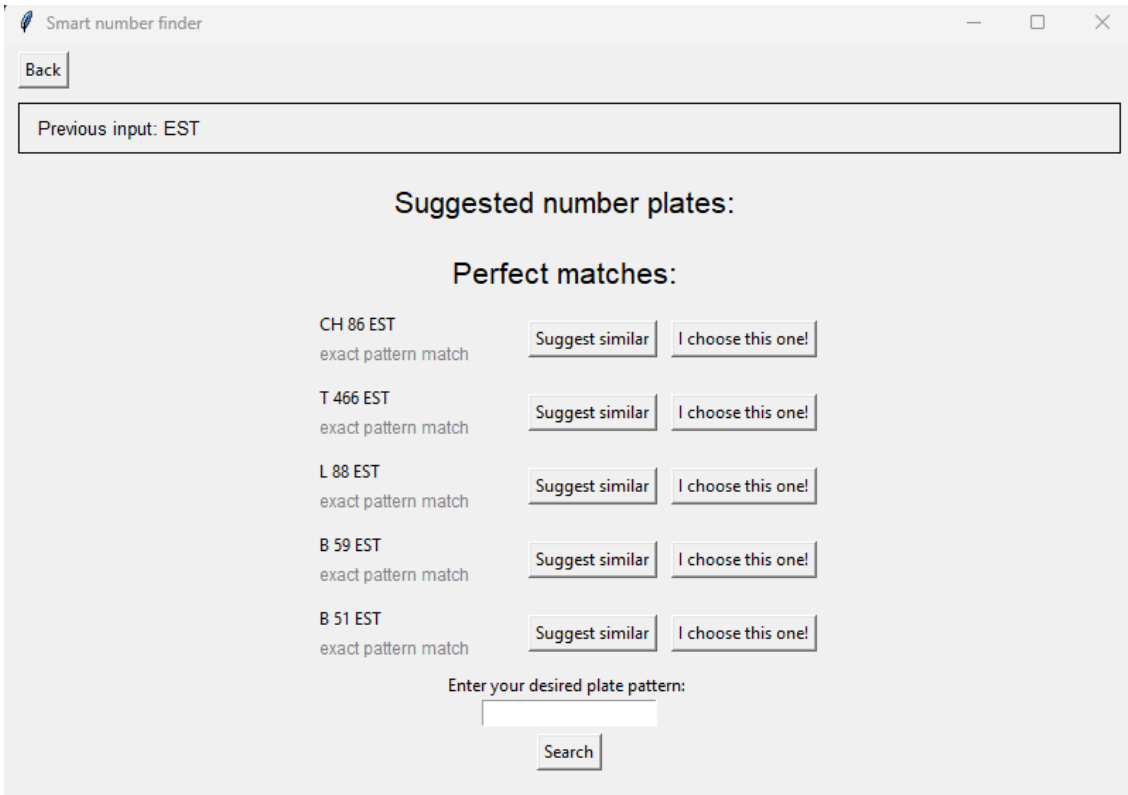

Figure 34: T28 outcome.

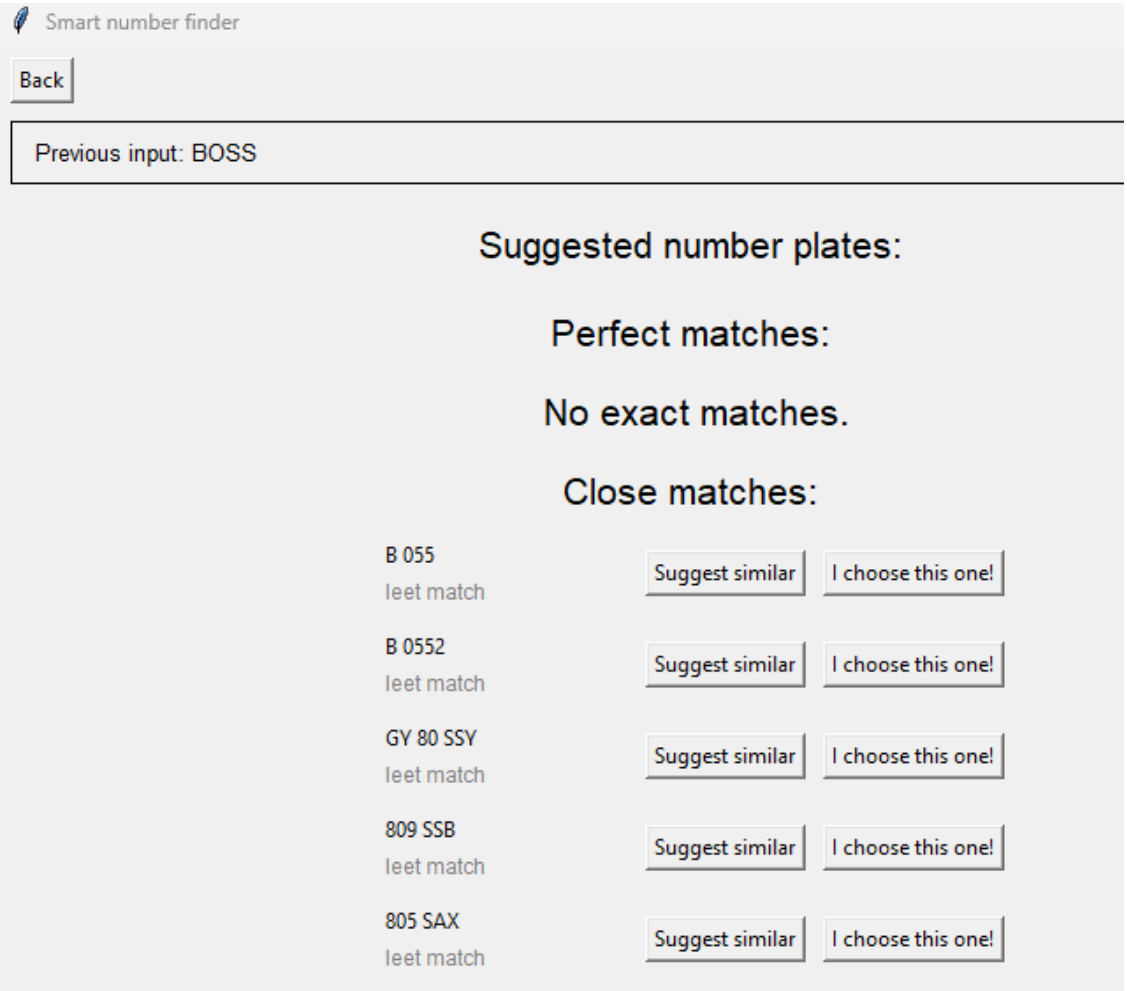

Figure 35: T29 outcome.

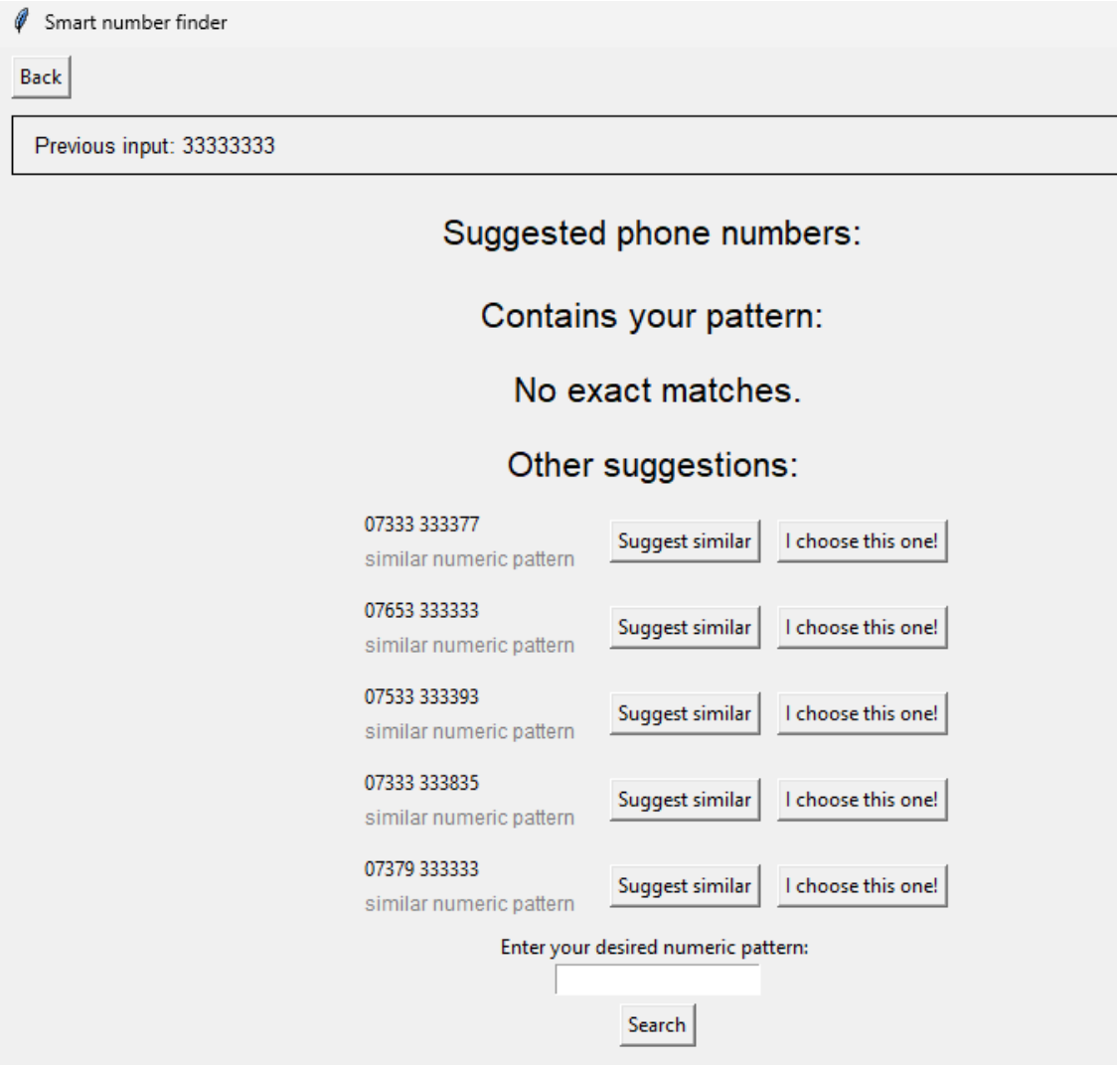

Figure 36: T37 outcome.

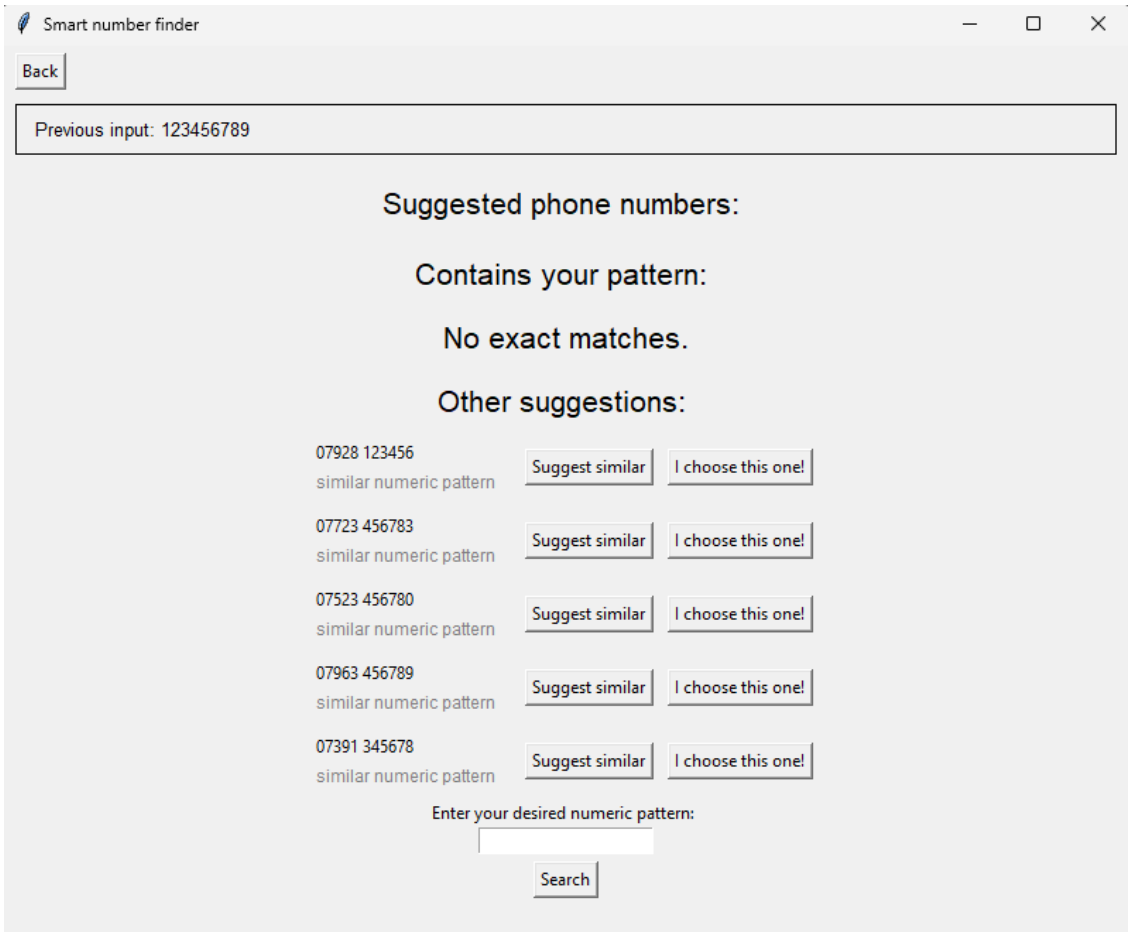

Figure 37: T38 outcome.

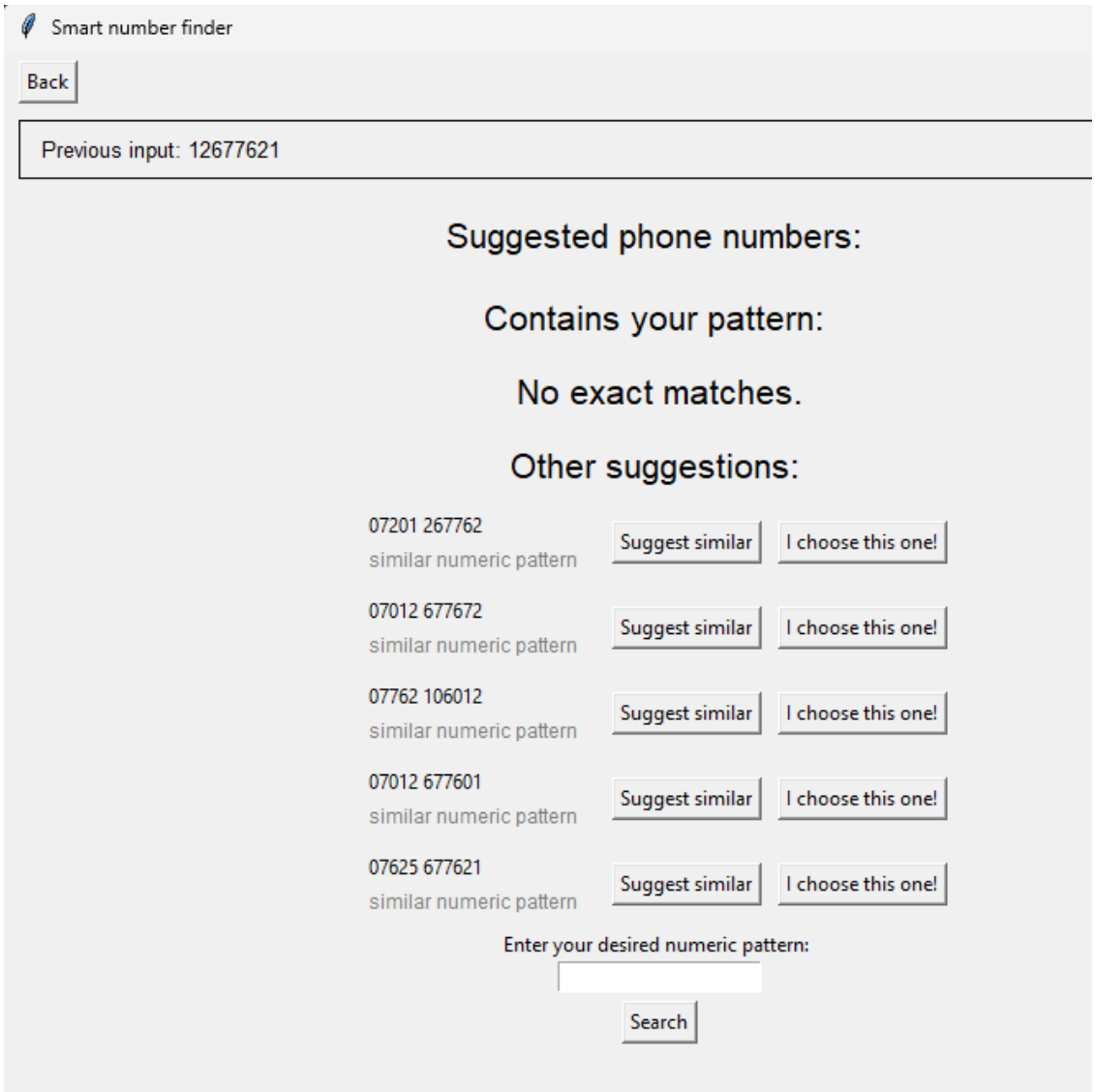

Figure 38: T39 outcome.

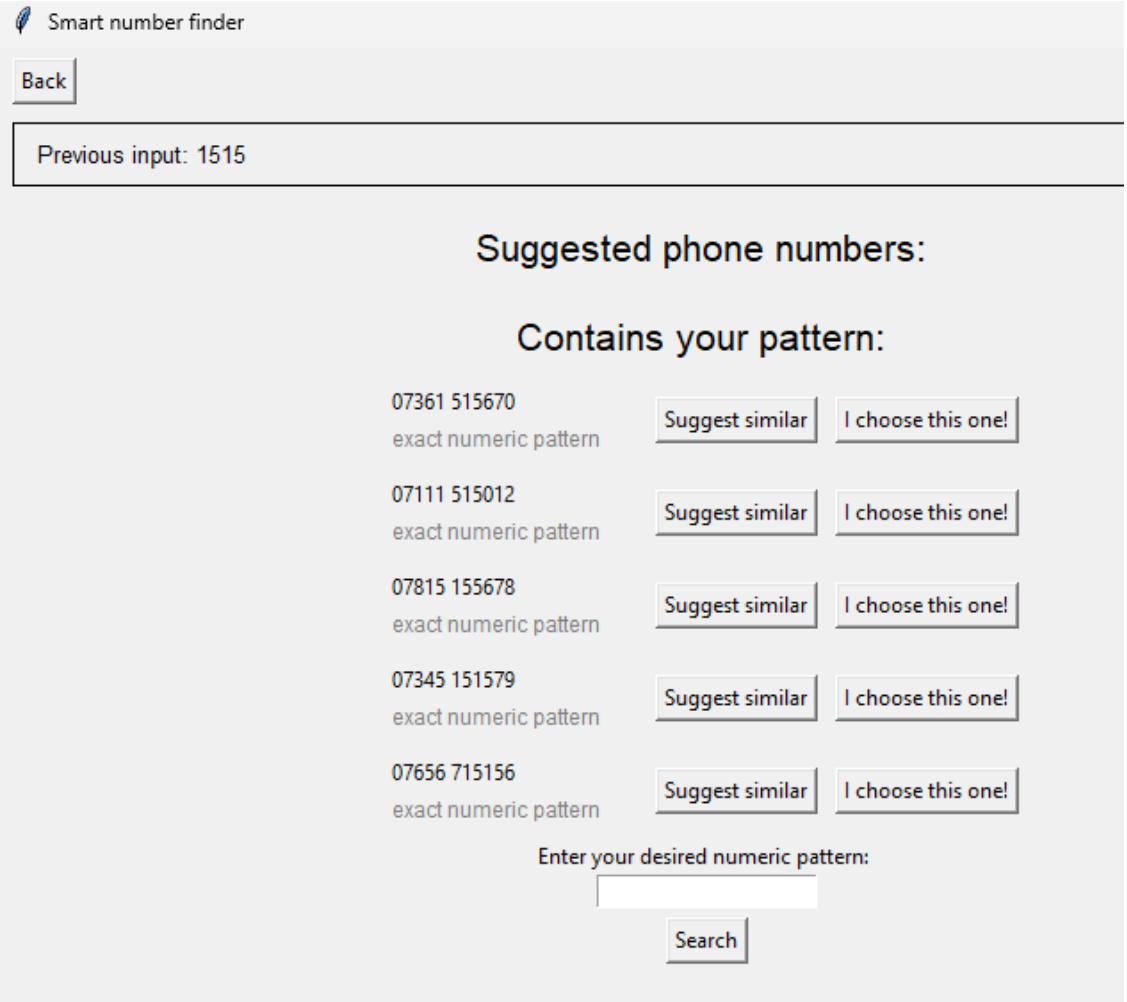

Figure 39: T40 outcome.

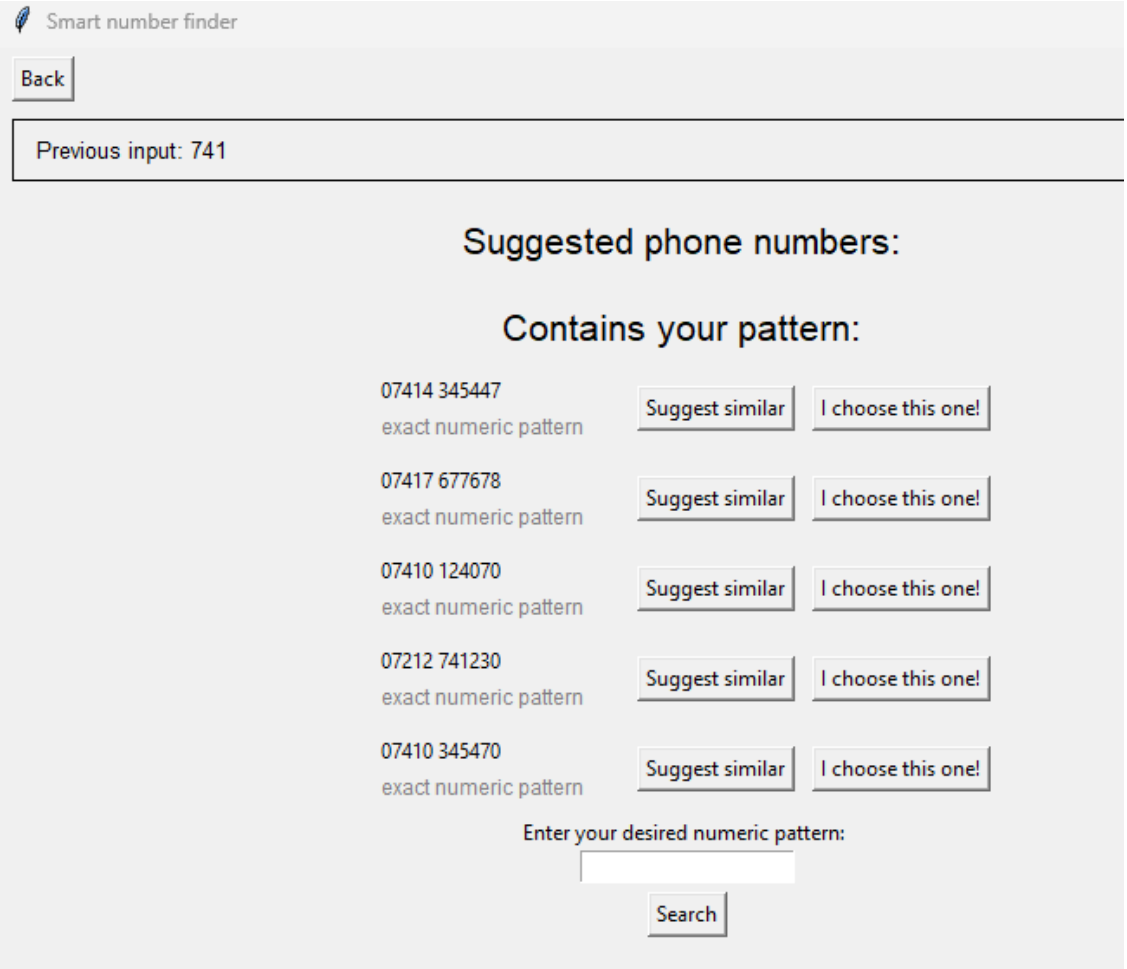

Figure 40: T41 outcome.

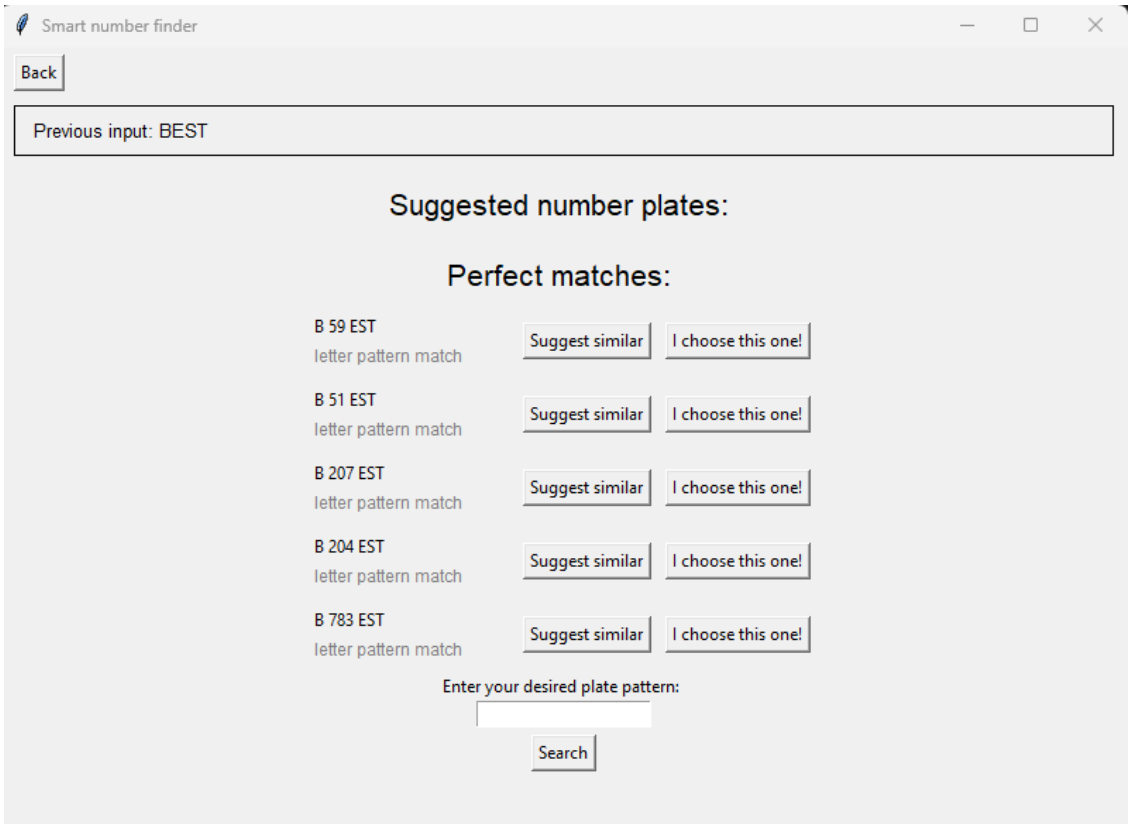

Figure 41: T42 outcome.

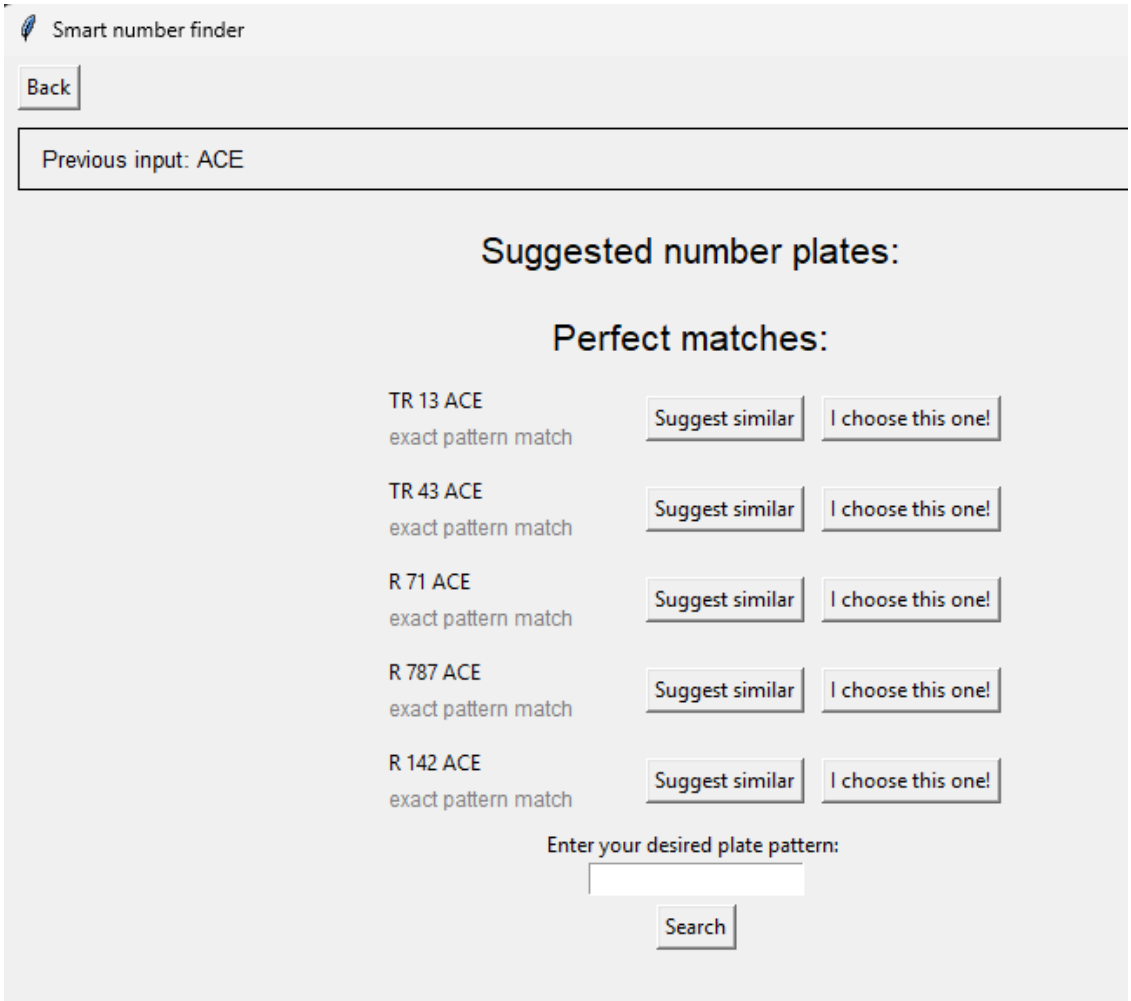

Figure 42: T43 outcome.

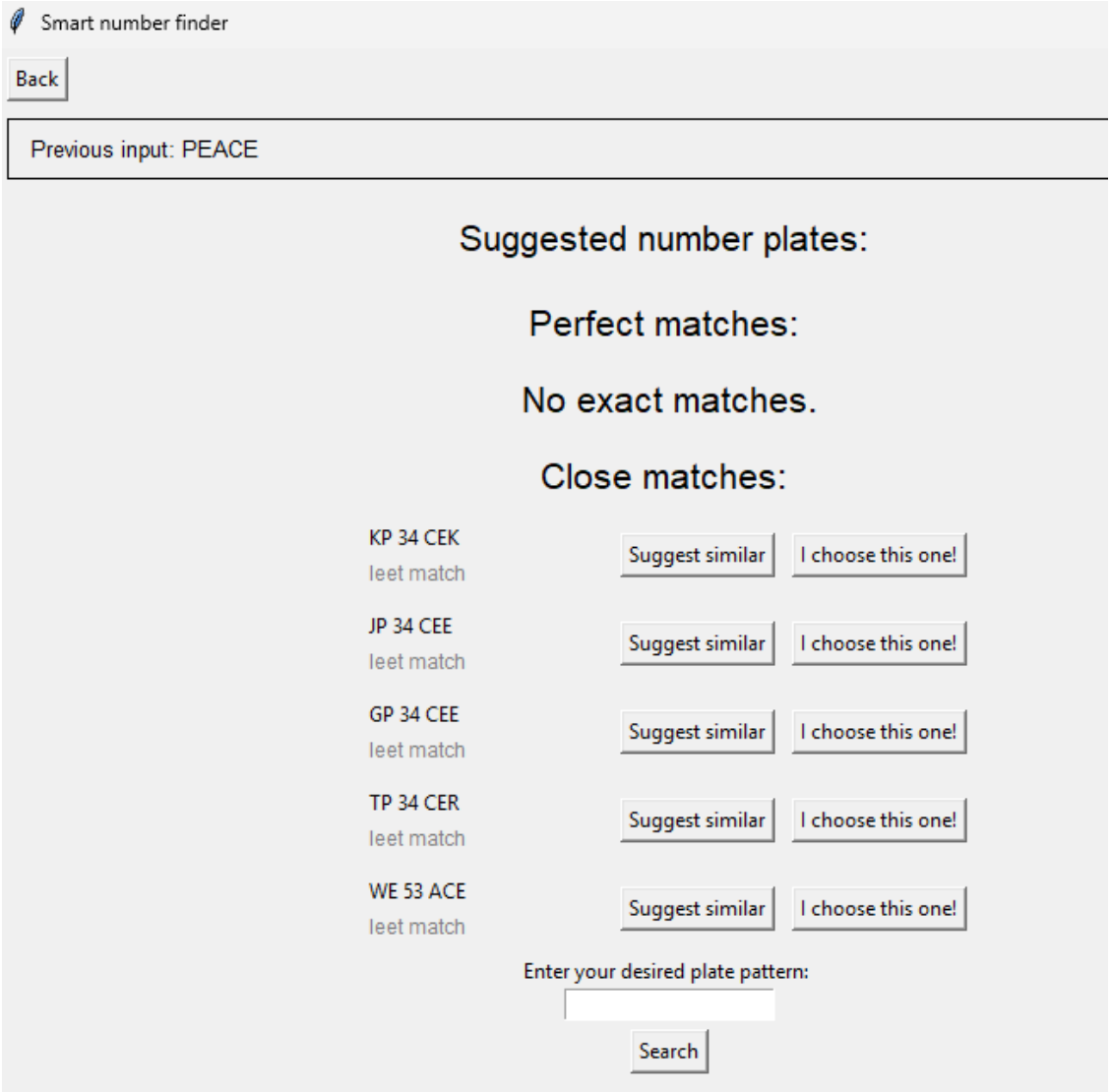

Figure 43: T44 outcome.

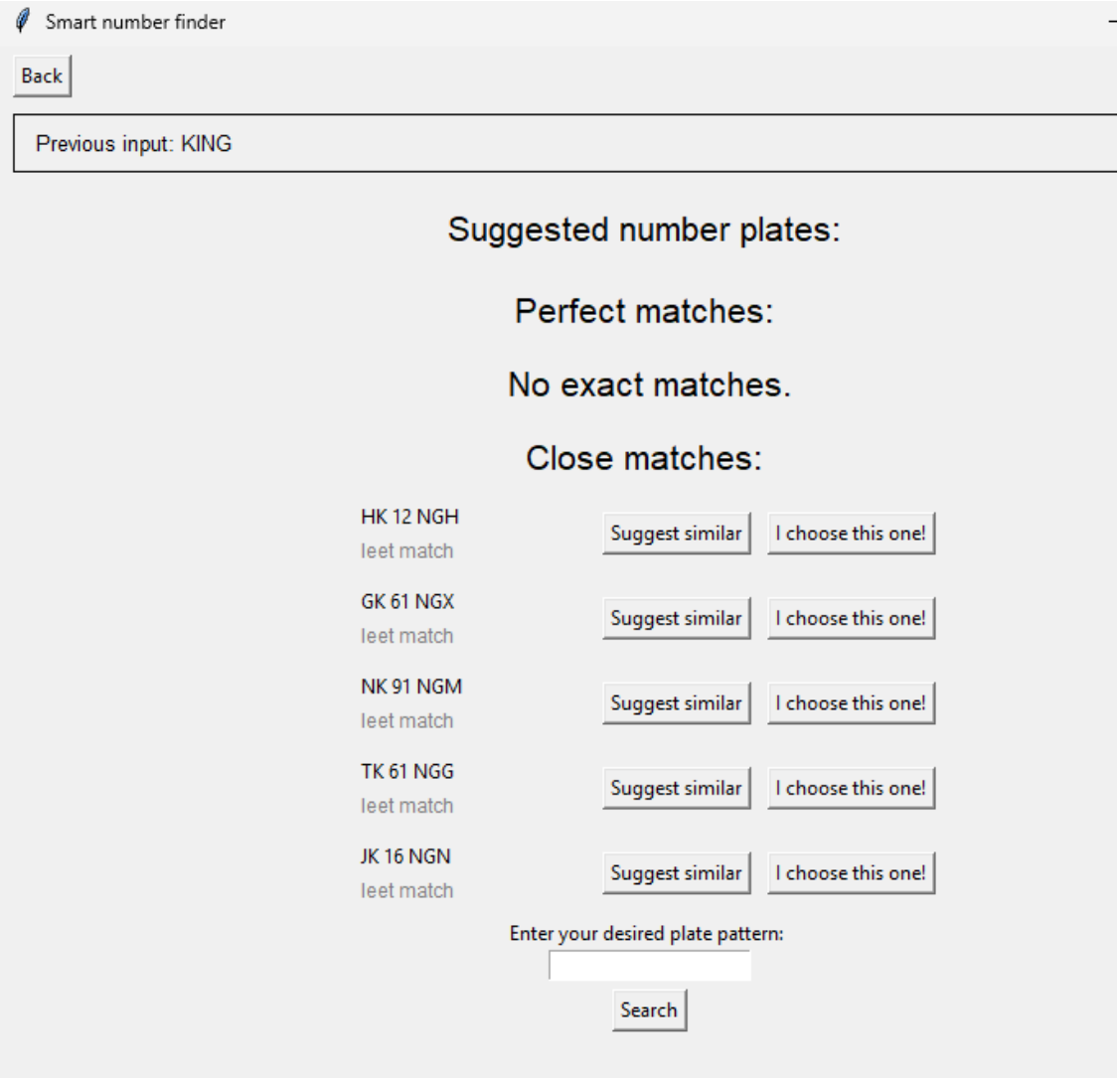

Figure 44: T45 outcome.

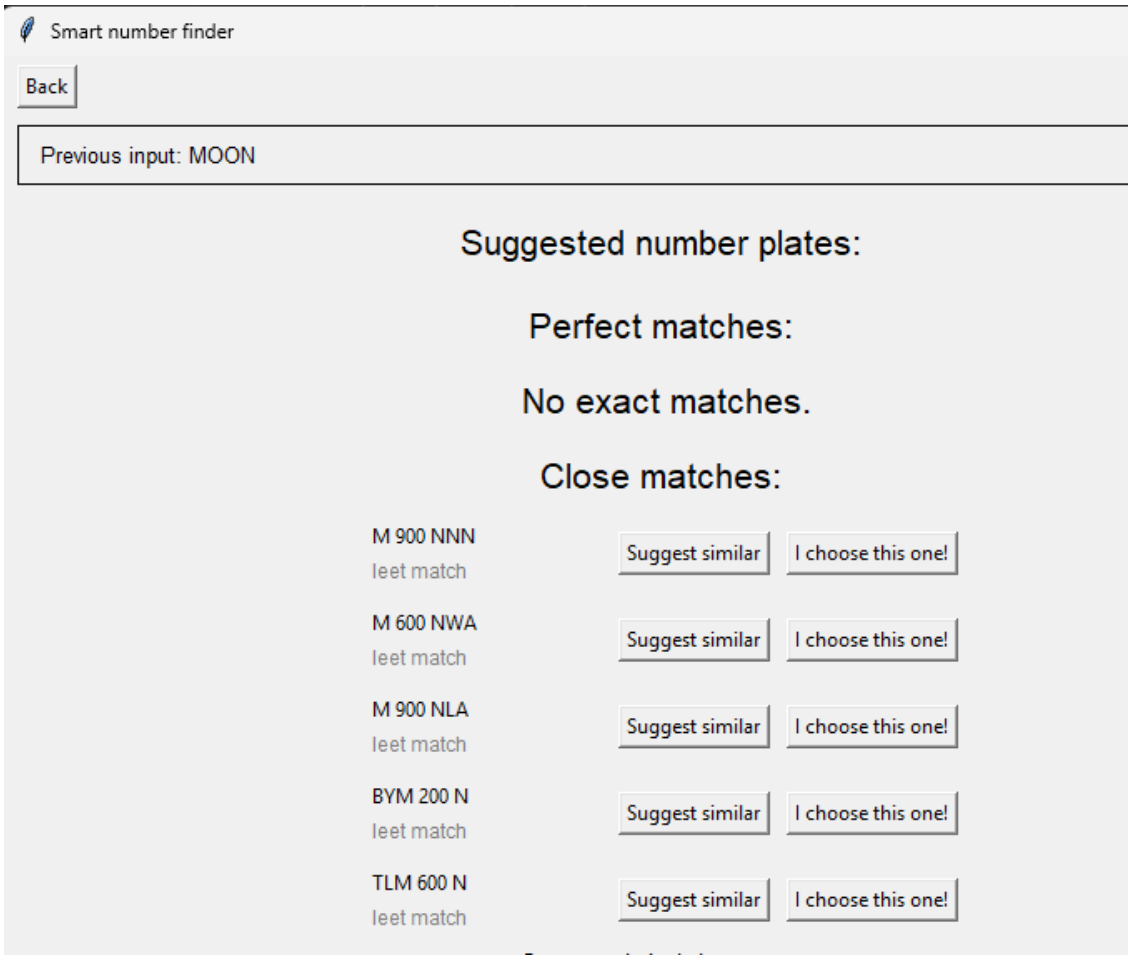

Figure 45: T46 outcome.

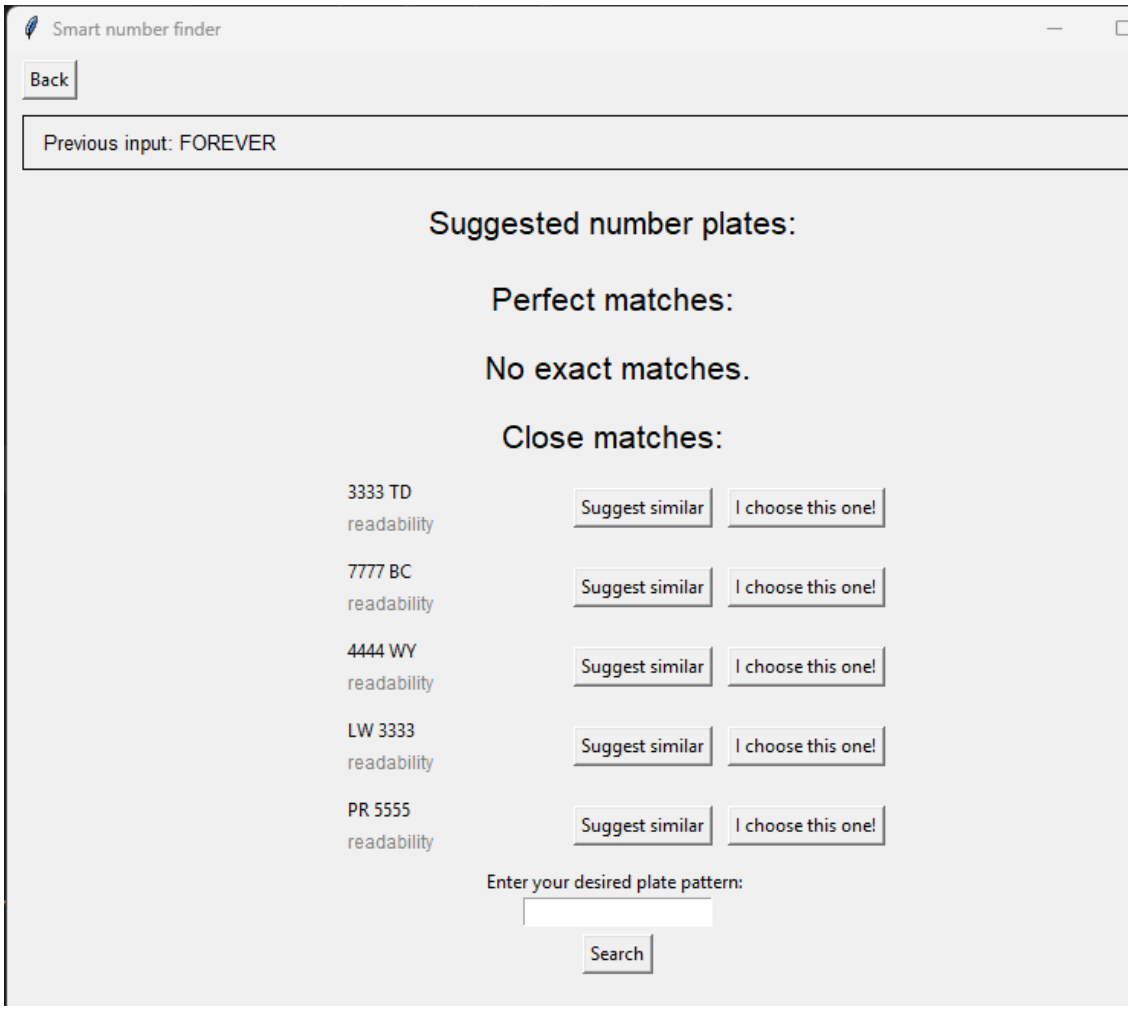

Figure 46: T47 outcome.

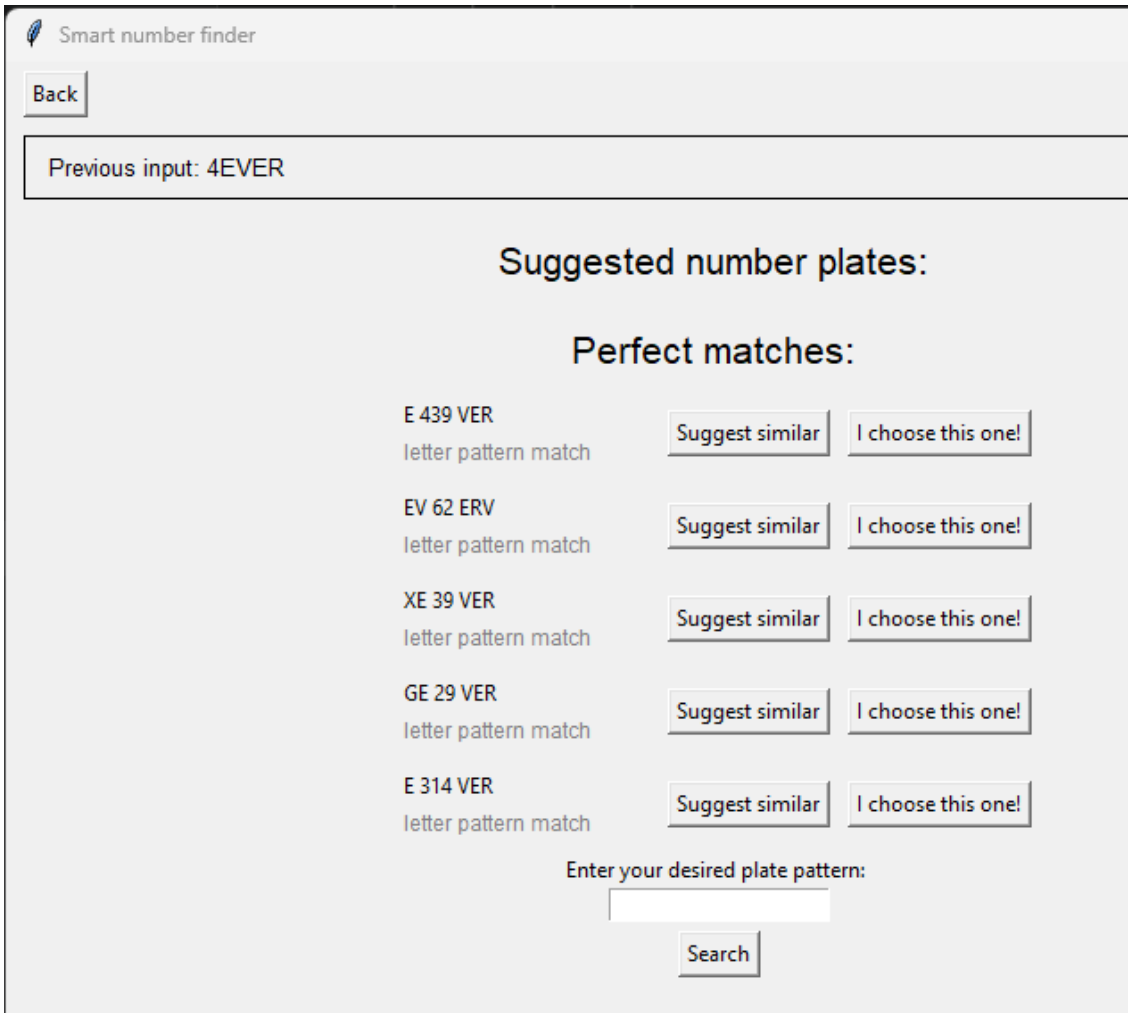

Figure 47: T48 outcome.

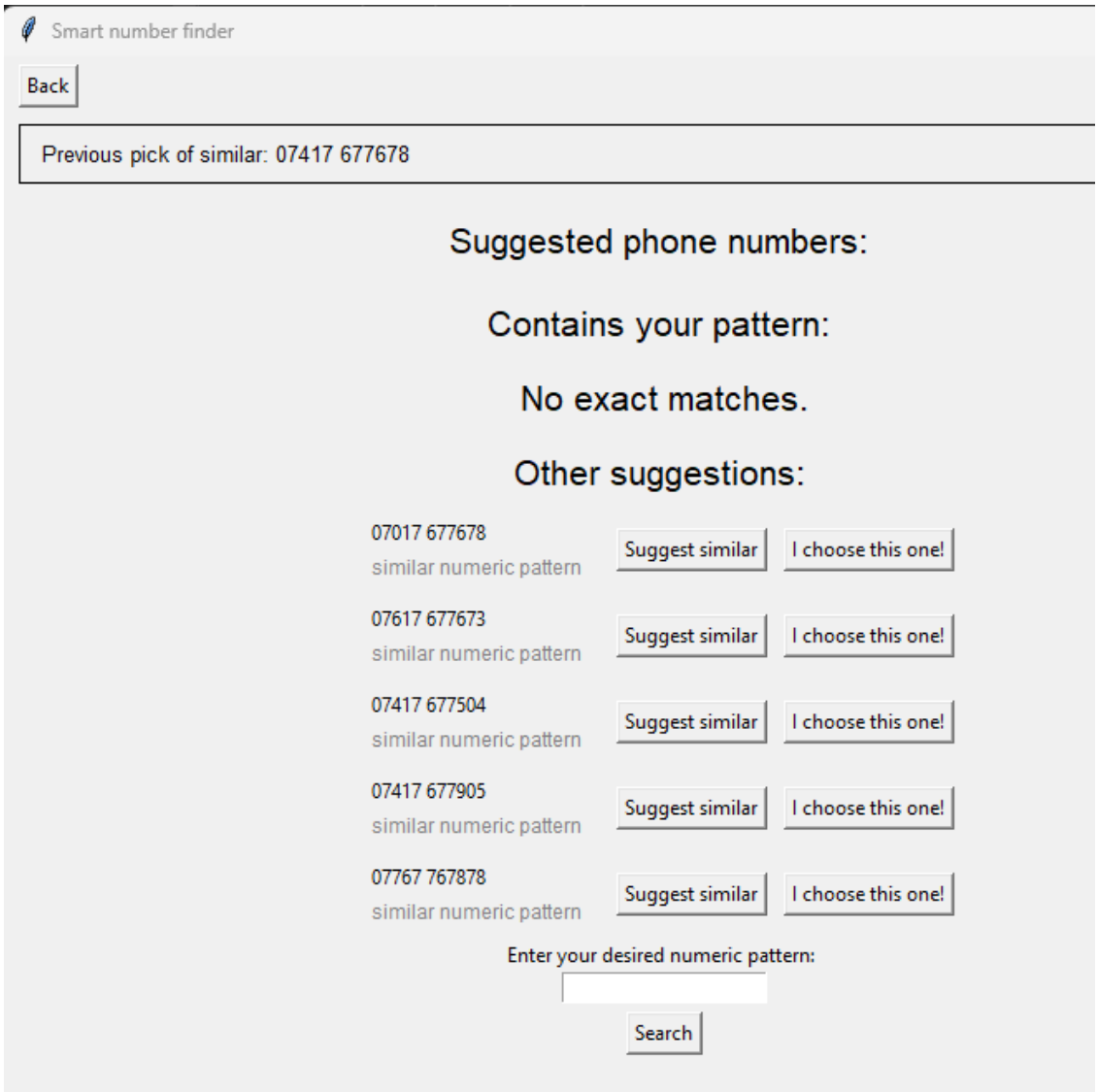

Figure 48: T49 outcome.

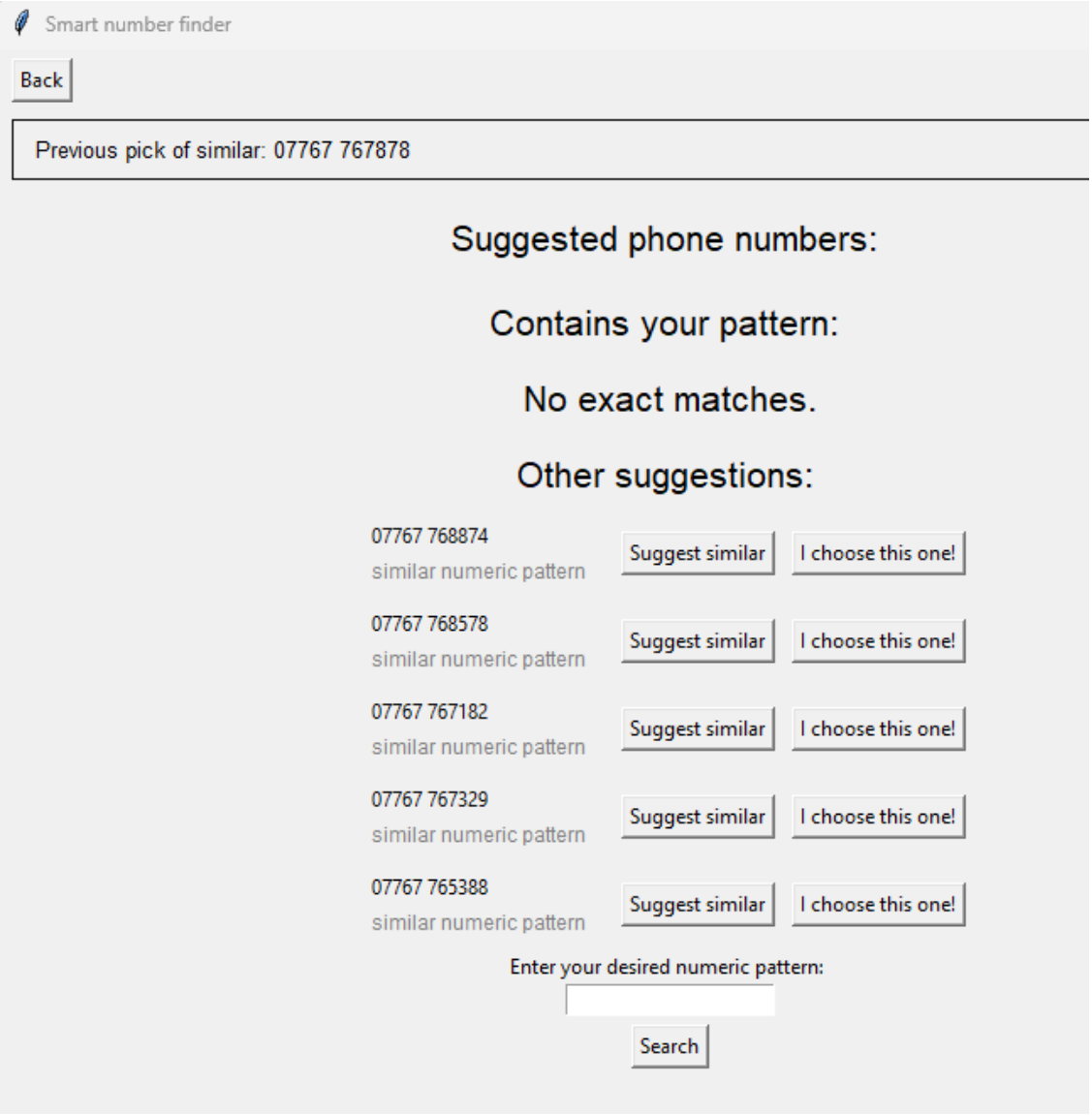

Figure 49: T50 outcome.

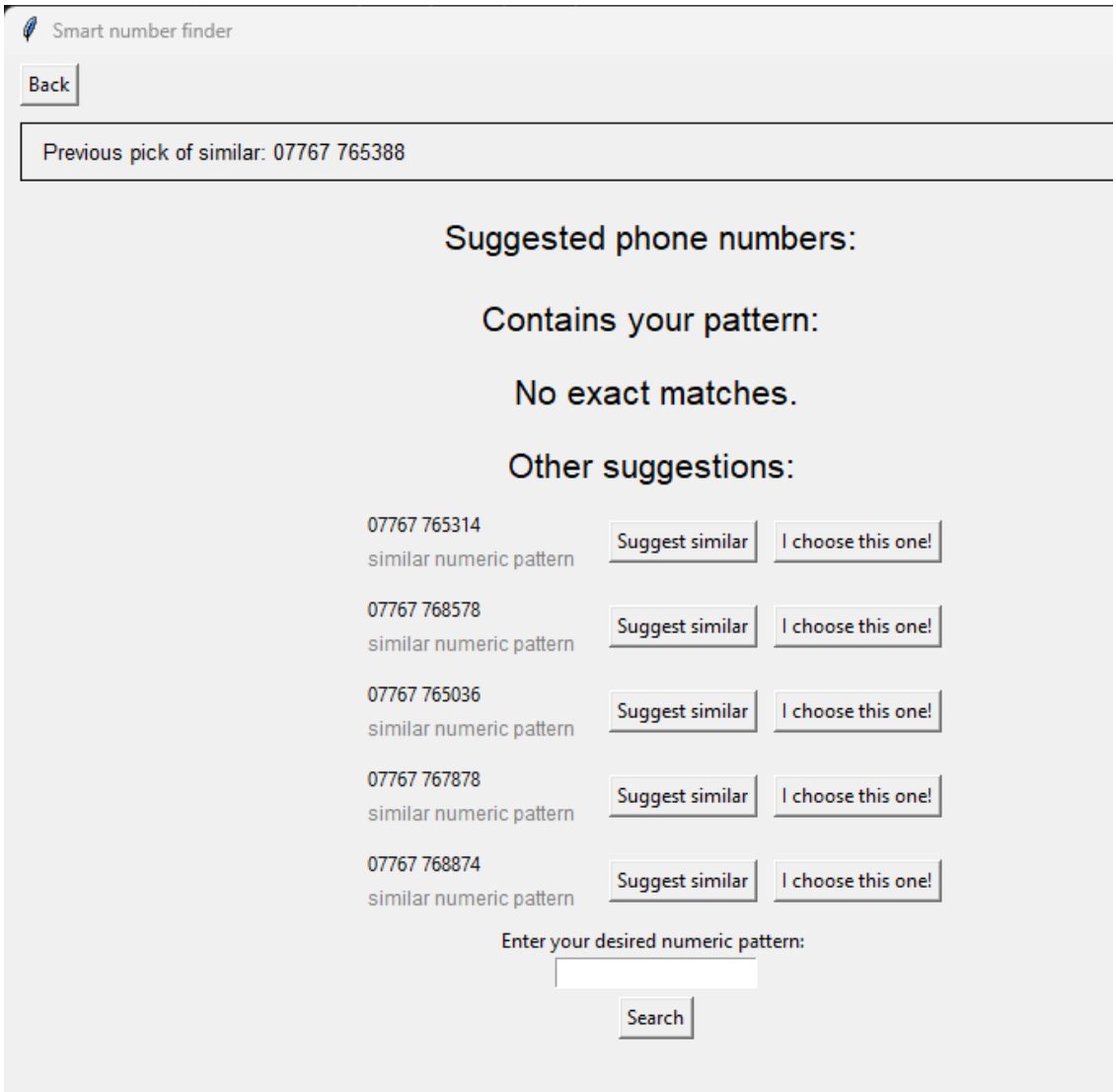

Figure 50: T51 outcome.

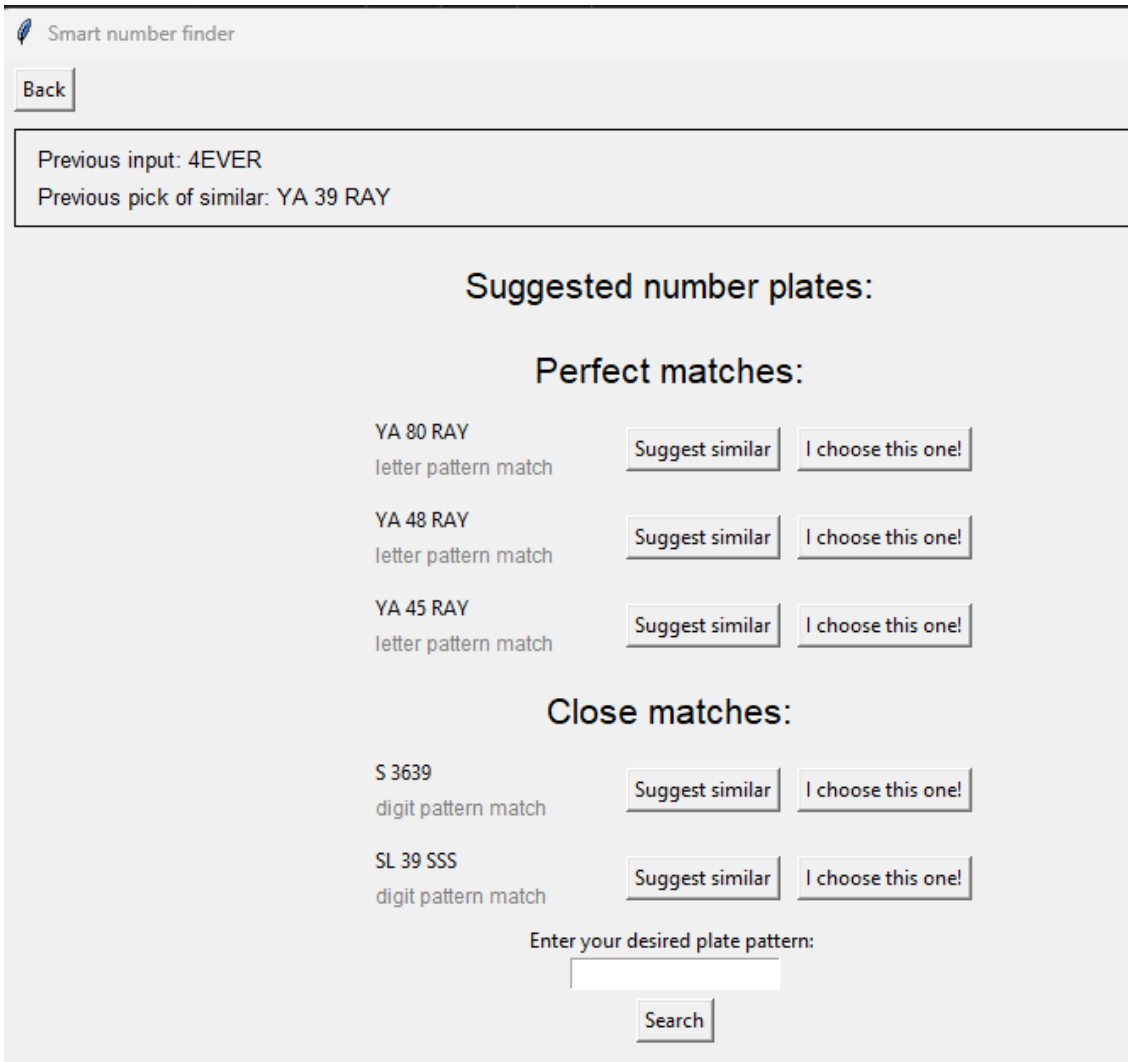

Figure 51: T52 outcome.

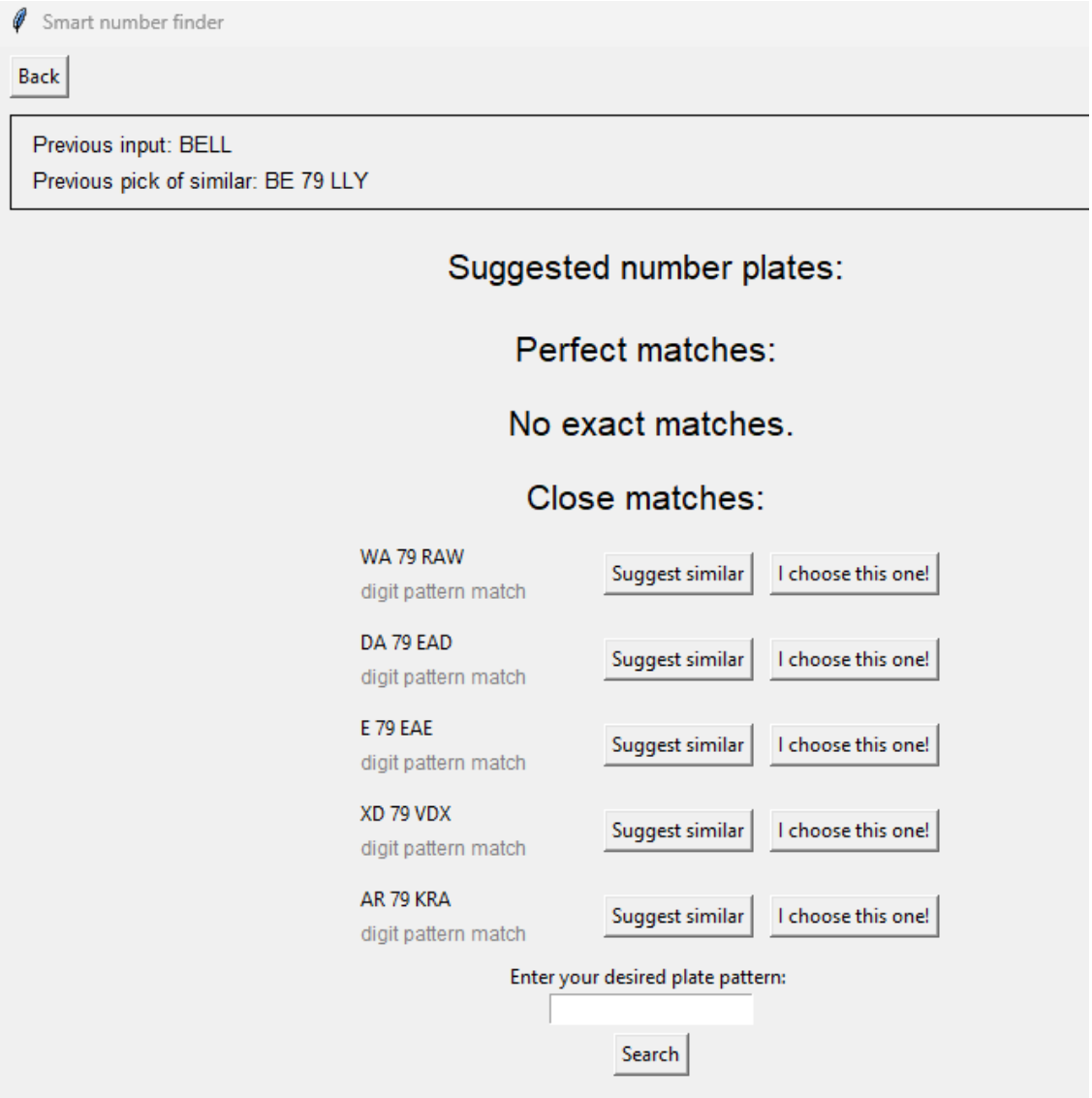

Figure 52: T53 outcome.

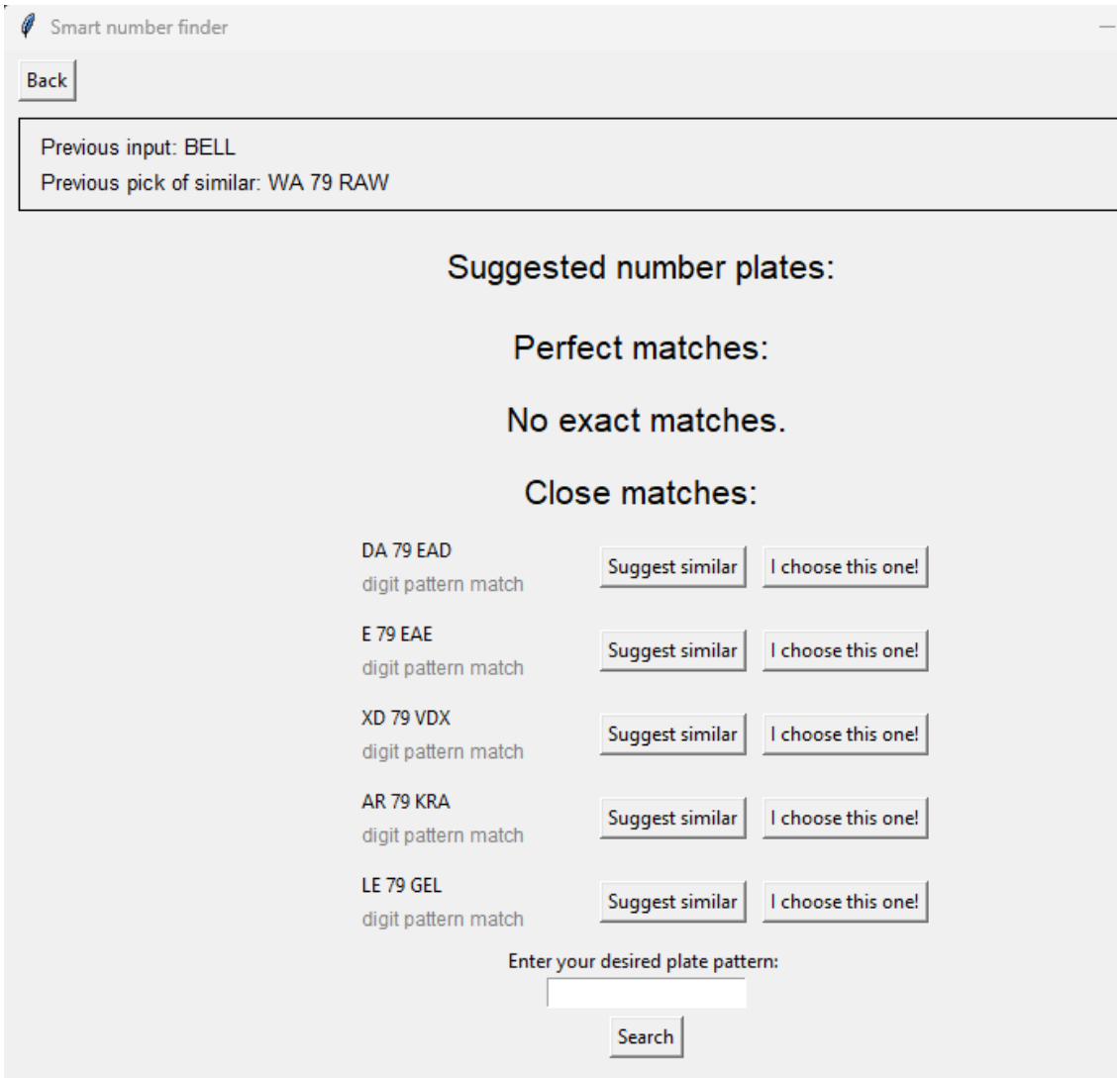

Figure 53: T54 outcome.

B. Generative AI

No generative AI was used in the making of this disertation.